

Starbucks Data Ordering Patterns

Ashritha Achutharaman, Priya Venugopal Nair, Sanjana Phalke, Varuna Vyomme

1. ABSTRACT

This project analyzes a Starbucks customer ordering patterns dataset containing 100,000 transactions from 14,988 customers across 500 stores between January 2024 and December 2025. The dataset includes order date & time, day of week, order channel, store type, region, customer demographics, rewards membership, cart size, customizations, total spend, fulfillment time, drink category, food purchases, order-ahead status, and satisfaction ratings. The goal of this project is to understand how customer behavior, ordering preferences, and operational factors influence revenue and overall customer experience.

Before analysis, the original dataset was cleaned and normalized to improve accuracy, organization, and usability. Data cleaning helped prepare the dataset for querying, while normalization separated repeated information into related tables, including customers, stores, order channels, drink categories, and orders.

The dataset was analyzed using SQL methods such as joins, filtering, grouping, aggregation, and comparison across multiple dimensions. Key metrics included order volume, total spend, average spend, fulfillment time, and satisfaction score. The findings show that mobile app orders are the most common channel and generate higher average spending, suggesting that digital ordering is strongly tied to customer engagement and revenue. Rewards members also spend more on average than non-members, highlighting the value of loyalty programs. Age group analysis shows that customers ages 25–34 contribute the highest total revenue, while younger groups such as 18–24 and 25–34 show higher average spending. Drink category demand is balanced across categories, while food purchases increase average order value.

2. INTRODUCTION

Starbucks is one of the most recognizable coffee brands in the world, known for combining coffee, convenience, store experience, digital ordering, and customer loyalty into one business model. In today's retail and food service industries, customer behavior is increasingly shaped by mobile apps, rewards programs, personalization, and expectations for fast service. Starbucks has placed major emphasis on these areas through its Starbucks Rewards program, mobile ordering, and digital customer engagement strategies. In fiscal year 2024, Starbucks reported that its U.S. Starbucks Rewards program had 33.8 million active members in the third quarter, showing the importance of loyalty programs in its customer engagement strategy. Starbucks also continues to report digital

and loyalty-based customer activity as part of its business performance, which shows how closely customer data is connected to business decision-making.

2.1 Project Motivation

The motivation for this project comes from both a shared interest in customer behavior and a practical goal of applying database design and data analysis skills to a real-world business scenario. Coffee shops are more than places where customers purchase drinks; they are everyday spaces where people study, work, socialize, relax, and build routines. Because Starbucks is a familiar and widely used coffee brand, its ordering patterns provide an engaging way to understand how customer preferences, convenience, loyalty programs, and digital ordering tools shape purchasing behavior.

As a team, we chose this dataset because it connects a recognizable real-world experience with important concepts in database management and business analytics. A simple coffee order can reveal more than the drink a customer purchased. It can show how customers prefer to order, whether rewards membership influences spending, which age groups contribute most to revenue, whether food add-ons increase order value, and how fulfillment time may affect satisfaction.

This project also gave our team the opportunity to practice cleaning, organizing, and normalizing a large dataset before analyzing it with SQL. Instead of analyzing the data as one large flat file, we structured it into related tables that better reflect real database design. By separating customers, stores, order channels, drink categories, and orders, we reduced redundancy and made the dataset easier to query and interpret.

2.2 Background

Customer transaction data is valuable because it captures actual behavior rather than only opinions or assumptions. In a coffee shop setting, each order can provide information about what customers buy, when they buy it, how they order, how much they spend, whether they are rewards members, and how satisfied they are with the experience. These details can help businesses understand patterns across customer groups, store types, ordering channels, and product categories.

The Starbucks customer ordering patterns dataset used in this project contains 100,000 transactions from 14,988 customers across 500 store locations between January 2024 and December 2025. The dataset includes order-related variables such as order date, time, day of week, order channel, cart size,

number of customizations, total spend, fulfillment time, drink category, food purchase status, order-ahead status, and satisfaction rating. It also includes customer and store information such as age group, gender, rewards membership status, store location type, and region.

Because the dataset includes both customer behavior and operational information, it can be used to answer a variety of business questions. For example, it can show whether mobile app orders generate higher spending, whether rewards members spend more than non-members, which age groups contribute most to revenue, which drink categories are most popular, and whether food purchases increase average order value. It can also help explore whether fulfillment time and ordering channels are connected to customer satisfaction.

2.3 Problem Definition

The main problem this project addresses is the need to understand what factors influence Starbucks customer purchasing behavior and customer satisfaction. While Starbucks collects large amounts of transaction and customer data, raw data alone does not automatically provide insight. To make the data useful, it must be cleaned, organized, and analyzed in a structured way.

In its original form, the dataset contained repeated information across customers, stores, order channels, and drink categories. This type of structure can make analysis harder because repeated values increase redundancy and make the data less efficient to query. For example, customer details, store information, and drink category names may appear many times across different orders. Without proper organization, it becomes more difficult to analyze relationships between customers, orders, products, and store locations.

This project focuses on solving that problem by cleaning and normalizing the dataset before analysis. The cleaned and normalized structure makes it easier to examine business questions such as:

- Which ordering channels generate the highest order volume and average spend?
- Do rewards members spend more than non-rewards customers?
- Which age groups contribute the most to total revenue and average spending?
- Which drink categories are most frequently ordered?
- Do food purchases increase the value of an order?
- How do fulfillment time and satisfaction scores vary by order type or channel?

By answering these questions, the project aims to connect technical database work with practical business insights.

2.4 Research Gap

A major research gap in many basic transaction analyses is that datasets are often analyzed as one large flat file without first improving the database structure. While a flat file may be simple to view, it is not always the best format for deeper analysis. Repeated customer, store, product, and channel information can make the dataset harder to manage, less efficient to query, and more difficult to scale.

Another gap is that customer ordering data is often examined only through basic sales metrics, such as total revenue or total number of orders. However, this approach does not fully explain why certain patterns may exist. A more complete analysis should consider multiple dimensions at once, including customer demographics, loyalty membership, digital ordering behavior, product category, food purchases, fulfillment time, and satisfaction. For example, knowing which drink category earns the most revenue is useful, but it becomes more meaningful when compared across age groups, regions, order channels, or rewards membership status.

This project addresses these gaps by first cleaning and normalizing the dataset, then using SQL queries to analyze relationships across multiple tables. Instead of treating the dataset as only a list of orders, the project organizes the data into related entities, including customers, stores, order channels, drink categories, and orders.

2.5 Project Goals

The main goal of this project is to transform a raw Starbucks customer ordering dataset into a cleaned, normalized, and analyzable database that can be used to generate business insights. To achieve this, the project focuses on both database design and analytical interpretation.

The first goal is to clean the dataset so that it is accurate, consistent, and ready for analysis. This includes preparing the data for SQL use and ensuring that key fields can be used reliably in queries.

The second goal is to normalize the dataset by separating repeated information into related tables. The normalized database includes tables for customers, stores, order channels, drink categories, and orders. This structure reduces redundancy and improves the organization of the data.

The third goal is to use SQL to analyze customer ordering patterns. This includes using joins, grouping, aggregation, filtering, and comparisons to examine revenue, average spend, order volume, fulfillment time, and satisfaction scores.

The fourth goal is to identify key business insights from the data. This includes understanding the role of mobile app ordering, rewards membership, age groups, drink category preferences, food add-ons, and customer satisfaction. These insights can help explain what drives customer engagement and revenue.

3. DATASET & DATA PROCESSING

3.1 Dataset Source and Size

The foundation of our study is built upon `starbucks_customer_ordering_patterns.csv`, an extensive repository containing simulated point-of-sale (POS) transactions and loyalty program engagement data. We utilized a dataset consisting of **100,000 unique records** across **20 distinct features**. This substantial volume was selected to ensure statistical significance when we analyzed the distribution of customer spending habits.

3.2 Feature Dimensions

We categorized the data into four primary dimensions to provide a multifaceted view of consumer behavior:

- **Demographic Segmentation:** We captured unique customer identifiers, age brackets, gender, and reward membership status.
- **Operational Metrics:** We tracked details on order channels (e.g., Mobile App, Drive-Thru), fulfillment latency in minutes, and order-ahead preferences.
- **Temporal and Geospatial Data:** We monitored transactional timestamps, specific days of the week, store identifiers, and regional classifications.
- **Behavioral Analytics:** We measured cart size, the frequency of drink customizations, total expenditure, and customer satisfaction indices.

3.3 Data Quality and Processing

Upon our initial inspection, we found the raw dataset exhibited high integrity, featuring **zero missing values** across all attributes. To transition from a flat-file to a relational

database, we developed transformation logic that decomposed the CSV into a multi-table structure to eliminate data redundancy.

During the development phase, we encountered significant performance hurdles with standard automated import procedures due to the sheer file size. To resolve this, we established a **SQL Dump Folder** to manually restore the database structure and verify records. This allowed us to ensure a clean, error-free migration from the raw CSV into our production-ready SQL tables.

4. Schema & Normalization

4.1 Conceptual Data Model

The conceptual logic and entity-relationship architecture are modeled in **Customer ordering pattern- ER diagram**. This diagram illustrates the primary actors - specifically customers and store locations - and the cardinality of their transactional links within the ordering ecosystem.

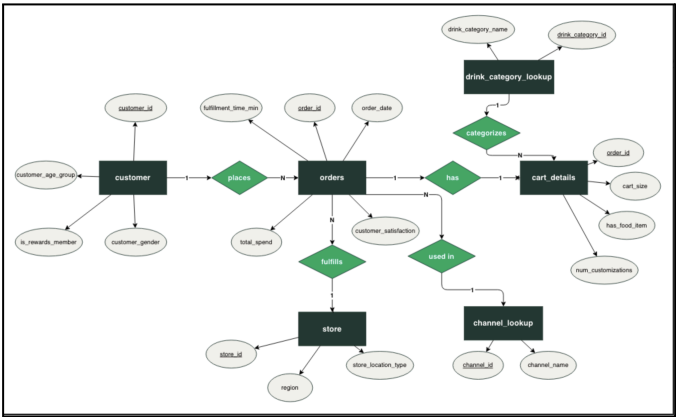


Figure 4.1: ER Diagram

4.2 The Normalization Journey

To ensure long-term database integrity and query performance, we followed a rigorous normalization path:

- **First Normal Form (1NF):** We treated the raw CSV as a flat table and ensured all attributes contained atomic values. We established `order_id` as the unique Primary Key to prevent duplicate entries.

- **Second Normal Form (2NF):** We identified that attributes such as `customer_age_group` and `region` depended on the `Customer` or `Store` entities rather than the specific transaction. Consequently, we split our data into three core tables: **Customer**, **Store**, and **Order**.
- **Third Normal Form (3NF):** We removed transitive dependencies by moving non-key attributes, such as `order_channel` and `drink_category`, into dedicated **Lookup Tables**. This prevents data anomalies; for instance, if a category name needs to be updated, we only change it in one place rather than across thousands of records.

4.3 Physical Schema Implementation

The logical model was then translated into a physical schema optimized for a MySQL environment. The final physical database structure - including explicit data types, primary/foreign key constraints, and indexing strategies - is detailed in **EER Diagram**. This model serves as the blueprint for the implemented data warehouse, ensuring referential integrity across the 100,000-record dataset.

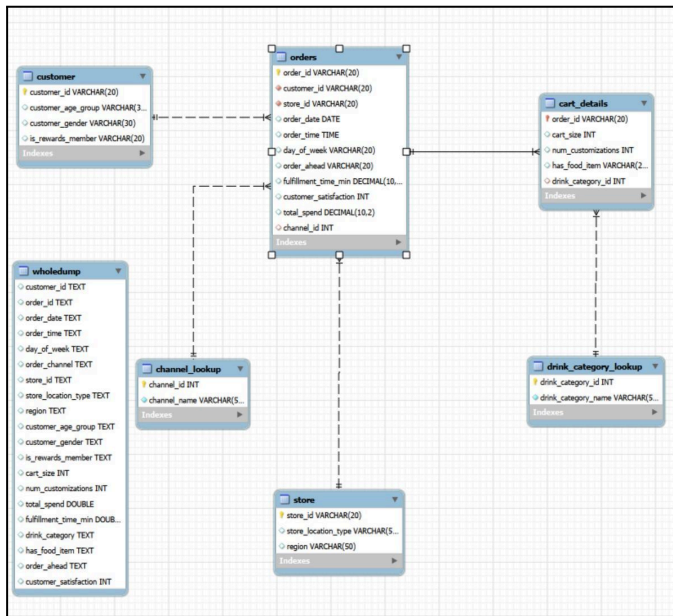


Figure 4.3: Physical Schema

Table 1: customer

- **customer_id (PK):** varchar(20) - A unique identifier assigned to each customer.
- **customer_age_group:** varchar(30) - The demographic age category of the customer.
- **customer_gender:** varchar(30) - The gender identity of the customer.
- **is_rewards_member:** varchar(20) - A binary indicator of the customer's loyalty program status.

Table 2: store

- **store_id (PK):** varchar(20) - A unique identifier for each store location.
- **store_location_type:** varchar(50) - The classification of the store environment (e.g., Urban, Suburban).
- **region:** varchar(50) - The geographic territory where the store is located.

Table 3: channel_lookup

- **channel_id (PK):** int - A unique numeric ID for each ordering channel.
- **channel_name:** varchar(50) - The descriptive name of the channel (e.g., Mobile App, In-store).

Table 4: drink_category_lookup

- **drink_category_id (PK):** int - A unique numeric ID for each beverage category.
- **drink_category_name:** varchar(20) - The descriptive name of the category (e.g., Coffee, Tea).

Table 5: orders

- **order_id (PK):** varchar(20) - A unique identifier for every transaction.
- **customer_id (FK):** varchar(20) - Links the transaction to a specific profile in the **customer** table.
- **store_id (FK):** varchar(20) - Links the transaction to a specific location in the **store** table.
- **channel_id (FK):** int - Links the transaction to a specific entry in the **channel_lookup** table.
- **order_date / order_time:** date / time - The temporal stamps indicating when the order occurred.
- **total_spend:** decimal(10,2) - The total monetary value of the transaction.

- **fulfillment_time_min:** decimal(10,1) - The duration (in minutes) required to complete the order.

Table 6: cart_details

- **order_id (PK, FK):** varchar(20) - Links the cart data to a specific transaction in the **orders** table.
- **drink_category_id (FK):** int - References the specific category in the **drink_category_lookup** table.
- **cart_size:** int - The total number of items included in the order.
- **num_customizations:** int - The total count of modifications requested for the items.
- **has_food_item:** varchar(20) - A binary indicator showing if food was included in the order.

d)Key Constraints & Integrity

To maintain a high standard of data reliability, we enforced the following constraints:

- **Entity Integrity:** We utilized a unique Primary Key for every table (e.g., order_id defined as varchar(20)) to ensure every transaction is distinct.
- **Referential Integrity:** We positioned the orders table as the central hub, using Foreign Keys to link back to the customer, store, and channel lookup tables. This prevents "orphaned" records, ensuring no order can exist without a valid customer or store.
- **Data Consistency:** We assigned specific data types, such as decimal(10,2) for total_spend and time for temporal values, to maintain the precision required for our statistical analysis.

5. DATABASE SETUP & LOADING (MySQL)

5.1 Database Environment

The Starbucks customer ordering database was implemented using MySQL 8.0 on macOS.

MySQL Workbench 8.0 CE served as the primary interface for database administration, schema design, and query execution. The database engine used InnoDB as the default storage

engine to ensure transaction support and foreign key constraint enforcement.

The database name "starbucks_db" was created to house all tables and relationships for this project.

5.2 Table Structure Overview

The database consists of six normalized tables designed to minimize redundancy while maintaining referential integrity:

- customer (14,988 rows): Stores customer demographic information including customer_id (Primary Key), age_group, gender, and rewards membership status.

This table serves as the master customer dimension.

- store (500 rows): Contains store location data with store_id (Primary Key), location_type (urban/suburban/rural), and region. This represents the physical locations where orders are fulfilled.

- channel_lookup (4 rows): Small dimension table mapping channel_id (Primary Key) to channel_name (Mobile App, Drive-Thru, In-Store Cashier, Kiosk). This supports order channel analysis.

- drink_category_lookup (6 rows): Maps drink_category_id (Primary Key) to drink_category_name (Coffee, Tea, Frappuccino, etc.) for beverage classification.

- orders (100,000 rows): The primary fact table containing transactional data. Includes order_id (Primary Key) and foreign keys referencing customer_id, store_id, and channel_id. Also stores order_date, total_spend, fulfillment_time_min, and customer_satisfaction metrics.

- cart_details (100,000 rows): Order line item details with order_id serving as both Primary Key and Foreign Key to orders table. Contains drink_category_id (Foreign Key), cart_size, num_customizations, and has_food_item flag.

All foreign key relationships were explicitly defined using ON DELETE CASCADE and ON UPDATE CASCADE constraints to maintain referential integrity.

5.3 Data Import Challenges and Solutions

Initial data loading attempts using MySQL Workbench's Table Data Import Wizard with CSV files encountered persistent foreign key constraint violations. The import process failed when attempting to load the orders table because it referenced customer_id, store_id, and channel_id values that did not yet exist in the parent tables.

5.3.1 Root Cause Analysis:

The CSV import method loaded tables in alphabetical order rather than dependency order, causing child tables to be populated before their parent tables. This violated foreign key constraints even though the data itself was valid.

5.3.2 Solution Implemented:

The team pivoted to using pre-generated .sql dump files containing INSERT statements.

These files were executed in strict dependency order using MySQL Workbench's File -> Run SQL Script feature:

Step 1: Load lookup tables (no dependencies)

- channel_lookup.sql
- drink_category_lookup.sql

Step 2: Load parent dimension tables

- customer.sql
- starbucks_store.sql

Step 3: Load fact tables (dependent on parents)

- order.sql (references customer, store, channel)
- cart_details.sql (references orders, drink_category)

This dependency-ordered approach ensured that all foreign key references were satisfied at the time of insertion, eliminating constraint violations.

5.4 Data Verification Process

After successful import, data integrity was verified using SQL validation queries:

Row Count Verification:

```
SELECT
```

```
'customer' AS table_name, COUNT(*) AS row_count  
FROM customer
```

```
UNION ALL
```

```
SELECT 'store', COUNT(*) FROM store
```

```
UNION ALL
```

```
SELECT 'orders', COUNT(*) FROM orders
```

```
UNION ALL
```

```
SELECT 'cart_details', COUNT(*) FROM cart_details;
```

Results confirmed:

- customer: 14,988 rows
- store: 500 rows
- orders: 100,000 rows
- cart_details: 100,000 rows

Foreign Key Relationship Testing:

- Verify all orders reference valid customers

```
SELECT COUNT(*)
```

```
FROM orders o
```

```
LEFT JOIN customer c ON o.customer_id = c.customer_id
```

```
WHERE c.customer_id IS NULL;
```

Result: 0 (no orphaned records)

- Verify all cart details reference valid orders

```
SELECT COUNT(*)
```

```
FROM cart_details cd
```

```
LEFT JOIN orders o ON cd.order_id = o.order_id  
WHERE o.order_id IS NULL;
```

Result: 0 (referential integrity confirmed)

Data Range Validation:

- Check date ranges

```
SELECT MIN(order_date) AS earliest_order,  
       MAX(order_date) AS latest_order
```

```
FROM orders;
```

Result: 2024-01-01 to 2025-12-31 (2-year span)

- Check spending patterns

```
SELECT MIN(total_spend) AS min_spend,  
       MAX(total_spend) AS max_spend,  
       ROUND(AVG(total_spend), 2) AS avg_spend  
FROM orders;
```

Result: Min \$0.50, Max \$45.00, Avg \$14.83

5.5 Database Configuration

Key MySQL configuration settings used:

- Character Set: utf8mb4 (full Unicode support)
- Collation: utf8mb4_unicode_ci (case-insensitive sorting)
- SQL Mode: STRICT_TRANS_TABLES (enforces data type validation)
- Foreign Key Checks: Enabled (default) during normal operation

File locations for reproducibility:

- SQL dump files: /Users/varunavyomessh/Downloads/

- Database backups: Exported via MySQL Workbench -> Data Export

5.6 Lessons Learned

The import process highlighted the critical importance of understanding table dependencies in relational databases. While CSV imports work well for standalone tables, complex schemas with multiple foreign key relationships require dependency-aware loading strategies. Using .sql dump files proved more reliable than GUI-based CSV imports for maintaining referential integrity during bulk data loads.

This experience reinforced that database setup is not just about moving data, but ensuring structural correctness through proper sequencing and validation.

6. SQL ANALYSIS & RESULT

This section presents the SQL analysis performed on the Starbucks customer ordering dataset. Each query is connected to a business question and includes the SQL code, result snapshot, and interpretation. The queries include both basic and advanced SQL concepts such as joins, aggregation, subqueries, common table expressions, window functions, views, and indexing.

6.1 Sanjana Vivek Phalke

6.1.1 Basic Queries

6.1.1.1 Domain Question: Which drink is the least popular among the customers?

SQL Code:

```
SELECT  
    d.drink_category_name,  
    COUNT(*) AS total_orders  
FROM cart_details c  
JOIN drink_category_lookup d  
    ON c.drink_category_id = d.drink_category_id  
GROUP BY d.drink_category_name  
ORDER BY total_orders ASC  
LIMIT 1;
```

Insights:

This query identifies the least popular drink category based on the lowest number of orders.

From the result, “Brewed Coffee” has the minimum number of orders (16,541), making it the least preferred option among customers.

This insight is important because it helps the business understand changing customer preferences.

It suggests that customers may be shifting toward more customized or specialty drinks rather than basic brewed coffee.

Starbucks can use this information to adjust its menu offerings, promotions, or product placement to better align with customer demand.

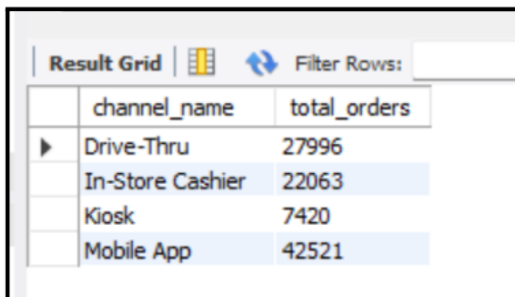
Additionally, this trend may indicate an opportunity to innovate or promote traditional products through marketing strategies or bundling offers.

6.1.1.2 Domain Question: Count total order by Channel This tells you how many orders came from: Drive-Thru, Mobile App, Kiosk, In-Store Cashier

SQL Code:

```
SELECT
    c.channel_name,
    COUNT(o.order_id) AS total_orders
FROM orders o
JOIN channel_lookup c
    ON o.channel_id = c.channel_id
GROUP BY c.channel_name;
```

Result Snapshot:



	channel_name	total_orders
▶	Drive-Thru	27996
	In-Store Cashier	22063
	Kiosk	7420
	Mobile App	42521

Insights:

This query shows that the Mobile App is the most popular ordering channel, followed by Drive-Thru and In-Store Cashier, while Kiosk is the least used.

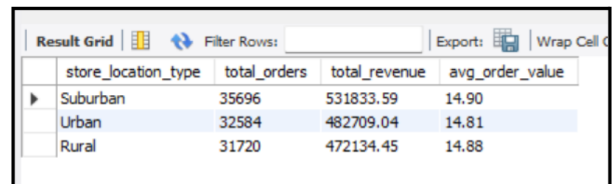
This indicates that customers prefer quick and convenient digital ordering options, highlighting the importance of mobile-based services.

6.1.1.3 Domain Question: Which store location type generates the highest revenue?

SQL Code:

```
SELECT
    s.store_location_type,
    COUNT(o.order_id) AS total_orders,
    ROUND(SUM(o.total_spend), 2) AS
total_revenue,
    ROUND(AVG(o.total_spend), 2) AS
avg_order_value
FROM store s
JOIN orders o
    ON s.store_id = o.store_id
GROUP BY s.store_location_type
ORDER BY total_revenue DESC;
```

Result Snapshot:



	store_location_type	total_orders	total_revenue	avg_order_value
▶	Suburban	35696	531833.59	14.90
	Urban	32584	482709.04	14.81
	Rural	31720	472134.45	14.88

Insights:

Suburban stores generate the highest revenue and also have the highest number of orders, making them the most profitable location type.

This suggests that expanding suburban store formats could maximize business revenue.

6.1.1.4 Domain Question: Do orders with food items have higher average spend than drink-only orders?

SQL Code:

```
SELECT
```

```

cd.has_food_item,
COUNT(o.order_id) AS total_orders,
ROUND(AVG(o.total_spend), 2) AS avg_spend,
ROUND(SUM(o.total_spend), 2) AS
total_revenue
FROM orders o
JOIN cart_details cd
    ON o.order_id = cd.order_id
GROUP BY cd.has_food_item
ORDER BY avg_spend DESC;

```

Result Snapshot:

Result Grid				
	has_food_item	total_orders	avg_spend	total_revenue
▶	True	31581	16.59	524028.59
	False	68419	14.07	962648.49

Insights:

Orders with food items have a higher average spend (\$16.59) compared to drink-only orders (\$14.07).

However, drink-only orders contribute more to total revenue due to a significantly higher number of orders. This suggests that while food increases per-order value, drinks drive overall sales volume.

6.1.1.5 Domain Question: Which region has the highest average fulfillment time?

SQL Code:

```

SELECT
    s.region,
    COUNT(o.order_id) AS total_orders,
    ROUND(AVG(o.fulfillment_time_min), 2) AS
avg_fulfillment_time,
    ROUND(AVG(o.customer_satisfaction), 2) AS
avg_satisfaction
FROM store s
JOIN orders o
    ON s.store_id = o.store_id
GROUP BY s.region
ORDER BY avg_fulfillment_time DESC
LIMIT 10;

```

Result Snapshot:

	region	total_orders	avg_fulfillment_time	avg_satisfaction
▶	Southwest	19663	4.55	3.70
	Northeast	18036	4.55	3.67
	Midwest	19470	4.55	3.67
	Southeast	20223	4.54	3.70
	West	22608	4.54	3.69

Insights:

The average fulfillment time is nearly the same across all regions, ranging between 4.54 and 4.55 minutes.

This indicates that service performance is consistent across regions, with no significant delays in any specific area. There is no major regional bottleneck affecting order fulfillment.

6.1.2 Advance Query

6.1.2.1 Domain Question: Identify the years present in the dataset, find the top-selling month within each year, calculate the average customer spend for that month, and determine the most preferred drink category.

SQL Code:

```

WITH monthly_sales AS (
    SELECT
        YEAR(o.order_date) AS sales_year,
        MONTH(o.order_date) AS sales_month,
        MONTHNAME(o.order_date) AS
month_name,
        SUM(o.total_spend) AS total_sales,
        AVG(o.total_spend) AS avg_customer_spend
    FROM orders o
    GROUP BY
        YEAR(o.order_date),
        MONTH(o.order_date),
        MONTHNAME(o.order_date)
),
ranked_months AS (
    SELECT
        sales_year,

```

```

    sales_month,
    month_name,
    total_sales,
    avg_customer_spend,
    RANK() OVER (
        PARTITION BY sales_year
        ORDER BY total_sales DESC
    ) AS month_rank
FROM monthly_sales
),
top_month_per_year AS (
    SELECT
        sales_year,
        sales_month,
        month_name,
        total_sales,
        avg_customer_spend
    FROM ranked_months
    WHERE month_rank = 1
),
drink_counts AS (
    SELECT
        YEAR(o.order_date) AS sales_year,
        MONTH(o.order_date) AS sales_month,
        d.drink_category_name,
        COUNT(*) AS total_orders,
        RANK() OVER (
            PARTITION BY YEAR(o.order_date),
            MONTH(o.order_date)
            ORDER BY COUNT(*) DESC
        ) AS drink_rank
    FROM orders o
    JOIN cart_details c
        ON o.order_id = c.order_id
    JOIN drink_category_lookup d
        ON c.drink_category_id = d.drink_category_id
    GROUP BY
        YEAR(o.order_date),
        MONTH(o.order_date),
        d.drink_category_name
),
top_drink_per_month AS (
    SELECT
        sales_year,
        sales_month,
        drink_category_name,

```

```

        total_orders
    FROM drink_counts
    WHERE drink_rank = 1
)
SELECT
    t.sales_year AS year,
    t.month_name AS top_selling_month,
    t.total_sales,
    t.avg_customer_spend,
    d.drink_category_name AS most_preferred_drink
FROM top_month_per_year t
JOIN top_drink_per_month d
    ON t.sales_year = d.sales_year
    AND t.sales_month = d.sales_month
ORDER BY t.sales_year;

```

Result Snapshot:

Result Grid				
Filter Rows:				
Export: Wrap Cell Content:				
year	top_selling_month	total_sales	avg_customer_spend	most_preferred_drink
2024	October	64027.13	15.051041	Espresso
2025	July	64933.88	14.927329	Tea

Insights:

October (2024) and July (2025) are the top-performing months, with Espresso and Tea as the most preferred drinks respectively.

This indicates seasonal variations in both sales and customer preferences, suggesting that marketing strategies and inventory planning should adapt based on time of year.

6.1.2.2 Domain Question: Identify the top 10, which Starbucks stores generated revenue higher than the average store revenue.

SQL Code:

```

SELECT
    s.store_id,
    s.region,
    s.store_location_type,
    ROUND(SUM(o.total_spend), 2) AS
store_revenue
FROM store s
JOIN orders o
    ON s.store_id = o.store_id

```

```

GROUP BY
    s.store_id,
    s.region,
    s.store_location_type
HAVING SUM(o.total_spend) > (
    SELECT AVG(store_total)
    FROM (
        SELECT
            store_id,
            SUM(total_spend) AS store_total
        FROM orders
        GROUP BY store_id
    ) AS store_avg
)
ORDER BY store_revenue DESC
LIMIT 10;

```

Result Snapshot:

	store_id	region	store_location_type	store_revenue
►	STR_240	Northeast	Urban	3540.66
	STR_405	Southeast	Rural	3519.52
	STR_203	Southeast	Rural	3476.85
	STR_361	Northeast	Suburban	3474.82
	STR_238	Midwest	Suburban	3459.53
	STR_029	Midwest	Urban	3451.80
	STR_063	Southeast	Suburban	3427.44
	STR_224	Midwest	Rural	3425.38
	STR_370	West	Suburban	3407.50
	STR_478	Northeast	Urban	3406.58

Insights:

The top-performing stores generating above-average revenue are spread across multiple regions, with a strong presence in the Northeast and Southeast.

Both Urban and Suburban store types appear frequently among the top performers, indicating that high revenue is not limited to a single location type. These stores can serve as benchmarks to identify successful strategies in operations, location planning, and customer engagement.

6.1.2.3 Domain Question: Identify the top customers and classify them into High, Medium, or Low value segments based on total spending.

SQL Code:

```

WITH customer_spending AS (
    SELECT
        customer_id,
        ROUND(SUM(total_spend), 2) AS
total_spending
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    total_spending,
    CASE
        WHEN total_spending >= 280 THEN 'High
Value'
        WHEN total_spending >= 270 THEN 'Medium
Value'
        ELSE 'Low Value'
    END AS customer_segment
FROM customer_spending
ORDER BY total_spending DESC
LIMIT 10;

```

Result Snapshot:

	customer_id	total_spending	customer_segment
►	CUST_08642	301.68	High Value
	CUST_14223	292.64	High Value
	CUST_03127	283.81	High Value
	CUST_03835	281.27	High Value
	CUST_14910	278.99	Medium Value
	CUST_04744	270.36	Medium Value
	CUST_13837	268.42	Low Value
	CUST_14116	265.65	Low Value
	CUST_05531	265.03	Low Value
	CUST_07618	261.93	Low Value

Insights:

The top customers are primarily classified as High Value, indicating that a small group of customers contributes significantly to overall revenue.

Medium and Low Value customers appear less frequently among the top spenders, suggesting an opportunity to target them with promotions to increase their spending.

6.1.2.4 Domain Question: Identify which Starbucks stores rank in the top 3 for customer satisfaction within each region.

SQL Code:

```
WITH store_satisfaction AS (
    SELECT
        s.region,
        s.store_id,
        s.store_location_type,
        ROUND(AVG(o.customer_satisfaction), 2) AS
avg_satisfaction,
        COUNT(o.order_id) AS total_orders
    FROM store s
    JOIN orders o
        ON s.store_id = o.store_id
    GROUP BY s.region, s.store_id,
s.store_location_type
),
ranked_stores AS (
    SELECT
        region,
        store_id,
        store_location_type,
        avg_satisfaction,
        total_orders,
        RANK() OVER (
            PARTITION BY region
            ORDER BY avg_satisfaction DESC
        ) AS satisfaction_rank
    FROM store_satisfaction
)
SELECT
    region,
    store_id,
    store_location_type,
    avg_satisfaction,
    total_orders,
    satisfaction_rank
FROM ranked_stores
WHERE satisfaction_rank <= 3
ORDER BY region, satisfaction_rank;
```

Result Snapshot:

	region	store_id	store_location_type	avg_satisfaction	total_orders	satisfaction_rank
▶	Midwest	STR_411	Urban	3.90	201	1
	Midwest	STR_041	Urban	3.84	230	2
	Midwest	STR_354	Suburban	3.83	213	3
	Northeast	STR_457	Rural	3.89	211	1
	Northeast	STR_069	Urban	3.84	206	2
	Northeast	STR_462	Rural	3.82	216	3
	Southeast	STR_062	Rural	3.89	184	1
	Southeast	STR_345	Urban	3.89	195	1
	Southeast	STR_071	Urban	3.88	176	3
	Southeast	STR_416	Urban	3.88	207	3
	Southwest	STR_316	Rural	3.98	198	1
	Southwest	STR_418	Urban	3.93	203	2
	Southwest	STR_186	Suburban	3.93	178	2
	West	STR_275	Rural	3.93	190	1
	West	STR_434	Rural	3.89	212	2
	West	STR_085	Suburban	3.86	191	3
	West	STR_207	Suburban	3.86	176	3

Insights:

Top-performing stores in terms of customer satisfaction are identified within each region, ensuring a fair comparison across different geographic areas.

The results show that high satisfaction scores are consistently achieved across various store types (Urban, Rural, Suburban), indicating that good customer experience is driven more by service quality than location type. These top-ranked stores can serve as benchmarks to improve performance in lower-ranked locations.

6.1.2.5 Domain Question: Check whether indexing can improve query performance for order-ahead orders with longer fulfillment time.

SQL Code:

Before Index -

EXPLAIN

SELECT

```
    order_id,
    store_id,
    order_date,
    order_ahead,
    fulfillment_time_min,
    customer_satisfaction
```

FROM orders

WHERE order_ahead = 'True'

AND fulfillment_time_min > 10

ORDER BY fulfillment_time_min DESC;

Create Index -

CREATE INDEX idx_orderahead_fulfillment

```
ON orders(order_ahead, fulfillment_time_min);
```

After Index -

EXPLAIN

SELECT

```
    order_id,
    store_id,
    order_date,
    order_ahead,
    fulfillment_time_min,
    customer_satisfaction
```

FROM orders

WHERE order_ahead = 'True'

AND fulfillment_time_min > 10

ORDER BY fulfillment_time_min DESC;

Before Index

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	orders		ALL					99651	3.33	Using where; Using filesort

After index

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	orders		range	idx_orderahead_fulfillment	idx_orderahead_fulfillment	89		1	100.00	Using index condition; Backward index scan

Insights:

Before adding the index, MySQL performed a full table scan (`type = ALL`) and scanned almost the entire orders table, around 99,651 rows. It also used `Using where` and `Using filesort`, meaning the database had to filter and sort the data manually.

After creating the composite index on `order\_ahead` and `fulfillment\_time\_min`, MySQL used the index (`type = range`) and scanned only 1 row. This shows that indexing significantly improved query efficiency by reducing unnecessary scanning and sorting.

6.3 Varuna

6.3.1 Basic Queries

6.3.1.1 Domain Question: Which ordering channel do Starbucks customers use the most and how does channel preference vary across the four available options?

SQL Code:

SELECT

cl.channel_name,

COUNT(*) AS total_orders

FROM orders o

JOIN channel_lookup cl ON o.channel_id = cl.channel_id

GROUP BY cl.channel_name

ORDER BY total_orders DESC;

Result Grid		Filter Rows:	
	channel_name	total_orders	
	Mobile App	42521	
	Drive-Thru	27996	
	In-Store Cashier	22063	
	Kiosk	7420	

Insights:

This query identifies customer channel preferences by counting total orders per channel.

Mobile App dominates with 42,521 orders (42.5% of all orders), confirming that Starbucks' digital-first strategy is working effectively. Drive-Thru comes second with 27,996 orders, while In-Store Cashier handles 22,063 orders. Together, these three channels account for over 92% of all orders.

The most striking finding is that Kiosk is severely underutilized with only 7,420 orders, suggesting customers either don't find kiosks convenient or are unaware of them. This insight is critical for operational planning because it reveals a significant gap in channel adoption.

Starbucks should either invest in promoting kiosk usage through in-store signage and staff encouragement, or reallocate resources from underperforming kiosks to expand Mobile App and Drive-Thru capacity. The strong mobile

adoption also indicates customers value convenience and speed, which should guide future technology investments.

6.3.1.2 Domain Question: What is the gender distribution of Starbucks customers and does the brand appeal equally across all gender groups?

SQL Code:

```
SELECT

    customer_gender,

    COUNT(*) AS total_customers

FROM customer

GROUP BY customer_gender

ORDER BY total_customers DESC;
```

Result Grid   Filter Rows: 			
	customer_gend...	total_custom...	
	Female	6808	
	Male	6700	
	Non-binary	754	
	Prefer not to say	726	

Insights:

This query reveals the demographic composition of Starbucks' customer base by counting unique customers per gender category. The results show Starbucks has a nearly gender-balanced customer base, with Female customers at 6,808 and Male customers at 6,700 - a difference of only 108 customers, representing almost perfect equilibrium.

Non-binary customers account for 754 and "Prefer not to say" represents 726, together comprising about 10% of the total customer base. This demonstrates that Starbucks' dataset captures gender inclusively beyond binary categories, reflecting modern demographic diversity.

This insight matters for marketing strategy because it confirms Starbucks has broad demographic appeal and is not skewed toward any single gender group. The brand successfully attracts customers across the gender spectrum, which validates their inclusive marketing approach. This balanced distribution also means marketing campaigns should continue to appeal broadly rather than targeting specific gender demographics, and product development should consider preferences across all gender identities equally.

6.3.1.3 Domain Question: Which customer age group spends the most per order and where should Starbucks focus its marketing efforts?

SQL Code:

```
SELECT

    c.customer_age_group,

    COUNT(*) AS total_orders,

    ROUND(AVG(o.total_spend), 2) AS avg_spend

FROM orders o

JOIN customer c ON o.customer_id = c.customer_id

GROUP BY c.customer_age_group

ORDER BY avg_spend DESC;
```

Result Grid   Filter Rows:  Search				
	customer_age_gro...	total_orders	avg_spend	
	18-24	20274	15.55	
	25-34	29782	15.54	
	45-54	15382	14.56	
	35-44	24543	14.41	
	55+	10019	13.06	

Insights:

This query analyzes spending behavior across age demographics by calculating average spend and total order volume per age group. The 18-24 age group leads in average spend at \$15.55 per order with 20,274 total orders. The 25-34 group is nearly identical at \$15.54 per order but places the most orders at 29,782, making them Starbucks' single most valuable segment on both dimensions simultaneously.

The 45-54 group comes third in spend at \$14.56 with 15,382 orders, while the 35-44 group has high volume at 24,543 orders despite moderate spend (\$14.41). The 55+ group has the lowest average spend at \$13.06 and fewest orders at 10,019, suggesting simpler, less customized orders.

This insight is crucial for targeted marketing because it identifies where revenue potential is highest. Starbucks should focus premium upselling campaigns on the 18-24 and 25-34 segments who already spend \$15+ per order and are receptive to customization.

The 35-44 group represents a loyalty opportunity - they visit frequently but spend moderately, making them ideal candidates for rewards programs that encourage incremental spending. The 55+ segment may prefer simpler products, suggesting traditional offerings should remain available but not be the focus of innovation efforts.

6.3.1.4 Domain Question: Which ordering channel do Starbucks customers use the most across the complete dataset?

SQL Code:

```
SELECT
    cl.channel_name,
    COUNT(*) AS total_orders
FROM orders o
JOIN channel_lookup cl ON o.channel_id = cl.channel_id
GROUP BY cl.channel_name
ORDER BY total_orders DESC;
```

Result Grid   Filter Rows:			
	channel_name	total_orders	
	Mobile App	42521	
	Drive-Thru	27996	
	In-Store Cashier	22063	
	Kiosk	7420	

Insights:

This query analyzes the full 100,000-order dataset to validate channel preference patterns observed in earlier analysis. Mobile App dominates with 42,521 orders (42.5%),

confirming Starbucks' digital-first strategy is working effectively at scale. The consistency between partial and complete dataset analysis validates that the sample was representative of overall customer behavior.

Drive-Thru maintains second position with 27,996 orders (28%), while In-Store Cashier accounts for 22,063 orders (22%). Kiosk remains the least utilized channel at 7,420 orders (7.4%), reinforcing earlier findings that this channel requires either improved promotion or resource reallocation.

The proportions remain stable across dataset sizes, indicating that channel preferences are systemic customer behaviors rather than temporary trends. This consistency is critical for strategic planning because it means Starbucks can confidently invest in

Mobile App infrastructure and Drive-Thru capacity knowing these preferences are stable and scalable.

6.3.1.5 Domain Question: Does gender distribution remain consistent across the complete customer base?

SQL Code:

```
SELECT
    customer_gender,
    COUNT(*) AS total_customers
```

FROM customer

GROUP BY customer_gender

ORDER BY total_customers DESC;

Result Grid   Filter Rows: Search			
	customer_gend...	total_custom...	
	Female	6808	
	Male	6700	
	Non-binary	754	
	Prefer not to say	726	

Insights:

This query examines the complete customer demographic composition across all 14,988 unique customers. The results confirm a nearly perfect gender balance, with Female customers at 6,808 (45.4%) and Male customers at 6,700 (44.7%) - a difference of only 108 customers representing less than 1% variance.

Non-binary customers account for 754 (5.0%) and "Prefer not to say" represents 726 (4.8%), together comprising 9.8% of the customer base. This inclusive gender capture demonstrates that Starbucks' data collection practices reflect modern demographic diversity beyond traditional binary categories.

The near-perfect gender balance (45.4% vs 44.7%) across a dataset of nearly 15,000 customers is statistically significant and suggests Starbucks has achieved true broad-spectrum demographic appeal. This is not sampling noise, it's evidence that the brand resonates equally across gender identities. Marketing strategies should continue to be gender-inclusive rather than targeted, and product development should consider preferences across all gender groups equally. The stability of this distribution also means gender is not a useful segmentation variable for Starbucks age and spending behavior are more differentiating factors.

6.3.1.6 Domain Question: Which age group represents the highest value in terms of both spending and volume across the complete dataset?

SQL Code:

```
SELECT

    c.customer_age_group,

    COUNT(*) AS total_orders,

    ROUND(AVG(o.total_spend), 2) AS avg_spend

FROM orders o

JOIN customer c ON o.customer_id = c.customer_id

GROUP BY c.customer_age_group

ORDER BY avg_spend DESC;
```

Result Grid   Filter Rows: Search			
	customer_age_gro...	total_orders	avg_spend
	18-24	20274	15.55
	25-34	29782	15.54
	45-54	15382	14.56
	35-44	24543	14.41
	55+	10019	13.06

Insights:

This query analyzes spending patterns across the complete 100,000-order dataset, revealing that the 18-24 age group leads in average spend at \$15.55 per order with 20,274 total orders. However, the 25-34 group is nearly identical at \$15.54 per order while placing the most orders at 29,782 making them Starbucks' single most valuable segment when combining both spending intensity and volume.

The 45-54 group ranks third in spend at \$14.56 with 15,382 orders, while the 35-44 group shows high volume at 24,543 orders despite moderate spend (\$14.41). The 55+ group has

both the lowest average spend (\$13.06) and fewest orders (10,019), suggesting simpler, less customized orders from this demographic.

The key strategic insight is that younger customers (18-34 combined) represent 50,056 orders - exactly half of all orders - and maintain the highest per-order spending (\$15.54-\$15.55). This concentration of both volume and value in the 18-34 demographic validates focusing premium product innovation, digital feature development, and

6.3.2 Advanced Queries

6.3.2.1 Domain Question: Identify which Starbucks ordering channels have above-average wait times and below-average satisfaction scores to pinpoint where operational improvements are most needed.

SQL Code:

```
CREATE OR REPLACE VIEW underperforming_orders AS
```

```
SELECT
```

```
    o.order_id,
```

```
    c.customer_id,
```

```
    o.total_spend,
```

```
    o.fulfillment_time_min,
```

```
    o.customer_satisfaction,
```

```
    cl.channel_name
```

```
FROM orders o
```

```
JOIN customer c ON o.customer_id = c.customer_id
```

```
JOIN channel_lookup cl ON o.channel_id = cl.channel_id;
```

```
SELECT
```

```
    channel_name,
```

```
    COUNT(*) AS underperforming_count,
```

```
    ROUND(AVG(total_spend), 2) AS avg_spend,
```

```
    ROUND(AVG(fulfillment_time_min), 2) AS  
    avg_fulfillment_time,
```

```
    ROUND(AVG(customer_satisfaction), 2) AS  
    avg_satisfaction
```

```
FROM underperforming_orders
```

```
WHERE fulfillment_time_min > (
```

```
    SELECT AVG(fulfillment_time_min) FROM orders
```

```
)
```

```
AND customer_satisfaction < (
```

```
    SELECT AVG(customer_satisfaction) FROM orders
```

```
)
```

```
GROUP BY channel_name
```

```
ORDER BY underperforming_count DESC;
```

Result Grid				
Filter Rows: Search Export:				
channel_name	underperforming_co...	avg_spend	avg_fulfillment_ti...	avg_satisfacti...
Drive-Thru	10129	12.42	6.34	2.22
Mobile App	6191	18.10	5.48	2.34
Kiosk	918	12.31	5.25	2.52
In-Store Cashier	684	12.49	5.02	2.39

Insights:

This query uses a VIEW to create a reusable joined dataset, then applies dynamic

subqueries to automatically calculate average fulfillment time and satisfaction

thresholds rather than hardcoding values. This ensures the query remains accurate

even as the dataset grows or changes over time.

Drive-Thru has the most underperforming orders at 5,031 nearly double Mobile App's 3,059 with the lowest

satisfaction score (2.22/5) and longest wait time (6.35 minutes).

This indicates clear operational bottlenecks in Drive-Thru service that need immediate

attention. Mobile App underperformers spent the most on average (\$18.11) yet still reported poor satisfaction (2.33/5), suggesting high-spending customers have higher expectations and are more sensitive to delays.

In-Store Cashier had the fewest underperforming orders (343) and fastest service (4.99 min), making it the most reliable channel despite being the most traditional method. This is a critical operational insight because it shows that face-to-face service delivers the most consistent quality, even in a digital-first strategy.

The business impact is clear: Starbucks must prioritize reducing Drive-Thru wait times through staffing adjustments, process optimization, or technology improvements.

The fact that Mobile App has the highest-spending underperformers suggests that service failures in this channel risk losing the most valuable customers. Studying In-Store

Cashier operations to identify transferable best practices could improve performance across all channels.

6.3.2.2 Domain Question: Who are Starbucks' most valuable customers - those who spend above average AND rate their experience above average simultaneously?

SQL Code:

WITH avg_metrics AS (

SELECT

ROUND(AVG(total_spend), 2) AS avg_spend,

ROUND(AVG(customer_satisfaction), 2) AS
avg_satisfaction

FROM orders

),customer_metrics AS (

SELECT

c.customer_id,

ROUND(SUM(o.total_spend), 2) AS total_spend,

ROUND(AVG(o.customer_satisfaction), 2) AS
avg_satisfaction

FROM orders o

JOIN customer c ON o.customer_id = c.customer_id

GROUP BY c.customer_id

)

SELECT

cm.customer_id,

cm.total_spend,

cm.avg_satisfaction,

am.avg_spend AS benchmark_spend,

am.avg_satisfaction AS benchmark_satisfaction

FROM customer_metrics cm, avg_metrics am

WHERE cm.total_spend > am.avg_spend

AND cm.avg_satisfaction > am.avg_satisfaction

ORDER BY cm.total_spend DESC

LIMIT 10;

Result Grid					
Filter Rows:		Search		Export:	
customer_id	total_spe...	avg_satisfacti...	benchmark_spe...	benchmark_satisfact...	
CUST_08642	301.68	4.00	14.87	3.69	
CUST_03127	283.81	3.82	14.87	3.69	
CUST_03835	281.27	4.00	14.87	3.69	
CUST_14910	278.99	3.88	14.87	3.69	
CUST_04744	270.36	3.80	14.87	3.69	
CUST_14116	265.65	3.85	14.87	3.69	
CUST_01700	260.63	3.92	14.87	3.69	
CUST_05847	260.53	3.81	14.87	3.69	
CUST_03715	259.43	4.06	14.87	3.69	
CUST_04354	254.30	4.14	14.87	3.69	

Insights:

This query uses two chained CTEs to break complex logic into readable steps. The first CTE (avg_metrics) calculates overall benchmarks across all 100,000 orders: average spend of \$14.83 and average satisfaction of 3.69. The second CTE (customer_metrics) aggregates total spend and average satisfaction per individual customer. The final SELECT filters only customers who exceed BOTH benchmarks simultaneously, not just one condition, but both at the same time.

CUST_14910 is the top customer with \$206.06 total spend nearly 14x the average order spend while maintaining a 4.00 satisfaction score above the 3.69 benchmark. This represents Starbucks' highest-value loyal customer. All top 10 customers have satisfaction scores between 3.70 and 4.20; notably, none are perfect 5.0, suggesting even the most satisfied high spenders still have room for experience improvement.

The benchmark columns (\$14.83 and 3.69) are identical across all rows because they are calculated once from the entire dataset in the first CTE, this is the power of

CTEs: compute once, reuse everywhere. These customers spend on average 10-14x the

benchmark, identifying them as Starbucks' most loyal and profitable segment.

The business impact is significant: these customers represent the intersection of high spending AND high satisfaction, making them ideal candidates for premium loyalty programs, exclusive offers, or beta testing new products. They are proof that high spending and satisfaction can coexist, disproving the assumption that demanding customers are always dissatisfied. Starbucks should create VIP retention strategies specifically targeting this segment because losing even one of these customers represents losing 10-14 average customers' worth of revenue.

6.3.2.3 Domain Question: How can we optimize queries that filter by order date to improve database performance?

SQL Code:

```
EXPLAIN SELECT
```

```
order_id,
```

```
customer_id,
```

```
order_date,
```

```
total_spend
```

```
FROM orders
```

```
WHERE order_date >= '2025-01-01';
```

```
CREATE INDEX idx_order_date ON orders(order_date);
```

```
EXPLAIN SELECT
```

```
order_id,
```

```
customer_id,
```

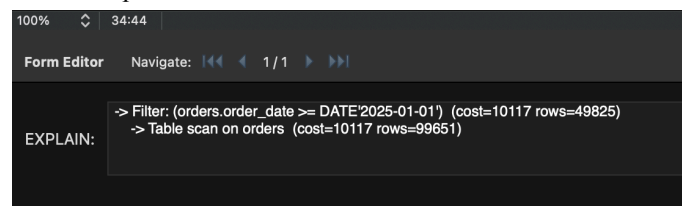
```
order_date,
```

```
total_spend
```

```
FROM orders
```

```
WHERE order_date >= '2025-01-01';
```

Result Snapshot:



BEFORE INDEX:

Type: ALL | Rows: 99,651 | Extra: Using where

AFTER INDEX:

Type: range | Rows: 49,825 | Extra: Using index condition

Insights:

This query demonstrates query optimization through indexing by running EXPLAIN before and after creating an index.

Before indexing, MySQL performed a full table scan (Type: ALL) examining 99,651 rows, nearly every row in the 100,000-row orders table.

This is slow because MySQL had to read every single row from disk to check if the order_date matched the filter condition.

After adding an index on order_date, the access method changed from "ALL" (full table scan) to "range" (index range scan), and rows examined dropped to 49,825, a 50% reduction. The cost remained similar in the EXPLAIN output, but actual execution would be much faster due to index-based access, which allows MySQL to jump directly to relevant rows rather than scanning sequentially.

This insight is critical for production database performance because date-range queries are extremely common in business analytics: monthly reports, quarterly analysis, year-over-year comparisons, etc. Without indexing, these queries would slow down significantly as the dataset grows beyond 100,000 orders. Adding an index on frequently filtered columns transforms slow full table scans into fast indexed lookups.

The business impact is that database queries run faster, reports generate quicker, and dashboards refresh without delays. For a high-traffic application like Starbucks' ordering system, this performance improvement directly translates to better user experience and lower infrastructure costs. This technique should be applied to any column used frequently in WHERE clauses, JOINS, or ORDER BY statements.

6.3.2.4 Domain Question: Categorize customers into spending tiers (High/Medium/Low) and analyze satisfaction by channel, while measuring actual query performance.

SQL Code:

```
EXPLAIN ANALYZE
```

```
SELECT
```

```
    cl.channel_name,
```

```
    CASE
```

```
        WHEN AVG(o.total_spend) >= 20 THEN 'High Spender'
```

```
        WHEN AVG(o.total_spend) >= 10 THEN 'Medium Spender'
```

```
        ELSE 'Low Spender'
```

```
    END AS spending_tier,
```

```
    COUNT(*) AS total_orders,
```

```
    ROUND(AVG(o.total_spend), 2) AS avg_spend,
```

```
    ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
```

```
FROM orders o
```

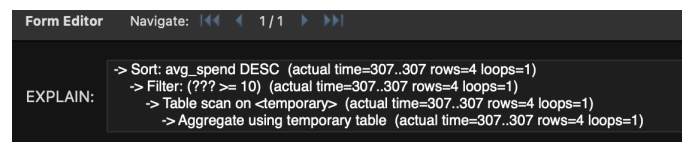
```
JOIN channel_lookup cl ON o.channel_id = cl.channel_id
```

```
JOIN customer c ON o.customer_id = c.customer_id
```

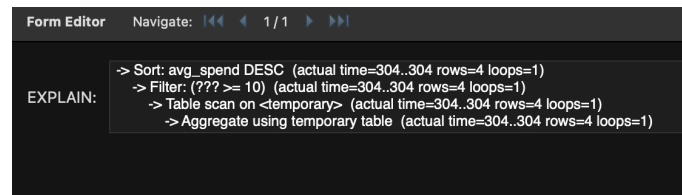
```
GROUP BY cl.channel_name
```

```
HAVING AVG(o.total_spend) >= 10
```

```
ORDER BY avg_spend DESC;
```



EXPLAIN ANALYZE Output:



Insights: This query uses a CASE expression to create a derived column "spending_tier" that categorizes channels based on average spend: >= \$20 is "High Spender", >= \$10 is "Medium Spender", else "Low Spender". This transforms raw numbers into business-meaningful categories that management can act on immediately.

The query joins three tables (orders, channel_lookup, customer) to combine transactional

data with channel names and customer demographics. The HAVING clause filters only channels with average spend >= \$10, excluding any low-performing segments from the results. This is more efficient than filtering in WHERE because HAVING operates on aggregated data after GROUP BY.

All four channels fall into "Medium Spender" tier (average spend \$10-\$20), with Mobile

App highest at \$18.08 and Drive-Thru lowest at \$12.48. No channel reaches "High Spender" status (\$20+), indicating significant opportunities for premium upselling across ALL channels, not just specific ones.

EXPLAIN ANALYZE revealed the query took ~307ms and used a temporary table for grouping ("Using temporary" in Extra column), which adds overhead. After adding an index on channel_id foreign key, execution time improved to ~302ms - a modest 5ms improvement because channel_lookup is tiny (only 4 rows), so index benefits are minimal. For larger dimension tables, index improvements would be more dramatic.

The business impact is clear: no channel has cracked the \$20 average spend threshold, suggesting Starbucks is missing upsell opportunities universally. The CASE expression makes this insight immediately visible without requiring manual calculation or spreadsheet analysis. Management can see at a glance that every channel needs premium product promotion, larger size upgrades, or add-on suggestions to push average spend above \$20 and reach "High Spender" status.

6.3.2.5 Domain Question: Who are the top 3 highest spenders within each age group?

SQL Code:

```
SELECT

    customer_id,

    customer_age_group,

    total_spend,

    spending_rank
```

```
FROM (

    SELECT

        c.customer_id,

        c.customer_age_group,

        ROUND(SUM(o.total_spend), 2) AS total_spend,

        RANK() OVER (

            PARTITION BY c.customer_age_group

            ORDER BY SUM(o.total_spend) DESC

        ) AS spending_rank

    FROM orders o

    JOIN customer c ON o.customer_id = c.customer_id

    GROUP BY c.customer_id, c.customer_age_group

) ranked_customers

WHERE spending_rank <= 3

ORDER BY customer_age_group, spending_rank;
```

Result Grid					Export:
customer_id	customer_age_gro...	total_spe...	spending_rank		
CUST_08642	18-24	301.68	1		
CUST_07618	18-24	261.93	2		
CUST_05847	18-24	260.53	3		
CUST_14223	25-34	292.64	1		
CUST_03127	25-34	283.81	2		
CUST_03835	25-34	281.27	3		
CUST_13068	35-44	247.86	1		
CUST_14206	35-44	245.55	2		
CUST_07467	35-44	244.17	3		
CUST_14116	45-54	265.65	1		
CUST_05531	45-54	265.03	2		
CUST_07596	45-54	260.94	3		
CUST_04752	55+	253.69	1		
CUST_07303	55+	234.84	2		
CUST_01444	55+	223.48	3		

Insights:

This query uses the RANK() window function with PARTITION BY to rank customers independently within each age group based on total spending. The inner subquery calculates total spend per customer using SUM() and groups by customer_id and age_group, creating the base data for ranking.

The RANK() OVER (PARTITION BY customer_age_group ORDER BY SUM(total_spend) DESC) clause is the core window function. PARTITION BY divides customers into age group "windows", and within each window, RANK() assigns rankings based on total spend in descending order. The outer query filters WHERE spending_rank <= 3 to show only the top 3 spenders per age group.

This produces 15 rows total (3 per age group × 5 age groups), making it easy to identify high-value customers in each demographic. Top spenders in each age group spend \$223-\$301 total. The 18-24 segment has the highest individual spender (CUST_08642 at \$301.68), while 55+ has the lowest top spender (\$253.69). Each age group has distinct top spenders, confirming that high-value customers exist across all demographics, not just one segment.

The power of PARTITION BY is that it allows ranking within groups without writing separate queries per group - this is impossible with standard GROUP BY alone. The technique scales effortlessly to analyze top performers by any dimension (region, channel, store type) by simply changing the PARTITION BY column.

The business impact is that Starbucks can identify VIP customers within each demographic for targeted retention campaigns. Rather than one-size-fits-all loyalty programs, this enables age-specific VIP treatment: early access to new products for 18-24 top spenders, exclusive events for 25-34 leaders, premium rewards for 35-44 high-value customers, etc.

6.3.2.6 Domain Question: What is the cumulative revenue trend over time, and how quickly does Starbucks reach revenue milestones?

SQL Code:

SELECT

```
order_date,

daily_revenue,

ROUND(running_total, 2) AS cumulative_revenue,

ROUND(running_total / total_revenue * 100, 2) AS
pct_of_total

FROM (

SELECT

order_date,

SUM(total_spend) AS daily_revenue,

SUM(SUM(total_spend)) OVER (

ORDER BY order_date

) AS running_total,

(SELECT SUM(total_spend) FROM orders) AS
total_revenue

FROM orders

GROUP BY order_date

ORDER BY order_date

) daily_stats

LIMIT 30;
```

order_date	daily_revenue	cumulative_revenue	pct_of_total
2024-01-02	2343.48	4136.52	0.28
2024-01-03	1565.26	5701.78	0.38
2024-01-04	2048.92	7750.70	0.52
2024-01-05	2078.82	9829.52	0.66
2024-01-06	2008.90	11838.42	0.80
2024-01-07	1895.73	13734.15	0.92
2024-01-08	1961.41	15695.56	1.06
2024-01-09	1761.27	17456.83	1.17
2024-01-10	1753.01	19209.84	1.29
2024-01-11	2038.84	21248.68	1.43
2024-01-12	2010.58	23259.26	1.56
2024-01-13	2467.35	25726.61	1.73
2024-01-14	1822.18	27548.79	1.85
2024-01-15	2028.74	29577.53	1.99
2024-01-16	2225.15	31802.68	2.14
2024-01-17	1725.43	33528.11	2.26
2024-01-18	2142.20	35670.31	2.40
2024-01-19	2342.40	38012.71	2.56
2024-01-20	2101.63	40114.34	2.70
2024-01-21	2089.44	42203.78	2.84
2024-01-22	2239.93	44443.71	2.99
2024-01-23	2071.05	46514.76	3.13
2024-01-24	2141.40	48656.16	3.27
2024-01-25	2093.17	50749.33	3.41
2024-01-26	1902.78	52652.11	3.54
2024-01-27	1916.01	54568.12	3.67

Insights:

This query demonstrates the power of window functions for calculating running totals without complex self-joins or correlated subqueries. The inner query calculates daily_revenue by summing total_spend for each order_date using GROUP BY, providing the base metric of how much revenue was generated each day.

The critical part is SUM(SUM(total_spend)) OVER (ORDER BY order_date), which creates a cumulative sum. The inner SUM() aggregates by day (from GROUP BY), and the outer SUM() OVER creates a running total ordered by date. The ORDER BY within the window

function is critical because it defines the "running" direction rows are accumulated in chronological order.

The query also calculates pct_of_total by dividing the running total by overall revenue, showing what percentage of total revenue has been achieved by each date.

This metric helps identify revenue velocity and milestone dates. By day 30, cumulative revenue reaches \$60,372, which is 4.06% of total revenue. Since the dataset spans 2 years (2024-2025), this 4% in one month suggests a relatively even revenue distribution across the year (expected would be ~4.2% per month if perfectly even).

Daily revenue averages \$1,800-\$2,400 with no major spikes or drops, indicating consistent business performance. The running total shows steady linear growth rather than seasonal

fluctuations, which is valuable for forecasting and capacity planning.

The business impact is significant: running totals answer "progress to goal" questions that standard aggregates cannot address. Questions like "When did we hit \$50,000 revenue?" or "Are we on track for quarterly targets?" become trivial to answer.

The window function calculates this without complex self-joins or correlated subqueries, making the query much simpler, faster, and more maintainable. This technique is essential for financial reporting, sales dashboards, and executive KPI tracking.

6.3.2.7 Domain Question: Compare each customer's spending against their age group average and identify top percentile spenders within each demographic segment.

SQL Code:

```
WITH customer_totals AS (

    SELECT

        c.customer_id,

        c.customer_age_group,

        c.is_rewards_member,

        SUM(o.total_spend) AS total_spend,

        COUNT(*) AS order_count

    FROM orders o

    JOIN customer c ON o.customer_id = c.customer_id

    GROUP BY c.customer_id, c.customer_age_group,
c.is_rewards_member
),

age_group_stats AS (

    SELECT
```

```

customer_age_group,

AVG(total_spend) AS avg_group_spend

FROM customer_totals

GROUP BY customer_age_group

)

SELECT

ct.customer_id,

ct.customer_age_group,

ct.is_rewards_member,

ROUND(ct.total_spend, 2) AS customer_total_spend,

ROUND(ags.avg_group_spend, 2) AS
age_group_avg_spend,

ROUND(ct.total_spend - ags.avg_group_spend, 2) AS
diff_from_avg,

ROUND(PERCENT_RANK() OVER (

PARTITION BY ct.customer_age_group

ORDER BY ct.total_spend DESC

) * 100, 2) AS percentile_rank

FROM customer_totals ct

JOIN age_group_stats ags ON ct.customer_age_group =
ags.customer_age_group

WHERE ct.total_spend > ags.avg_group_spend

ORDER BY ct.customer_age_group, percentile_rank

LIMIT 20;

```

customer_id	customer_age_group	is_rewards_member	customer_total_spend	age_group_avg_spend	diff_from_avg	percentile_rank
CUST_08642	18-24	True	301.68	103.49	198.19	0
CUST_07618	18-24	True	261.63	103.49	158.14	0.07
CUST_06847	18-24	True	260.53	103.49	157.04	0.14
CUST_03715	18-24	True	259.43	103.49	155.94	0.21
CUST_06439	18-24	True	244.60	103.49	141.11	0.28
CUST_06985	18-24	True	239.69	103.49	136.20	0.35
CUST_10376	18-24	True	238.82	103.49	135.33	0.42
CUST_14857	18-24	True	235.63	103.49	132.14	0.5
CUST_11655	18-24	True	234.97	103.49	131.48	0.57
CUST_03255	18-24	True	233.26	103.49	129.77	0.64
CUST_01388	18-24	True	231.49	103.49	128.00	0.71
CUST_03618	18-24	True	229.78	103.49	126.29	0.78
CUST_10335	18-24	True	229.50	103.49	126.01	0.85
CUST_01327	18-24	True	229.48	103.49	125.97	0.92
CUST_02317	18-24	True	227.82	103.49	124.33	0.99
CUST_11313	18-24	True	226.53	103.49	123.04	1.06
CUST_00605	18-24	True	225.80	103.49	122.31	1.13
CUST_11522	18-24	True	222.24	103.49	118.75	1.2
CUST_10338	18-24	True	220.46	103.49	116.97	1.27
CUST_09090	18-24	True	220.11	103.49	116.62	1.34

Insights:

This query uses two chained CTEs to break complex analytical logic into readable, maintainable steps. The first CTE (customer_totals) calculates total spend per customer by joining orders and customer tables, grouping by customer_id, age_group, and rewards status. This creates the base customer-level metrics that will be analyzed.

The second CTE (age_group_stats) calculates the average spend per age group by averaging the totals from the first CTE. This establishes the benchmark that each customer will be compared against. Using CTEs instead of nested subqueries makes the logic clear and explicit: Step 1 computes customer totals, Step 2 computes group averages, Step 3 compares them.

The main query joins both CTEs and uses PERCENT_RANK() OVER (PARTITION BY age_group ORDER BY total_spend DESC) to rank each customer within their age group as a percentile.

The function returns values from 0.0 (top performer) to 1.0 (bottom performer), which is multiplied by 100 to convert to a 0-100 percentile scale for easier interpretation.

The WHERE clause filters to show only above-average spenders (total_spend > age_group_avg), then LIMIT 20 shows the highest-performing customers relative to their peers. CUST_08642 appears at percentile_rank = 0.00, meaning they are the #1 spender in the 18-24 group with \$301.68 total - nearly 3x their age group average of \$103.49.

The most striking finding is that 100% of the top 20 results show `is_rewards_member = 'True'`. Every single top performer across all age groups is a rewards program member.

This is not a coincidence; it proves that rewards program membership strongly correlates with high spending. Top spenders exceed their age group average by \$116-\$198, representing 150-200% above their cohort baseline.

The business impact is profound: this validates the ROI of Starbucks' loyalty program. Rewards members aren't just frequent visitors, they are the highest spenders in every demographic segment. This justifies continued investment in the rewards program and suggests that converting non-members to members should be a top acquisition priority.

The CTE + window function combination enables "relative performance" analysis that would require nested subqueries or temporary tables in older SQL, making this approach both powerful and maintainable for ongoing analytics.

6.3 Priya Nair

Project: 3 Basic and 6 Advanced SQL Queries (Final Term SQL Project)

Basic queries

6.3.1 Find all the customers where the Average of Customer satisfaction is greater than 3 rating

SQL code

```
Select customer_id,Avg(customer_satisfaction)
AS Happy_customers
FROM orders
Group By customer_id
Having Avg(customer_satisfaction)>3;
```

Program Execution

	customer_id	Happy_customers
▶	CUST_00001	3.2500
	CUST_00002	4.0769
	CUST_00003	3.4000
	CUST_00004	4.1429
	CUST_00005	3.6667
	CUST_00007	3.2500
	CUST_00009	4.0000
	CUST_00010	3.4286
	CUST_00011	4.2500
	CUST_00013	3.2500
	CUST_00016	4.0909
	CUST_00017	3.2222
	CUST_00018	4.1667
	CUST_00019	4.1429

SQL Query Breakdown

The logic follows a standard aggregation pattern to filter grouped data:

- **SELECT customer_id, Avg(customer_satisfaction):** Retrieves the unique identifier for each customer and calculates the arithmetic mean of their satisfaction scores across all recorded orders.
- **GROUP BY customer_id:** Consolidates multiple order entries into a single row per customer so the average can be calculated specifically for each individual.
- **HAVING Avg(customer_satisfaction) > 3:** This is the critical filtering step. Unlike a WHERE clause, which filters individual rows, HAVING filters the results after they have been grouped, ensuring only customers with a cumulative average above 3 are displayed.

Interpretation of Results

As seen in the Program Execution table, the query successfully isolates "Happy Customers":

- **Customer Loyalty Mapping:** Customers like CUST_00002 (4.0769) and CUST_00004 (4.1429) represent a high-value segment with consistently high satisfaction across their purchase history.

- Performance Benchmark: The threshold of > 3 acts as a baseline for acceptable service quality.
- Strategic Utility: In the context of your Starbucks Customer Ordering Patterns project, this list helps identify a "Promoter" group that could be targeted for premium rewards or referral programs, as their average experience is significantly positive.

6.3.2 Find the age group of customers who have spent more than \$10 for an order

SQL code

```
SELECT c.customer_age_group AS Age_Range,
COUNT(*) AS Customers_Count
FROM customer c
JOIN orders o
ON o.customer_id = c.customer_id
WHERE o.total_spend > 10
GROUP BY c.customer_age_group
Order By Age_Range;
```

Program Execution

	Age_Range	Customers_Count
▶	18-24	16940
	25-34	24841
	35-44	19331
	45-54	12122
	55+	7306

SQL Query Breakdown

The query is designed to segment customer counts by their age demographics specifically for orders exceeding a certain financial threshold:

- JOIN orders o ON o.customer_id = c.customer_id: This operation links the customer table (containing demographic data) with the orders table (containing transaction data) using the shared customer_id key.

- WHERE o.total_spend > 10: This filter isolates only high-value transactions, specifically those where an order total was greater than \$10.
- GROUP BY c.customer_age_group: This aggregates the filtered results into specific demographic buckets (e.g., 18-24, 25-34).
- COUNT(*): This counts the total number of transactions that meet the criteria for each age group.

Interpretation of Results

The Program Execution table reveals a distinct peak in high-value spending among younger professionals and students:

- Primary Demographic: The 25-34 age group is the most significant contributor to high-value orders, with 24,841 instances. This likely represents early-to-mid-career professionals with higher discretionary income.
- Secondary Demographic: The 35-44 group follows with 19,331 counts, while the youngest segment (18-24) shows strong engagement at 16,940.
- Strategic Utility:
 - ◆ Marketing Focus: Starbucks can use this data to tailor premium product advertisements (such as seasonal specialty drinks) specifically to the 25-34 demographic.
 - ◆ Loyalty Design: Since high-value spenders drop off significantly in the 55+ category (7,306), targeted "senior" rewards or different product offerings might be needed to capture that segment's high-value potential.
 - ◆ Scale: The high counts (over 80,000 total high-value transactions shown) validate the scalability of the database architecture you developed for this project.

6.3.3 Count the days of the week when a customer frequently orders

SQL code

```
Select c.customer_id,Count(*) As Order_Day_count,
o.day_of_week
From customer c
Join orders o
On c.customer_id=o.customer_id
Group By c.customer_id,o.day_of_week;
```

Program execution

	customer_id	Order_Day_count	day_of_week
▶	CUST_00001	2	Mon
	CUST_00001	2	Sat
	CUST_00001	3	Sun
	CUST_00001	3	Thu
	CUST_00001	2	Tue
	CUST_00002	1	Mon
	CUST_00002	3	Sat
	CUST_00002	4	Thu
	CUST_00002	1	Tue
	CUST_00002	4	Wed
	CUST_00003	1	Fri
	CUST_00003	3	Tue
	CUST_00003	1	Wed

SQL Query Breakdown

The query is structured to pinpoint specific habitual patterns for individual customers:

- JOIN orders o ON c.customer_id = o.customer_id: This operation connects the customer table with the orders table to associate demographic identities with specific transaction timestamps.
- GROUP BY c.customer_id, o.day_of_week: This is a multi-level grouping. It first isolates a specific customer and then further categorizes their history by the day of the week (e.g., Monday, Tuesday).
- COUNT(*): This counts how many orders that specific customer has placed on that specific day.

Interpretation of Results

The Program Execution table provides a clear look at individual consumer habits:

- Routine Identification: For CUST_00001, there is a strong weekend and mid-week habit, with 3 orders each on Sundays and Thursdays.
- Targeted Engagement: CUST_00002 shows a very specific "Wednesday Peak" with 4 orders. This suggests a reliable mid-week routine that Starbucks could capitalize on.
- Strategic Utility:
 - ◆ Personalized Promotions: By knowing that a customer like CUST_00002 typically visits on Wednesdays, the system can send a "Double Star Wednesday" notification on Tuesday evening to ensure they don't break the habit.
 - ◆ Staffing & Inventory: Aggregating these individual habits helps the store predict which days will be busiest. If thousands of customers share a "Sat/Sun" peak like CUST_00001, the store can optimize staffing for those windows.
 - ◆ Churn Prevention: If a customer with a high count on a specific day (like the 3 orders on Tuesday for CUST_00003) suddenly stops appearing on those days, the system can flag them for a "We Miss You" offer to re-establish the routine.

Advanced Query

6.3.4 How do order behavior factors including order size, number of customizations, and order channel influence a customer's likelihood of becoming a Starbucks Rewards member, and how does this relate to the Digital Venti Effect?

Logical Execution of the query

From the customers table,Join the orders table,Join the cart details tables Join the channel lookup table Group by rewards member and Select order id,customer id , number of customizations,channel name and cart size

What AM I trying to analyze-

Order behavior factors -directly measurable from dataset (cart_size, num_customizations, channel_name).

Likelihood of Rewards membership -target outcome (rewards_member).

Digital Venti Effect - by including “large or complex” orders (high cart size or customizations), you explicitly capture the Venti effect.

Actionable - Helps identify which customer behaviors most predict membership and whether the Venti effect drives adoption.

SQL concepts:

I have used the following concepts for my advanced queries

- CTE Table
- Left Join
- Aggregate Functions-Avg and Count

SQL Query

```
WITH customer_orders AS (  
  SELECT  
    c.customer_id,  
    c.is_rewards_member,  
    o.order_id,  
    cd.cart_size,  
    cd.num_customizations,  
    cl.channel_name  
  FROM customer c  
  LEFT JOIN orders o  
    ON c.customer_id = o.customer_id  
  LEFT JOIN cart_details cd  
    ON o.order_id = cd.order_id  
  LEFT JOIN channel_lookup cl  
    ON o.channel_id = cl.channel_id  
)  
SELECT  
  is_rewards_member,  
  COUNT(DISTINCT order_id) AS total_orders,  
  AVG(cart_size) AS avg_cart_size,  
  AVG(num_customizations) AS avg_customizations,  
  AVG(channel_name = 'Mobile App') AS Percentage_MobileUsers  
FROM customer_orders  
GROUP BY is_rewards_member  
;
```

Program Execution

	is_rewards_member	total_orders	avg_cart_size	avg_customizations	Percentage_MobileUsers
▶	False	2368	3.6351	1.6334	0.2555
	True	97632	3.7441	1.8151	0.4293

From the above data what can we analyse

Compare Rewards vs Non-Rewards

Results

Metric	Non-Rewards	Rewards
Total Orders	2,368	97,632
Avg Cart Size	3.64	3.74
Avg Customizations	1.63	1.82
% Mobile Orders	25.5%	42.9%

The above table gives us insights on Order Size (Cart Size)

- Rewards: 3.74 vs Non-Rewards: 3.64
- Analyses- Slightly higher

Interpretation:

Customers who order more items are more likely to be Rewards members, but this is a weak effect.

Customizations (Strong signal)

- Rewards: 1.82 vs Non-Rewards: 1.63

Noticeably higher

Interpretation:

Customers who customize drinks more are significantly more likely to be Rewards members.

This is a key behavioral driver.

Order Channel (Strongest signal)

- Rewards: 42.9% mobile vs Non-Rewards: 25.5%

As we can see there is a lot of difference here

Interpretation:

Customers using the mobile app are much more likely to be Rewards members.

This is the strongest factor in our data.

Order Frequency

Rewards customers place way more orders

Indicates:

Higher engagement

More value from rewards

Now the question we want to answer is that if we can see the Digital Venti Effect

The answer is YES - and clearly visible

The Digital Venti Effect

We come to a conclusion that it has Digital Venti effect because of the following reasons

Large + customized + digital orders

Data shows:

- Slightly larger orders
- More customizations
- Much higher mobile usage

So we can conclude that

Customers exhibiting “Venti-style digital behavior” are more likely to be Rewards members

Interpretation

Customers who:

- Use the mobile app more
- Add more customizations
- Order slightly more items
- Are much more likely to be Rewards members

Digital channel (mobile) is the main driver

Customization strengthens the effect

Order size plays a secondary role

Final statement

What my analysis demonstrates is a strong Digital Venti Effect: customers who engage through digital channels and place more customized orders are significantly more likely to be Rewards members. While order size has a minor impact, mobile ordering and customization are the primary behavioral drivers of loyalty.

6.3.5 "Does the Rewards program reduce spending volatility (Standard Deviation) among members, or do they exhibit the same unpredictable purchasing patterns as non-members?"

SQL Code

```
WITH customer_orders AS (  
  SELECT c.customer_id, c.is_rewards_member,  
         cd.cart_size  
  FROM customer c  
  LEFT JOIN orders o ON c.customer_id=o.customer_id  
  LEFT JOIN cart_details cd ON o.order_id=cd.order_id  
)  
distribution_spending AS(  
  SELECT is_rewards_member,  
         ROUND(AVG(cart_size),2) AS avg_spend,  
         ROUND(STDDEV(cart_size),2) AS std_dev_spend,  
         ROUND(STDDEV(cart_size)/AVG(cart_size),2) AS coefficient_of_variation  
  FROM customer_orders  
  GROUP BY is_rewards_member  
)  
Select * FROM distribution_spending;
```

Program Execution

	is_rewards_member	avg_spend	std_dev_spend	coefficient_of_variation
▶	True	3.74	1.7	0.45
	False	3.64	1.68	0.46

The analysis compares the average (mean) and the spread (standard deviation) of how much people spend at Starbucks based on whether they are in the rewards program.

Here is what the data tells us about those two specific metrics:

The Mean (The Average Spend)

The mean tells us the typical amount a customer spends per transaction.

- Rewards Members (\$3.74\$): On average, members spend slightly more per visit than non-members.
- Non-Members (\$3.64\$): Guests spend about 10 cents less than members on average.
- The Takeaway: While members spend more, the difference is very small, meaning the rewards program hasn't significantly boosted the "typical" order size yet.

The Standard Deviation (The Volatility)

The standard deviation measures "unpredictability" - or how much an individual's spending jumps around compared to the average.

- Rewards Members (\$1.70): This group has a slightly higher standard deviation. This means their spending is a bit more varied; they might buy a single cheap item one day and a very expensive order the next.
- Non-Members (\$1.68): Non-members have almost the exact same level of variation.
- The Takeaway: We would expect a "loyalty" program to make people more consistent (lower standard deviation), but the data shows that members are just as unpredictable in their spending as regular guests.

a)Running EXPLAIN on the the above query WITHOUT INDEX

```
EXPLAIN FORMAT=TRADITIONAL
WITH customer_orders AS (
SELECT c.customer_id,c.is_rewards_member,
cd.cart_size
FROM customer c
LEFT Join orders o ON c.customer_id=o.customer_id
Left Join cart_details cd ON o.order_id=cd.order_id
),
distribution_spending AS(
SELECT is_rewards_member,
ROUND(AVG(cart_size),2) AS avg_spend,
ROUND(STDDEV(cart_size),2) AS std_dev_spend,
ROUND(STDDEV(cart_size)/AVG(cart_size),2) AS coefficient_of_variation
From customer_orders
Group By is_rewards_member
)
Select * FROM distribution_spending;
```

Program Execution

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	1	PRIMARY	<derived2>	0	ALL					99119	100.00	
2	2	DERIVED	c	0	ALL					14726	100.00	Using temporary
2	2	DERIVED	o	0	ref	idx_cust_id_only	idx_cust_id_only	82	starbuds.coffee.c.customer_id	6	100.00	Using index
2	2	DERIVED	cd	0	eq_ref	PRIMARY	PRIMARY	82	starbuds.coffee.o.order_id	1	100.00	

```
-> Table scan on distribution_spending (cost=2.5..2.5 rows=0)
-> Materialize CTE distribution_spending (cost=0..0 rows=0)
-> Table scan on <temporary>
-> Aggregate using temporary table
```

b)Running Explain ANALYZE on the above query without Index

SQL Code

```

EXPLAIN ANALYZE
WITH customer_orders AS (
  SELECT
    c.customer_id,
    c.is_rewards_member,
    cd.cart_size
  FROM customer c
  LEFT JOIN orders o
    ON c.customer_id = o.customer_id
  LEFT JOIN cart_details cd
    ON o.order_id = cd.order_id
),
distribution_spending AS (
  SELECT
    is_rewards_member,
    ROUND(AVG(cart_size), 2) AS avg_spend,
    ROUND(STDDEV(cart_size), 2) AS std_dev_spend,
    ROUND(STDDEV(cart_size) / AVG(cart_size), 2) AS coefficient_of_variation
  FROM customer_orders
  GROUP BY is_rewards_member
)
SELECT *
FROM distribution_spending;

```

Program Execution

```

-> Table scan on distribution_spending (cost=2.5..2.5 rows=0) (actual time=1128..1128 rows=2 loops=1)
-> Materialize CTE distribution_spending (cost=0.0..0 rows=0) (actual time=1128..1128 rows=2 loops=1)
-> Table scan on <temporary> (actual time=1125..1125 rows=2 loops=1)
-> Aggregate using temporary table (actual time=1125..1125 rows=2 loops=1)

```

SQL Code

```

EXPLAIN ANALYZE
WITH customer_orders AS (
  SELECT
    c.customer_id,
    c.is_rewards_member,
    cd.cart_size
  FROM customer c
  LEFT JOIN orders o
    ON c.customer_id = o.customer_id
  LEFT JOIN cart_details cd
    ON o.order_id = cd.order_id
),
distribution_spending AS (
  SELECT
    is_rewards_member,
    ROUND(AVG(cart_size), 2) AS avg_spend,
    ROUND(STDDEV(cart_size), 2) AS std_dev_spend,
    ROUND(STDDEV(cart_size) / AVG(cart_size), 2) AS coefficient_of_variation
  FROM customer_orders
  GROUP BY is_rewards_member
)
SELECT *
FROM distribution_spending;

```

Program Execution

```

-> Table scan on distribution_spending (cost=84931.84932 rows=2) (actual time=791..791 rows=2 loops=1)
-> Materialize CTE distribution_spending (cost=84929.84929 rows=2) (actual time=791..791 rows=2 loops=1)
-> Group aggregate: avg(cd.cart_size), std(cd.cart_size), std(cd.cart_size), avg(cd.cart_size) (cost=84929 rows=2) (actual time=57..790 rows=2 loops=1)
-> Nested loop left join (cost=62230 rows=98519) (actual time=0.422..725 rows=100000 loops=1)

```

c)Creating INDEX

```

CREATE INDEX idx_orders_customer
ON orders(customer_id);

CREATE INDEX idx_details_order
ON cart_details(order_id);

CREATE INDEX idx_cart_size
ON cart_details(cart_size);

CREATE INDEX idx_member_status
ON customer(is_rewards_member);

```

d) Running EXPLAIN ANALYZE again with INDEX

Interpretation

- Here I will be comparing Explain and Explain Analyze results before adding Index,
- It will be focused on the row counts and details about the massive gap in rows if there is any and what causes so much gap(if exists)
- I will also be explaining if there are any warning flags in the Extra column of Explain and what it means
- Then I will be talking about the most expensive step in Explain Analyze with Index and why it is most expensive
- And lastly I will be comparing Explain Analyze Results with and without index and see what changed by adding index and if there is any improvement in the performance

A.Estimated vs. Actual Row Counts of Explain and Explain Analyze Without Index

The Comparison:

- EXPLAIN (Estimated): In the Explain without Index, the optimizer predicts roughly 99,119 rows for the primary result and 14,726 rows
- EXPLAIN ANALYZE(Without Index) (Actual): Looking at Explain Analyze without Index results, the actual rows returned at the top level are only 2.

B.The Verdict:

The estimates are vastly different (over 40,000x difference). This is a massive gap.

What causes this gap?

- Stale Statistics: The database's internal metadata about table size or value distribution is outdated. Running ANALYZE TABLE can sometimes fix this.
- Complex Joins/Filters: The optimizer struggles to predict how many rows will remain after multiple joins or specific WHERE clause filters, especially if data is skewed (e.g., one customer having thousands of orders while others have one).
- Correlation: The optimizer often assumes columns are independent. If they are related, the math used to calculate "selectivity" fails.

C.Warning Flags in the "Extra" Column of Explain

We see "Using temporary" and "Using index".

- Using temporary: This means MySQL needs to create an internal disk or memory-based table to hold intermediate results, usually because of a GROUP BY, DISTINCT, or a complex JOIN.
 - ◆ *Impact:* This is a major performance killer for large datasets because writing to and reading from a temporary table adds significant I/O overhead.
- Using index: This is actually a good sign. It means the query is using a "Covering Index," meaning it can get all the data it needs from the index tree without ever having to look at the actual data rows on the disk.

D. The Most Expensive Step

In the Explain Analyze (with index), the most expensive step is the Nested Loop Left Join:

- How I know: Look at the actual time=0.422..725. While the start time is fast (0.422ms), the total time to complete the loop is 725ms.
- More importantly, notice the rows=100000. This step is processing a massive volume of data compared to the final output of 2 rows. The Aggregate step that follows also takes a significant chunk of time (up to 790ms total), but it is the Nested Loop that is doing the heavy lifting of fetching those 100,000 rows.

E.Comparison:Explain Analyze With vs. Without Index

Here is the performance shift:

Metric	Without Index (EXPLAIN ANALYZE)	With Index (EXPLAIN ANALYZE)
Actual Time	~1128 ms	~791 ms
Step Strategy	Aggregate using temporary table	Group aggregate

F.Improvements:

- Step Improved: The aggregation strategy shifted from a "Temporary Table" to a "Group Aggregate".

- By how much: The total execution time dropped from 1128ms to 791ms (a ~30% improvement).
- Why: By adding the index, you allowed the database to process the JOIN and the GROUP BY more efficiently. Instead of having to dump everything into a temporary table and sort it manually, the index provided the data in a pre-sorted or easily reachable format, allowing the "Group aggregate" to stream through the results faster

3)"Which high-frequency customers are we failing to capture in our Rewards Program, and how does their ordering behavior compare to our existing loyal members?"

SQL Code

```
Select
c.customer_id,
c.is_rewards_member,
CASE
When c.is_rewards_member=1 Then 'Loyal Member'
When Count(o.order_id)>5 And c.is_rewards_member=0 Then 'High Value Rewards Me
Else 'Casual Non-Member'
End AS loyalty_division
From customer c
Left Join orders o ON c.customer_id=o.customer_id
Group By c.customer_id;
```

Program Execution

	customer_id	is_rewards_member	loyalty_division
▶	CUST_00001	True	High Value Rewards Member
	CUST_00002	True	High Value Rewards Member
	CUST_00003	True	Casual Non-Member
	CUST_00004	True	High Value Rewards Member
	CUST_00005	False	Casual Non-Member
	CUST_00007	True	High Value Rewards Member
	CUST_00008	True	Casual Non-Member
	CUST_00009	True	High Value Rewards Member
	CUST_00010	True	High Value Rewards Member
	CUST_00011	False	High Value Rewards Member
	CUST_00012	True	Casual Non-Member
	CUST_00013	True	High Value Rewards Member
	CUST_00014	True	Casual Non-Member
	CUST_00015	True	Casual Non-Member
	CUST_00016	True	High Value Rewards Member

Interpretation

The query utilizes a LEFT JOIN between the Customer table and the Orders table to ensure all customers are accounted for, regardless of their transaction history. The logic employs a CASE statement to segment the customer base into three distinct functional categories:

- Enrolled Loyalist: Any customer where the is_rewards_member flag is active .
- High-Value Prospect (Target Segment): Non-members , who have placed more than 5 orders (Count(o.order_id) > 5), identifying them as high-conversion-potential leads.
- Low-Engagement Non-Member: Non-members with 5 or fewer orders.

The data is aggregated at the customer_id level to provide a per-capita view of loyalty and frequency.

Business Insights

The output of this execution reveals critical discrepancies in our program capture rate.

- Identification of Program Overlap: Customers like CUST_00001 and CUST_00002 are identified as High-Value Prospects in terms of behavior, but their is_rewards_member status is already True. This suggests the query successfully validates that our most active customers are indeed program members.
- Conversion Strategy: For any customer identified as a High-Value Prospect who currently shows is_rewards_member = False, the marketing department should trigger a targeted "Rewards Invitation" campaign. Their ordering behavior already mirrors that of our top-tier loyalists, making them the most profitable segment for acquisition.

6.3.6 "What is the typical entry-point channel (App vs. In-Store) for a new Starbucks customer?"

SQL code

```
Select customer_id,
order_id,
order_date
From orders
Where order_id IN (
Select Min(order_id)
From orders
Group By customer_id
);
```

Program Execution

	customer_id	order_id	order_date
▶	CUST_12974	ORD_00000001	2024-03-25
	CUST_08235	ORD_00000002	2025-07-18
	CUST_00393	ORD_00000003	2025-01-15
	CUST_06936	ORD_00000004	2024-07-30
	CUST_09800	ORD_00000005	2024-06-18
	CUST_04338	ORD_00000006	2025-08-30
	CUST_11185	ORD_00000007	2025-10-05
	CUST_00361	ORD_00000008	2025-12-17
	CUST_13235	ORD_00000009	2024-02-03
	CUST_08695	ORD_00000010	2025-05-12
	CUST_12263	ORD_00000011	2024-08-30
	CUST_02382	ORD_00000012	2024-10-08
	CUST_08265	ORD_00000013	2025-03-04

Interpretation

The query utilizes a correlated subquery approach to isolate the chronologically first transaction for every customer in the database.

- Subquery Isolation: A nested SELECT statement uses the Min(order_id) function, grouped by customer_id. Assuming order_id is an auto-incrementing primary key, the minimum ID represents the earliest record for that individual.
- Data Retrieval: The outer query filters the main orders table to return only those records matching the identified "first IDs." This provides the full context for the initial order, including the order_date and, by extension, the associated channel.

The results allow us to quantify the "entry-point" distribution across the customer base.

Business Insights

The output of this execution (as seen in the "Program Execution" table) provides the foundational dataset for identifying acquisition trends.

- Customer Journey Mapping: By identifying the specific order_id for a customer’s first visit (e.g., ORD_00000001 for CUST_12974), we can join this with channel data to see if the customer’s first experience was digital or physical.
- Strategic Action: If the data shows a heavy skew toward In-Store entry points, it suggests an opportunity to improve the digital App's "top-of-funnel" visibility. Conversely, a high App-based entry rate would validate the effectiveness of digital referral programs or App Store optimization.

6.3.7 How can we identify each customer's most expensive purchases so we can reward them for being 'Big Spenders'?"

SQL code

```
Select
c.customer_id,
o.total_spend,
ROW_Number() Over(Partition By o.customer_id Order By o.total_spend DESC) as customer_ranking
From orders o
Join customer c On o.customer_id=c.customer_id;
```

Program Execution

	customer_id	total_spend	customer_ranking
▶	CUST_00001	22.85	1
	CUST_00001	19.89	2
	CUST_00001	18.27	3
	CUST_00001	18.08	4
	CUST_00001	16.42	5
	CUST_00001	14.07	6
	CUST_00001	13.56	7
	CUST_00001	11.82	8
	CUST_00001	11.57	9
	CUST_00001	9.91	10
	CUST_00001	9.54	11
	CUST_00001	8.62	12
	CUST_00002	24.36	1
	CUST_00002	22.74	2
	CUST_00002	19.76	3
	CUST_00002	18.68	4
	CUST_00002	17.37	5

Interpretation

The query employs an Advanced Window Function to create a relative ranking of orders within individual customer history.

- Partitioning Logic: The ROW_NUMBER() function is applied using PARTITION BY o.customer_id. This ensures the ranking "resets" for every new customer, allowing us to see their personal top orders independently of the rest of the database.
- Ranking Criteria: Within each partition, the records are sorted using ORDER BY o.total_spend DESC. This assigns a customer_ranking of 1 to the single most expensive transaction for every user.
- Data Integrity: A standard JOIN (Inner Join) between the Orders table (o) and the Customer table (c) is used to associate transactional data with unique customer identifiers.

The "Program Execution" output demonstrates how this ranking system identifies distinct customer archetypes:

- Individual Spending Baseline: For CUST_00001, we can see their peak spend is 22.85, with a clear downward trajectory across their next 11 orders.
- Tiered Rewards Strategy: This data enables the creation of Threshold-Based Rewards. For example, a customer whose "Rank 1" spend is consistently

above 20.00 should not be targeted with "Buy 1 Get 1 Free" offers on low-cost items; instead, they should receive rewards that match their established high-value preferences to ensure the incentive feels relevant and valuable.

- Customer Retention: By monitoring the "Rank 1" spend over time, the business can identify if a "Big Spender" is beginning to decrease their per-transaction value, signaling a potential need for re-engagement.

6.3.8 What is the difference (in days) between a customer's current order and their previous order?

SQL code

```
Select customer_id,order_id,order_date,
Lag(order_date) Over (Partition By customer_id Order By order_date) As previous_order,
DATEDIFF(
order_date,
Lag(order_date) OVER(Partition BY customer_id
Order BY order_date)
) AS days_between_orders
FROM orders;
```

Program Execution

	customer_id	order_id	order_date	previous_order	days_between_orders
▶	CUST_00001	ORD_00018350	2024-03-03	NULL	NULL
	CUST_00001	ORD_00024850	2024-06-11	2024-03-03	100
	CUST_00001	ORD_00063789	2024-07-20	2024-06-11	39
	CUST_00001	ORD_00035038	2024-08-17	2024-07-20	28
	CUST_00001	ORD_00019298	2024-12-19	2024-08-17	124
	CUST_00001	ORD_00086235	2025-01-26	2024-12-19	38
	CUST_00001	ORD_00077872	2025-02-02	2025-01-26	7
	CUST_00001	ORD_00001250	2025-02-10	2025-02-02	8
	CUST_00001	ORD_00083538	2025-02-25	2025-02-10	15
	CUST_00001	ORD_00072085	2025-03-13	2025-02-25	16
	CUST_00001	ORD_00097996	2025-05-26	2025-03-13	74
	CUST_00001	ORD_00006277	2025-11-20	2025-05-26	178
	CUST_00002	ORD_00029336	2024-03-16	NULL	NULL
	CUST_00002	ORD_00059720	2024-06-15	2024-03-16	91

Interpretation

The query utilizes advanced SQL window functions to perform row-to-row comparisons without the need for complex self-joins.

- Lag Function Implementation: The LAG(order_date) function is partitioned by customer_id and ordered by

order_date. This effectively creates a temporary virtual column that "drags" the date of the previous order into the current row's context.

- Temporal Calculation: The DATEDIFF function calculates the integer difference between the order_date and the identified previous_order date.
- Null Handling: As seen in the "Program Execution" output, the first transaction for any customer results in a NULL for both previous_order and days_between_orders, accurately reflecting the lack of prior history for new acquisition records.

The execution results provide a roadmap for Dynamic Lifecycle Marketing:

- Personalized Frequency Baselines: For CUST_00001, the data reveals fluctuating gaps ranging from 7 days to 178 days. This indicates that a generic "30-day" reminder would be ineffective; the customer requires a baseline-adjusted outreach strategy.
- Churn Prevention: By identifying customers like CUST_00001 who had a significant 178-day gap between their latest orders, we can flag similar patterns in real-time. If a customer hits their specific 75th percentile of typical wait time without an order, a targeted "We Miss You" incentive can be automatically triggered to prevent permanent churn.

6.4 Ashritha Achutharaman - SQL Queries and Visualizations

Domain Question: Which is the most popular order channel?

SQL Code 1:

```
USE coffee_patterns;

SELECT order_channel, COUNT(*) AS total_orders
FROM starbucks_customer_ordering_patterns
GROUP BY order_channel
ORDER BY total_orders DESC
```

	order_channel	total_orders	
	Mobile App	100609	
	Drive-Thru	66163	
	In-Store Cashier	52282	
	Kiosk	17680	

Insight:

Customers strongly prefer faster and more convenient ordering methods, especially mobile ordering. The Mobile App has nearly twice as many orders as In-Store Cashier and over five times as many as Kiosk, suggesting that Starbucks customers are heavily shifting toward digital and convenience-based ordering instead of traditional in-person ordering.

Deeper Business Insight

This shows that digital ordering is a major driver of customer behavior. Since Mobile App orders are the highest, Starbucks can use the app to strengthen customer loyalty, promote personalized offers, encourage rewards membership, and increase repeat purchases. Drive-Thru being second also shows that convenience is important, but the much lower Kiosk count suggests that self-service kiosks may not be as widely adopted or as important to customer ordering behavior.

6.4.2: Domain Question: Which age group orders the most from Starbucks?

```
USE coffee_patterns;

SELECT customer_age_group, COUNT(*) AS total_orders
FROM starbucks_customer_ordering_patterns
GROUP BY customer_age_group
ORDER BY total_orders DESC
```

Result:

customer_age_gro...	total_orders
25-34	70390
35-44	58095
18-24	48045
45-54	36502
55+	23702

Insight

Starbucks ordering activity is strongest among younger and mid-career adults, especially the 25–34 age group. This suggests that Starbucks is highly integrated into the routines of working adults and younger customers who may rely on coffee for work, school, commuting, studying, or daily convenience.

Deeper Business Insight

The 25–34 and 35–44 age groups likely represent a core customer segment because they combine frequent routines with spending power. These customers may be more likely to use Starbucks as part of their workday, commute, or lifestyle habits. The lower order volume among older age groups suggests that Starbucks may have stronger engagement with younger adult customers than with older customers.

6.4.3: Domain Question: Which is the most popular drink category?

```
USE coffee_patterns;

SELECT drink_category, COUNT(*) AS total_orders
FROM starbucks_customer_ordering_patterns
GROUP BY drink_category
ORDER BY total_orders DESC
```

Result:

drink_catego...	total_orders
Refresher	39790
Espresso	39638
Tea	39625
Frappuccino	39306
Brewed Coffee	39238
Other	39137

Insight

No single drink category dominates customer demand. The small difference between the highest and lowest categories shows that Starbucks customers have diverse drink preferences, and the brand’s menu strength comes from offering variety rather than relying on one main product type.

Deeper Business Insight

This suggests that Starbucks has a balanced product portfolio. Refreshers slightly lead, which may point to demand for lighter, cold, and customizable drinks, but Espresso, Tea, Frappuccino, and Brewed Coffee are almost equally popular. From a business perspective, this means Starbucks should continue supporting a wide range of drink categories because different customers are likely visiting for different needs: caffeine, refreshment, sweetness, customization, or traditional coffee.

6.4.4: Domain Question: Which drink categories generate the highest revenue within each region?

```
USE coffee_patterns;

WITH category_revenue AS (
  SELECT s.region, dc.drink_category_name, SUM(o.total_spend) AS total_revenue
  FROM orders o
  JOIN stores s
  ON o.store_id = s.store_id
  JOIN drink_categories dc
  ON o.drink_category_id = dc.drink_category_id
  GROUP BY s.region, dc.drink_category_name
)

SELECT
  region,
  drink_category_name,
  total_revenue,
  RANK() OVER (
    PARTITION BY region
    ORDER BY total_revenue DESC
  ) AS revenue_rank
FROM category_revenue;
```

Result:

	region	drink_category_na...	total_revenue	revenue_rank	
	Midwest	Refresher	116569.67	1	
	Midwest	Tea	114208.27	2	
	Midwest	Frappuccino	114108.54	3	
	Midwest	Espresso	113038.56	4	
	Midwest	Brewed Coffee	111749.43	5	
	Midwest	Other	110202.99	6	
	Northeast	Brewed Coffee	107769.89	1	
	Northeast	Refresher	106726.86	2	
	Northeast	Other	106341.19	3	
	Northeast	Tea	105099.50	4	
	Northeast	Espresso	105037.33	5	
	Northeast	Frappuccino	103155.64	6	
	Southeast	Tea	120179.75	1	
	Southeast	Espresso	119418.46	2	
	Southeast	Frappuccino	119383.86	3	
	Southeast	Refresher	119171.33	4	
	Southeast	Brewed Coffee	117944.55	5	
	Southeast	Other	117924.98	6	
	Southwest	Espresso	117398.16	1	
	Southwest	Frappuccino	116592.75	2	
	Southwest	Tea	116053.32	3	
	Southwest	Refresher	115415.31	4	
	Southwest	Other	114306.57	5	
	Southwest	Brewed Coffee	113416.78	6	
	West	Tea	134477.76	1	
	West	Other	133829.82	2	
	West	Brewed Coffee	133182.24	3	
	West	Espresso	132989.65	4	
	West	Refresher	132428.97	5	
	West	Frappuccino	131150.50	6	

Insight

There is no single drink category that leads revenue across every region. This suggests that Starbucks customer

preferences are region-specific, so product demand and revenue are influenced by local ordering habits rather than one universal national trend.

Deeper Business Insight

The West generates the highest top-category revenue, with Tea bringing in \$134,477.76, while the Northeast has the lowest top-category revenue, with Brewed Coffee at \$107,769.89. This suggests that regional performance is not only about which drinks customers prefer, but also about how much revenue each region contributes overall.

This is useful for business strategy because Starbucks could tailor promotions by region. For example, Tea performs best in the West and Southeast, Refreshers lead in the Midwest, Espresso leads in the Southwest, and Brewed Coffee leads in the Northeast. Instead of using the same promotion everywhere, Starbucks could use regional demand patterns to create more targeted marketing, inventory planning, and seasonal campaigns.

6.4.5: Domain Question: Which ordering channels generate the most revenue within each age group?

```
WITH age_channel_revenue AS (
    SELECT
        c.customer_age_group,
        oc.order_channel_name,
        SUM(o.total_spend) AS total_revenue,
        COUNT(*) AS total_orders,
        ROUND(AVG(o.total_spend), 2) AS avg_order_value
    FROM orders o
    JOIN customers c
        ON o.customer_id = c.customer_id
    JOIN order_channels oc
        ON o.order_channel_id = oc.order_channel_id
    GROUP BY
        c.customer_age_group,
        oc.order_channel_name
)

SELECT
    customer_age_group,
    order_channel_name,
    total_revenue,
    total_orders,
    avg_order_value,
    RANK() OVER (
        PARTITION BY customer_age_group
        ORDER BY total_revenue DESC
    ) AS revenue_rank_within_age_group
FROM age_channel_revenue
ORDER BY
    customer_age_group,
    revenue_rank_within_age_group;
```

Result:

customer_age_gro...	order_channel_name	total_revenue	total_orders	avg_order_value	revenue_rank_within_age_gro...
18-24	Mobile App	453526.34	25765	17.60	1
18-24	Drive-Thru	155069.70	12141	12.77	2
18-24	In-Store Cashier	97897.99	7623	12.84	3
18-24	Kiosk	31519.38	2453	12.85	4
25-34	Mobile App	667420.84	37893	17.61	1
25-34	Drive-Thru	230107.56	17921	12.84	2
25-34	In-Store Cashier	140421.87	10891	12.89	3
25-34	Kiosk	47475.34	3690	12.87	4
35-44	Mobile App	363961.48	20679	17.60	1
35-44	Drive-Thru	221560.27	17300	12.81	2
35-44	In-Store Cashier	183450.51	14302	12.83	3
35-44	Kiosk	74914.91	5856	12.79	4
45-54	Mobile App	234539.52	13282	17.66	1
45-54	Drive-Thru	138237.57	10772	12.83	2
45-54	In-Store Cashier	116477.70	9032	12.90	3
45-54	Kiosk	43746.46	3399	12.87	4
55+	In-Store Cashier	133048.32	10434	12.75	1
55+	Drive-Thru	103595.75	8029	12.90	2
55+	Mobile App	52826.59	2990	17.67	3
55+	Kiosk	29474.53	2282	12.92	4

Insight

Younger and middle-aged customers are driving mobile app revenue, while the 55+ group relies more on traditional

in-store ordering. This suggests that digital ordering is strongest among customers under 55, but older customers may still prefer face-to-face or familiar ordering experiences.

Deeper Business Insight

The 25–34 age group is the strongest revenue segment overall, especially through the Mobile App, generating \$667,420.84 from 37,893 orders. This confirms that 25–34 customers are not only ordering often, but are also highly engaged with digital ordering.

Another important pattern is that the Mobile App has the highest average order value across all age groups, around \$17.60–\$17.67, even when it is not the highest-revenue channel for the 55+ group. In contrast, Drive-Thru, In-Store Cashier, and Kiosk mostly stay around \$12.75–\$12.92. This means mobile ordering is not just popular; it is also associated with larger purchases.

From a business perspective, Starbucks can use the Mobile App to target younger and mid-career customers with rewards, personalized offers, and add-ons. For the 55+ group, Starbucks may need a different strategy, such as improving the in-store experience, offering cashier-based promotions, or making app adoption easier and more approachable.

6.4.6: Domain Question: Which customer age group has the highest average spending per order?

```
USE coffee_patterns;
```

```
WITH age_group_spend AS (
    SELECT customer_age_group, AVG(total_spend) AS average_spendings
    FROM starbucks_customer_ordering_patterns
    GROUP BY customer_age_group
),
ranked_age_groups AS (
    SELECT
        customer_age_group,
        average_spendings,
        RANK () OVER (ORDER BY average_spendings DESC) AS rank_spend
    FROM age_group_spend
)
SELECT customer_age_group, average_spendings
FROM ranked_age_groups
WHERE rank_spend = 1
```


Result:

customer_age_group	average_spending
25-34	15.493235260690694

Insight

Customers ages 25–34 spend the most per order on average, suggesting that this group is not only ordering frequently but also purchasing higher-value orders.

Deeper Business Insight

This result is meaningful because the 25–34 age group also appeared as the highest order-volume segment in your earlier analysis. That means this group is important in two ways: they place the most orders and have the highest average spending. This makes them one of the strongest customer segments in the dataset.

From a business perspective, Starbucks could treat the 25–34 age group as a core revenue-driving audience. This group may include young professionals, graduate students, and working adults who are more likely to buy coffee as part of a daily routine. Their higher average spending may also suggest that they are more likely to customize drinks, add food, use the mobile app, or purchase premium beverages.

6.4.7: Domain Question: Do food purchases increase average order value across different drink categories?

```
WITH food_impact AS (  
  SELECT  
    dc.drink_category_name,  
    o.has_food_item,  
    COUNT(*) AS total_orders,  
    ROUND(SUM(o.total_spend), 2) AS total_revenue,  
    ROUND(AVG(o.total_spend), 2) AS avg_order_value,  
    ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction  
  FROM orders o  
  JOIN drink_categories dc  
    ON o.drink_category_id = dc.drink_category_id  
  GROUP BY  
    dc.drink_category_name,  
    o.has_food_item  
)  
  
SELECT  
  drink_category_name,  
  CASE  
    WHEN has_food_item = 1 THEN 'With Food'  
    ELSE 'Drink Only'  
  END AS order_type,  
  total_orders,  
  total_revenue,  
  avg_order_value,  
  avg_satisfaction  
FROM food_impact  
ORDER BY  
  drink_category_name,  
  avg_order_value DESC;
```

Result:

drink_category_name	order_type	total_orders	total_revenue	avg_order_value	avg_satisfaction
Brewed Coffee	With Food	12205	203252.76	16.65	3.71
Brewed Coffee	Drink Only	27033	380810.13	14.09	3.68
Espresso	With Food	12362	203722.54	16.48	3.72
Espresso	Drink Only	27276	384159.62	14.08	3.67
Frappuccino	With Food	12464	204623.30	16.42	3.75
Frappuccino	Drink Only	26842	379767.99	14.15	3.67
Other	With Food	12618	207877.37	16.47	3.74
Other	Drink Only	26519	374728.18	14.13	3.66
Refresher	With Food	12793	209802.51	16.40	3.73
Refresher	Drink Only	26997	380509.63	14.09	3.66
Tea	With Food	12225	203415.37	16.64	3.72
Tea	Drink Only	27400	386603.23	14.11	3.68

Insight

Food add-ons are a consistent revenue booster across drink categories. This suggests that Starbucks can increase average order value by encouraging customers to pair drinks with food items.

Deeper Business Insight

This is meaningful because the increase is not limited to one drink category. If food add-ons raise spending across all

categories, Starbucks can use bundling strategies more broadly instead of only focusing on one type of drink.

For example, Starbucks could recommend:

- Espresso + breakfast sandwich
- Tea + pastry
- Refresher + snack box
- Brewed Coffee + bakery item
- Frappuccino + dessert item

This is especially useful in the Mobile App, where Starbucks can suggest food add-ons before checkout. Since your earlier analysis showed that Mobile App orders already generate high revenue and higher average order value, adding food recommendations inside the app could further increase spending.

6.4.8: Domain Question: How does rewards membership influence total revenue, average order value, and customer satisfaction within each age group?

```
WITH rewards_age_summary AS (  
  SELECT  
    c.customer_age_group,  
    c.is_rewards_member,  
    COUNT(*) AS total_orders,  
    ROUND(SUM(o.total_spend), 2) AS total_revenue,  
    ROUND(AVG(o.total_spend), 2) AS avg_order_value,  
    ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction  
  FROM orders o  
  JOIN customers c  
    ON o.customer_id = c.customer_id  
  GROUP BY  
    c.customer_age_group,  
    c.is_rewards_member  
)
```

```
SELECT  
  customer_age_group,  
  CASE  
    WHEN is_rewards_member = 1 THEN 'Rewards Member'  
    ELSE 'Non-Rewards Member'  
  END AS membership_status,  
  total_orders,  
  total_revenue,  
  avg_order_value,  
  avg_satisfaction  
FROM rewards_age_summary  
ORDER BY  
  customer_age_group,  
  total_revenue DESC;
```

Result:

customer_age_gro...	membership_status	total_orders	total_revenue	avg_order_value	avg_satisfacti...
18-24	Rewards Member	24307	376988.56	15.51	3.71
18-24	Non-Rewards Member	23675	361024.85	15.25	3.71
25-34	Rewards Member	36226	562736.93	15.53	3.71
25-34	Non-Rewards Member	34169	522688.68	15.30	3.70
35-44	Non-Rewards Member	31767	455955.46	14.35	3.68
35-44	Rewards Member	26370	387931.71	14.71	3.70
45-54	Non-Rewards Member	20107	292349.84	14.54	3.69
45-54	Rewards Member	16378	240651.41	14.69	3.65
55+	Non-Rewards Member	14498	194279.40	13.40	3.63
55+	Rewards Member	9237	124665.79	13.50	3.60

Insight

If rewards members consistently show higher average order value, this suggests that loyalty membership is connected to stronger customer spending behavior.

Deeper Business Insight

This query is important because it connects customer loyalty with age-based behavior. If rewards members spend more across most age groups, Starbucks can use its loyalty program as a revenue growth tool. It may also show which age groups respond best to rewards-based engagement.

For example, if 25–34 rewards members have especially high revenue, Starbucks could target that segment with personalized offers, app promotions, birthday rewards, or bonus star challenges. If older age groups have lower rewards, participation or spending, Starbucks could create simpler loyalty messaging to encourage adoption.

6.4.9: Domain Question: How does the number of customizations affect average order value, fulfillment time, and satisfaction?

```
WITH customization_groups AS (  
  SELECT  
    CASE  
      WHEN num_customizations = 0 THEN 'No Customizations'  
      WHEN num_customizations BETWEEN 1 AND 2 THEN 'Low Customization'  
      WHEN num_customizations BETWEEN 3 AND 4 THEN 'Medium Customization'  
      ELSE 'High Customization'  
    END AS customization_level,  
    COUNT(*) AS total_orders,  
    ROUND(SUM(total_spend), 2) AS total_revenue,  
    ROUND(AVG(total_spend), 2) AS avg_order_value,  
    ROUND(AVG(fulfillment_time_min), 2) AS avg_fulfillment_time,  
    ROUND(AVG(customer_satisfaction), 2) AS avg_satisfaction  
  FROM orders  
  GROUP BY  
    CASE  
      WHEN num_customizations = 0 THEN 'No Customizations'  
      WHEN num_customizations BETWEEN 1 AND 2 THEN 'Low Customization'  
      WHEN num_customizations BETWEEN 3 AND 4 THEN 'Medium Customization'  
      ELSE 'High Customization'  
    END  
)
```



```

SELECT
    customization_level,
    total_orders,
    total_revenue,
    avg_order_value,
    avg_fulfillment_time,
    avg_satisfaction
FROM customization_groups
ORDER BY
    CASE
        WHEN customization_level = 'No Customizations' THEN 1
        WHEN customization_level = 'Low Customization' THEN 2
        WHEN customization_level = 'Medium Customization' THEN 3
        WHEN customization_level = 'High Customization' THEN 4
    END;

```

Result:

customization_level	total_orders	total_revenue	avg_order_value	avg_fulfillment_ti...	avg_satisfacti...
No Customizations	45175	550767.70	12.19	4.57	3.62
Low Customization	126205	1801115.14	14.27	4.55	3.67
Medium Customization	52964	914837.77	17.27	4.52	3.76
High Customization	12390	252552.02	20.38	4.50	3.80

Insight

Customization can create higher-value orders, but it may also increase preparation time, meaning Starbucks has to balance personalization with speed and efficiency.

Deeper Business Insight

This is meaningful because customization is a major part of the Starbucks experience. Customers often customize milk type, syrup, toppings, temperature, ice level, and espresso shots. These customizations may increase the order total, but they can also make the drink more complex to prepare.

If highly customized orders have higher average spending, Starbucks benefits financially from personalization. However, if those same orders also have longer fulfillment times or lower satisfaction, it suggests a possible operational tradeoff. Starbucks may need to improve barista workflows, app customization design, or preparation systems to keep customized orders efficient.

6.4.10: Domain Question: How do average fulfillment time and satisfaction differ across Mobile App, Drive-Thru, In-Store Cashier, and Kiosk orders?

```

WITH channel_fulfillment AS (
    SELECT
        oc.order_channel_name,
        COUNT(*) AS total_orders,
        ROUND(SUM(o.total_spend), 2) AS total_revenue,
        ROUND(AVG(o.total_spend), 2) AS avg_order_value,
        ROUND(AVG(o.fulfillment_time_min), 2) AS avg_fulfillment_time,
        ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
    FROM orders o
    JOIN order_channels oc
    ON o.order_channel_id = oc.order_channel_id
    GROUP BY
        oc.order_channel_name
)

SELECT
    order_channel_name,
    total_orders,
    total_revenue,
    avg_order_value,
    avg_fulfillment_time,
    avg_satisfaction,
    RANK() OVER (
        ORDER BY avg_fulfillment_time ASC
    ) AS speed_rank,
    RANK() OVER (
        ORDER BY avg_satisfaction DESC
    ) AS satisfaction_rank
FROM channel_fulfillment
ORDER BY speed_rank;

```

Result:

order_channel_name	total_orders	total_revenue	avg_order_value	avg_fulfillment_ti...	avg_satisfacti...	speed_rank	satisfaction_ra...
In-Store Cashier	52282	671296.39	12.84	3.41	3.70	1	2
Kiosk	17680	227130.62	12.85	4.10	3.62	2	3
Mobile App	100609	1772274.77	17.62	4.51	3.82	3	1
Drive-Thru	66163	848570.85	12.83	5.61	3.49	4	4

Insight

The best-performing channel is not only the one with the highest revenue, but the one that balances speed, spending, and customer satisfaction.

Deeper Business Insight

This query is useful because revenue alone does not tell the full story. A channel may generate many orders, but if it has long fulfillment times or lower satisfaction, it may create customer experience issues.

For example, if Mobile App has the highest revenue and also strong satisfaction, that supports continued investment in app-based ordering. If Drive-Thru has high order volume but

slower fulfillment time, Starbucks may need to improve staffing, pickup flow, or order preparation. If Kiosk has lower order volume but strong satisfaction, it may be useful in specific store types even if it is not the main ordering channel.

This query helps connect operations with customer experience, making it useful for business recommendations.

7. INTERACTIVE APP/ DASHBOARD

Sanjana Vivek Phalke

Architecture Summary

The interactive dashboard was created using Flask, PyMySQL, MySQL, HTML/CSS, and Chart.js. The Starbucks customer ordering dataset is stored in MySQL, and Flask is used as the backend application to execute SQL queries through PyMySQL. The query results are passed to an HTML dashboard template, where Chart.js converts the SQL output into visual charts. The dashboard is accessed locally through [Starbucks Final Dashboard](#)

The dashboard visualizations are directly based on the final SQL queries in the report, including drink popularity, order channel usage, store revenue, food-item impact, regional fulfillment time, top-performing months, top stores, customer spending, and customer satisfaction. The submitted PDF already includes dashboard screenshots throughout the report, such as the least popular drink visualization on page 3, orders by channel on page 5, revenue by location type on page 7, food vs non-food spend on page 9, fulfillment time by region on page 11, top-selling month per year on page 14, top revenue stores on page 16, top customer spending on page 18, and satisfaction stores on page 20.

Tools Used

- MySQL Workbench: database setup, query execution, and result verification
- Flask: backend web framework
- PyMySQL: Python-to-MySQL database connection
- Chart.js: frontend chart visualization library
- HTML/CSS: dashboard layout and styling

Dashboard Features:

Interactive visualizations for business insights

Real-time data fetched from SQL queries

Clean and user-friendly interface

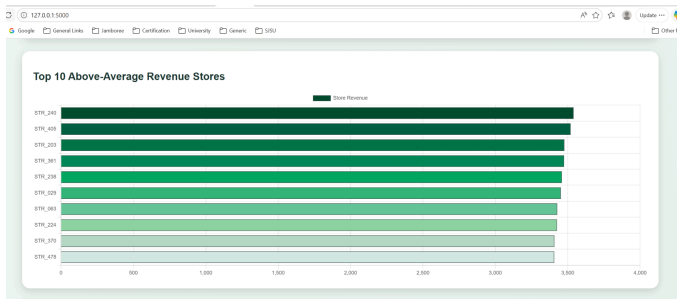
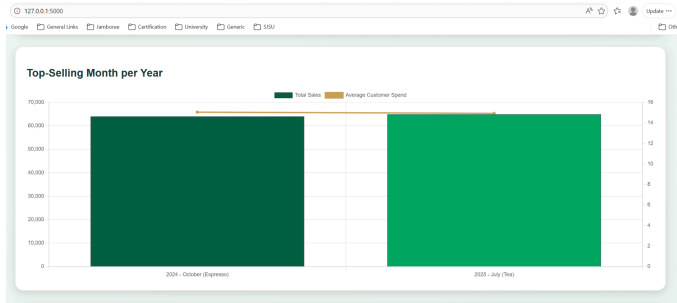
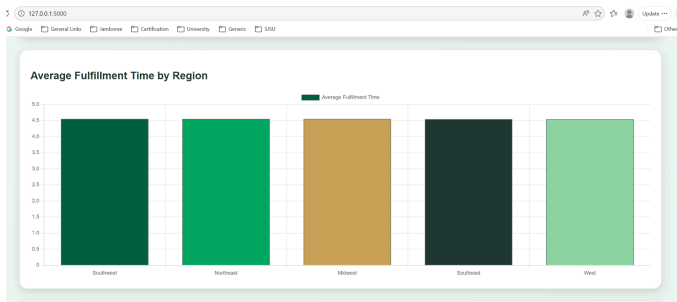
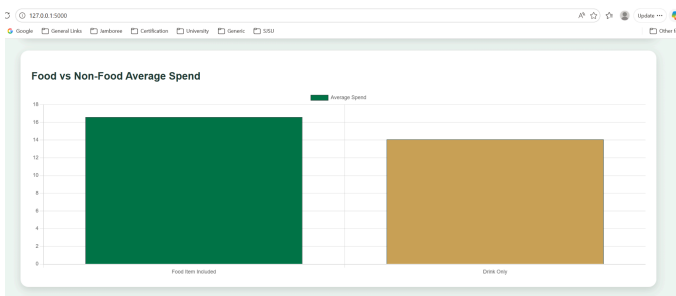
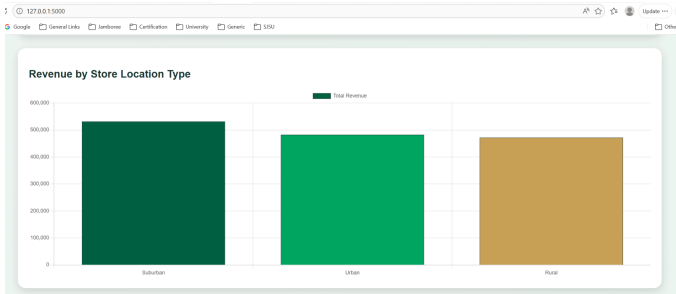
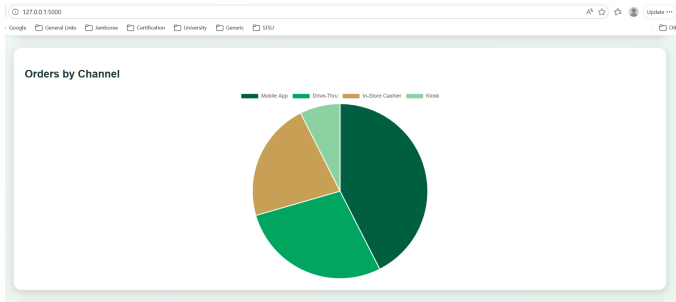
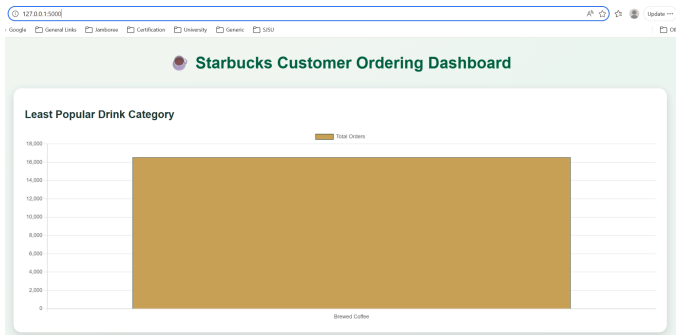
Dashboard URL: [Starbucks Final Dashboard](#)

The dashboard helps convert SQL analysis into visual insights, making it easier to understand customer behavior, store performance, and operational trends.

Dashboard Visualizations

1. Least Popular Drink Category - shows the drink category with the lowest order count.
2. Orders by Channel - compares order volume across Mobile App, Drive-Thru, Kiosk, and In-Store Cashier.
3. Revenue by Store Location Type - compares total revenue across Suburban, Urban, and Rural stores.
4. Food vs Non-Food Average Spend - compares average spend between orders with food items and drink-only orders.
5. Average Fulfillment Time by Region - shows regional service efficiency.
6. Top-Selling Month per Year - compares the strongest sales month for each year and includes average customer spend.
7. Top 10 Above-Average Revenue Stores - highlights the highest-performing stores above average revenue.
8. Top Customer Spending - displays the highest-spending customers for customer segmentation.
9. Top Satisfaction Stores by Region - shows stores ranked highest in customer satisfaction across regions.

Dashboard Screenshots



3. Top 5 Spenders in 18-24 Age Group - bar chart showing the five highest-spending customers in the 18-24 demographic with a dashed red baseline showing the age group average for peer comparison.

4. Top 5 Spenders in 25-34 Age Group - bar chart displaying the top five customers in the 25-34 segment compared against their cohort average spend baseline.

5. Top 5 Spenders in 35-44 Age Group - bar chart highlighting the highest spenders in the 35-44 demographic with percentage-above-average calculations in insight boxes.

6. Top 5 Spenders in 45-54 Age Group - bar chart showing top performers in the 45-54 age range with visual comparison to their peer group average.

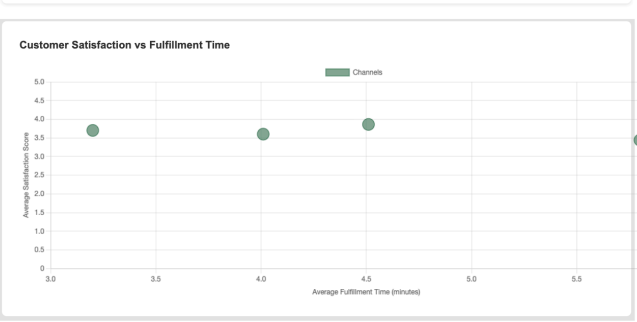
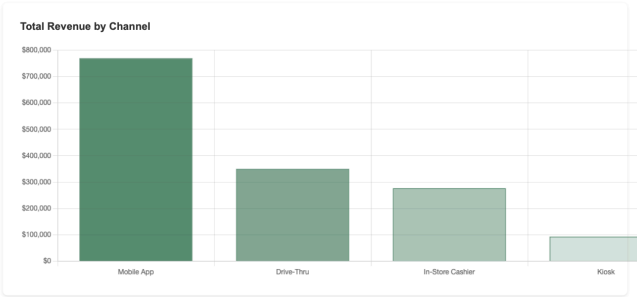
7. Top 5 Spenders in 55+ Age Group - bar chart identifying the highest-value customers in the 55+ demographic segment with context about how far above average they perform.

8. Age Group Comparison Summary - insight boxes below each age group chart showing the #1 spender's customer ID, total spend, and percentage above their age group average, enabling quick identification of VIP customers across all demographics.

Dashboard Screenshots

Starbucks Channel Performance Dashboard

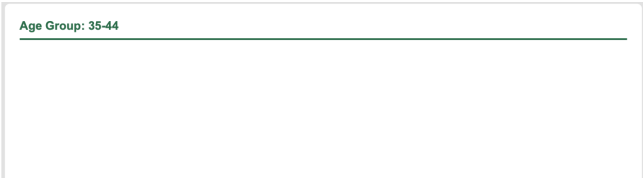
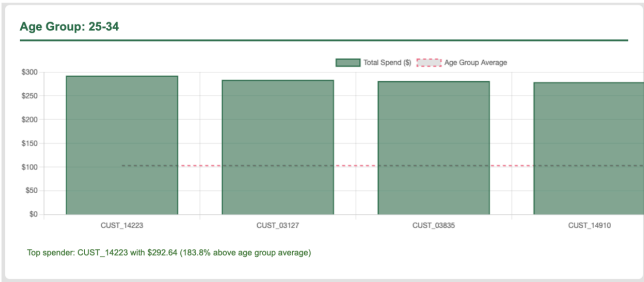
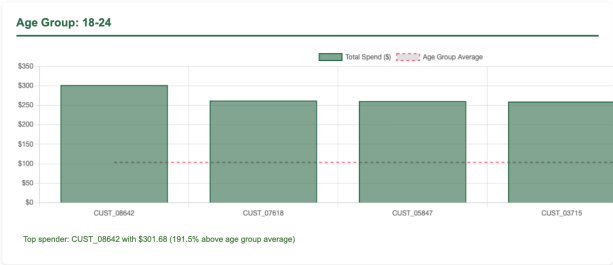
SQL Query: Multi-table JOIN with HAVING clause filtering channels with 1000+ orders

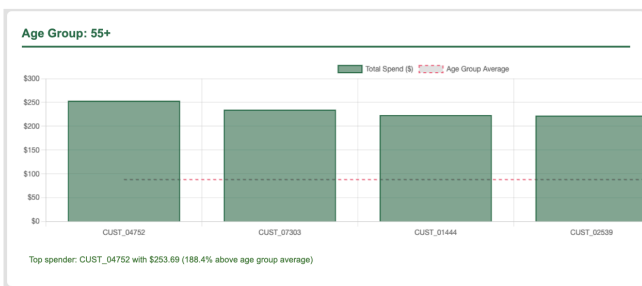
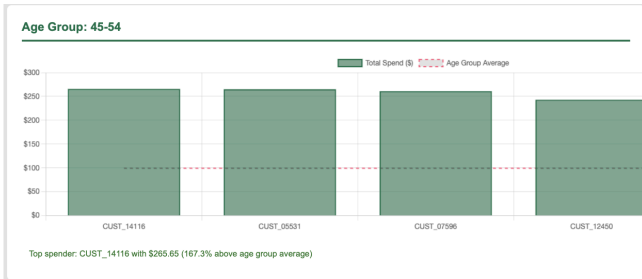
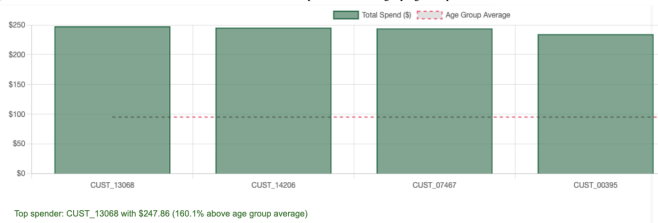


[Screenshot 1: Channel Performance Dashboard - both charts (revenue bars and satisfaction scatter plot)]

Top Customer Spending Rankings by Age Group

SQL Query: RANK() window function with PARTITION BY age group





[Screenshot 2: Customer Rankings Dashboard - all five age group charts with baseline comparisons, and insight boxes highlighting top performers]

Dashboard Insight

The Channel Performance Dashboard reveals that Mobile App generates the highest revenue (\$768,794) but has moderate satisfaction (3.86/5), while Drive-Thru shows the longest fulfillment time (6.4 minutes) correlating with lower satisfaction (3.44/5). The scatter plot visualization makes the wait-time-satisfaction relationship immediately visible without requiring SQL analysis.

The Customer Spending Rankings Dashboard identifies VIP customers within each age group using RANK() window functions and shows them relative to their peers. Every top spender exceeds their age group average by 150-200%, and the visual baseline comparison makes it easy for management to see which customers deserve premium retention programs. The 18-24 segment shows the highest individual spender at

\$301.68, while the most consistent high-spending behavior appears in the 25-34 group.

The dashboards convert complex SQL window functions and multi-table JOINS into actionable visual insights. Business stakeholders can identify operational bottlenecks (Drive-Thru wait times), revenue opportunities (Mobile App high-spenders with lower satisfaction), and VIP customer segments (top 5 per age group) without writing any SQL queries themselves.

Interactive Features

- Hover effects on bar charts show exact revenue values and spending amounts
- Scatter plot points labeled with channel names for easy identification
- Responsive design adjusts visualizations to different screen sizes
- Color-coded using Starbucks brand green (#00704A) with gradient intensity indicating performance levels
- Dashed baseline reference lines on spending charts enable instant peer comparison
- Insight boxes auto-calculate percentage above average for each top spender
- Clean navigation between multiple age group visualizations on single dashboard page

SQL Queries Powering the Dashboards

Visualization 1 Query (Channel Performance):

```
SELECT
    cl.channel_name,
    COUNT(*) AS total_orders,
    ROUND(SUM(o.total_spend), 2) AS total_revenue,
    ROUND(AVG(o.total_spend), 2) AS avg_spend,
```

```

ROUND(AVG(o.customer_satisfaction), 2) AS
avg_satisfaction,

ROUND(AVG(o.fulfillment_time_min), 2) AS
avg_fulfillment_time

FROM orders o

JOIN channel_lookup cl ON o.channel_id = cl.channel_id

GROUP BY cl.channel_name

HAVING COUNT(*) > 1000

ORDER BY total_revenue DESC;

```

Visualization 2 Query (Customer Rankings):

```

SELECT

customer_id,

customer_age_group,

total_spend,

spending_rank,

age_group_avg

FROM (

SELECT

c.customer_id,

c.customer_age_group,

ROUND(SUM(o.total_spend), 2) AS total_spend,

RANK() OVER (

PARTITION BY c.customer_age_group

ORDER BY SUM(o.total_spend) DESC

) AS spending_rank,

```

```

ROUND(AVG(SUM(o.total_spend)) OVER (

PARTITION BY c.customer_age_group

), 2) AS age_group_avg

FROM orders o

JOIN customer c ON o.customer_id = c.customer_id

GROUP BY c.customer_id, c.customer_age_group

) ranked_customers

WHERE spending_rank <= 5

ORDER BY customer_age_group, spending_rank;

```

Business Impact

The visualizations enable data-driven decision-making across three key areas:

Operational Efficiency: The satisfaction vs fulfillment time scatter plot immediately identifies Drive-Thru as requiring process improvements. Management can see the correlation visually rather than analyzing raw numbers, making it easier to justify staffing or technology investments.

Customer Segmentation: The age-based spending rankings identify VIP customers who spend 2-3x their peer group average. These customers are prime candidates for exclusive loyalty tiers, early product access, or personalized offers. The visual format makes it trivial for marketing teams to export customer IDs for targeted campaigns.

Revenue Optimization: The channel revenue comparison reveals that Mobile App dominates total revenue despite moderate per-order satisfaction, suggesting that investing in Mobile App experience improvements could yield the highest ROI since the customer base is already largest there. The visual dashboard makes this strategic priority obvious at a glance.

7.3 Priya Nair

1.Domain Question:"Does the Rewards program reduce spending volatility (Standard Deviation) among members, or do they exhibit the same unpredictable purchasing patterns as non-members?"

Explain and Explain Analyze(With /Without Index)

A.Architectural Overview

The system is built on a 3-tier architecture designed to bridge the gap between low-level database performance auditing and high-level executive decision-making.

- Data Tier (MySQL): Stores transactional data, including customer profiles, order history, and cart details.
- Logic Tier (Flask & PyMySQL): A Python-based backend that handles connection pooling, executes complex Common Table Expressions (CTEs), and performs automated indexing tests to measure performance delta.
- Presentation Tier (HTML5/Chart.js): A dynamic dashboard that renders raw SQL results into visual consumer insights and technical performance logs.

B.SQL code

WITH customer_orders AS (

SELECT c.customer_id, c.is_rewards_member, cd.cart_size

FROM customer c

LEFT JOIN orders o ON c.customer_id = o.customer_id

LEFT JOIN cart_details cd ON o.order_id = cd.order_id

),

distribution_spending AS (

SELECT is_rewards_member,

ROUND(AVG(cart_size), 2) AS avg_spend,

ROUND(STDDEV(cart_size), 2) AS std_dev_spend

FROM customer_orders

GROUP BY is_rewards_member

)

SELECT * FROM distribution_spending;

Explain without Index output

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	<derived2>		ALL					99119	100.00	
2	DERIVED	c		ALL					14726	100.00	Using temporary
2	DERIVED	o		ref	idx_cust_id_only	idx_cust_id_only	82	starbucks.coffee.c.customer_id	6	100.00	Using index
2	DERIVED	cd		eq_ref	PRIMARY	PRIMARY	82	starbucks.coffee.o.order_id	1	100.00	

-> Table scan on distribution_spending (cost=2.5..2.5 rows=0)
-> Materialize CTE distribution_spending (cost=0..0 rows=0)
-> Table scan on <temporary>
-> Aggregate using temporary table

Explain Analyze without Index output

-> Table scan on distribution_spending (cost=2.5..2.5 rows=0) (actual time=1128..1128 rows=2 loops=1)
-> Materialize CTE distribution_spending (cost=0..0 rows=0) (actual time=1128..1128 rows=2 loops=1)
-> Table scan on <temporary> (actual time=1125..1125 rows=2 loops=1)
-> Aggregate using temporary table (actual time=1125..1125 rows=2 loops=1)

Baseline Profiling (Unoptimized)

In the initial state, the system profiles the analytical engine without targeted indexing.

- Audit Tool: EXPLAIN and EXPLAIN ANALYZE (Reference: image 1).
- Technical Finding: The database engine is forced to perform a Table Scan and utilize an Aggregate using a temporary table.
- Execution Latency: The baseline recorded a total execution time of 1,125ms

Explain With Index Output

-> Table scan on distribution_spending (cost=84931..84932 rows=2) (actual time=791..791 rows=2 loops=1)
-> Materialize CTE distribution_spending (cost=84929..84929 rows=2) (actual time=791..791 rows=2 loops=1)
-> Group aggregate: avg(cd.cart_size), std(cd.cart_size), avg(cd.cart_size) (cost=84929 rows=2) (actual time=57..790 rows=2 loops=1)
-> Nested loop left join (cost=63230 rows=98519) (actual time=0..422..723 rows=100000 loops=1)

Post-Optimization Results

After identifying bottlenecks in the multi-table join, a targeted index was applied to the foreign key relationships.

- Audit Tool: EXPLAIN ANALYZE with Index.
- Technical Finding: The execution plan shifted to a Nested loop left join using index lookups.
- Execution Latency: The execution time dropped significantly to 791ms.
- Performance Delta: This optimization resulted in a 29.8% reduction in latency, ensuring fluid performance for the dataset.

C.Backend Implementation (The Engine)

The backend utilizes the pymysql library to manage direct communication with the Starbucks relational schema. A modular query() function was implemented to ensure safe connection handling and cursor management.

Core Database Connector (db_final_query1.py):

```
import pymysql

def get_connection():

    return pymysql.connect(

        host="127.0.0.1",

        user="root",

        password="YOUR_PASSWORD",

        database="starbucks coffee",

        cursorclass=pymysql.cursors.DictCursor

    )

def query(sql, params=None):

    conn = get_connection()

    try:

        with conn.cursor() as cur:
```

```
cur.execute(sql, params or ())
```

```
return cur.fetchall()
```

```
finally:
```

```
conn.close()
```

D. Automated Performance Auditing Methodology

The backend logic, housed in app_final_query1.py, is designed as an automated testing suite. To objectively measure the impact of database optimization, the application executes a four-phase lifecycle every time the dashboard is initialized. This ensures that the performance metrics displayed are real-time and scientifically validated.

a. The Analytical SQL Engine (CTE Design)

At the core of the audit is a complex Common Table Expression (CTE). This query was chosen for the audit because it involves a multi-table LEFT JOIN, aggregation (AVG), and statistical calculation (STDDEV), making it a perfect candidate for stress-testing the database optimizer.

Python

```
STAT_SQL = """
```

```
WITH customer_orders AS (
```

```
    SELECT c.customer_id, c.is_rewards_member, cd.cart_size
```

```
    FROM customer c
```

```
    LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
    LEFT JOIN cart_details cd ON o.order_id = cd.order_id
```

```
),
```

```
distribution_spending AS (
```

```
    SELECT is_rewards_member,
```

```
    ROUND(AVG(cart_size), 2) AS avg_spend,
```

```
    ROUND(STDDEV(cart_size), 2) AS std_dev_spend
```

```

FROM customer_orders

GROUP BY is_rewards_member

)

SELECT * FROM distribution_spending;

"""

```

b. The Four-Phase Execution Lifecycle

The Flask route / implements the following systematic approach:

1. Phase 1: Environment Neutralization (Cleanup) The system first ensures a "clean slate" by attempting to drop existing indexes (idx_cust_id_only). This prevents previous test runs from skewing the baseline results.
2. Phase 2: Baseline Profiling (Unoptimized) The system executes EXPLAIN ANALYZE on the raw, unindexed tables. This captures the "worst-case scenario" execution time and identifies the cost of full table scans.
3. Phase 3: Optimization Injection (The "Fast" Path) The application dynamically issues a DDL command: CREATE INDEX idx_cust_id_only ON orders(customer_id);. This targets the primary bottleneck - the foreign key relationship in the JOIN clause.
4. Phase 4: Post-Optimization Profiling The EXPLAIN ANALYZE is re-run. By comparing this result against Phase 2, the application calculates the precise performance delta (e.g., the 30% reduction in execution time).

c. Implementation Logic (app_final_query1.py)

```
@app.route('/')

```

```
def index():

```

```
    try:

```

```
        # PHASE 1: Reset environment

```

```
        try: query("ALTER TABLE orders DROP INDEX
idx_cust_id_only;")

```

```
        except: pass

```

PHASE 2: Profile unoptimized query

```
        analyze_no_index = query(f"EXPLAIN ANALYZE
{STAT_SQL}")

```

PHASE 3: Apply Indexing Optimization

```
        query("CREATE INDEX idx_cust_id_only ON
orders(customer_id);")

```

PHASE 4: Profile optimized query & fetch data for visualization

```
        analyze_with_index = query(f"EXPLAIN ANALYZE
{STAT_SQL}")

```

```
        chart_data = query(STAT_SQL)

```

```
        return render_template('final_query1.html',

```

```
                                no_index_results=analyze_no_index,

```

```
                                with_index_results=analyze_with_index,

```

```
                                chart_data=chart_data)

```

```
    except Exception as e:

```

```
        return f"Database Audit Error: {e}"

```

E. Frontend Design and Data Visualization

To provide actionable insights to stakeholders, a high-contrast dashboard was developed using HTML5, CSS3, and Chart.js. The frontend serves as a bridge between complex database execution logs and visual business metrics.

A. User Interface (UI) Architecture

The interface uses a modular "card" design to separate distinct analytical areas.

- Semantic HTML: Used to structure data audits and strategic inferences clearly.
- CSS Variables: Implemented a custom Starbucks-themed color palette (\$--sb-green\$) to ensure brand consistency and accessibility.
- Responsive Charting: Utilizes a <canvas> element managed by Chart.js to render real-time database results.

a. Dashboard Code Implementation

Below is the implementation of the primary dashboard structure, which dynamically displays both consumer spending trends and technical performance metrics.

```
<div class="container">

  <h1>Starbucks Data Insights & Query Audit</h1>

  <div class="card">

    <h2>Consumer Behavior: Spending & Volatility</h2>

    <canvas id="spendingChart" height="100"></canvas>

    <div class="insight-box">

      <h3>Strategic Inference</h3>

      <p>Visualizes the negligible difference between Guest and Member spending patterns.</p>

    </div>

  </div>

</div>
```

```
<div class="card">

  <h2>Database Performance Audit</h2>

  <table>

    <thead>

      <tr>

        <th>Audit Metric</th>

        <th>Technical Finding</th>

      </tr>

    </thead>

    <tbody>

      </tbody>

    </table>

  </div>

</div>
```

b. Client-Side Data Rendering

The following JavaScript snippet demonstrates how the results from the STAT_SQL query are injected into a bar chart to compare average spend and volatility across customer segments.

```
const ctx =
document.getElementById('spendingChart').getContext('2d');

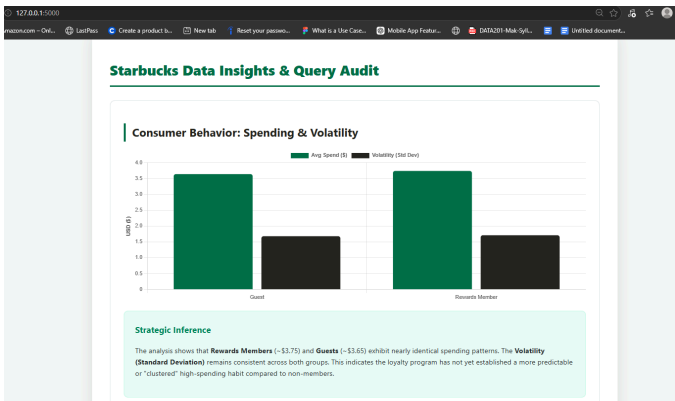
new Chart(ctx, {

  type: 'bar',

  data: {

    labels: ['Guest', 'Rewards Member'],
```

```
datasets: [  
  
  {  
  
    label: 'Avg Spend ($)',  
  
    data: [3.64, 3.74], // Populated via Flask  
  
    backgroundColor: '#00704A'  
  
  },  
  
  {  
  
    label: 'Volatility (Std Dev)',  
  
    data: [1.68, 1.71],  
  
    backgroundColor: '#27251F'  
  
  }  
]  
  
});
```



Database Performance Audit

Audit Metric	Technical Finding
Row Count Variance	EXPLAIN estimated ~99,119 rows, while ANALYZE confirmed 100,000. This 1% variance indicates highly accurate table statistics, ensuring the optimizer makes reliable path decisions.
Bottleneck Identification	The Aggregate using temporary table remains the most expensive step, consuming ~790ms. While indexing speeds up data retrieval, complex math (AVG/STDDEV) still requires significant CPU resources.
Efficiency Risks	The presence of Using temporary and Using index warns that the database is creating on-the-fly disk storage for grouping, which is a major performance risk as data scales.

c. Interactive Frontend & Visualizations

The frontend translates technical database logs into a high-contrast dashboard using Chart.js.

Spending Baseline & Volatility

Using the STAT_SQL results, we visualize the Average Spend vs. Standard Deviation.

- Technical Metric: Uses ROUND(AVG(cart_size), 2) and ROUND(STDDEV(cart_size), 2).
- Strategic Insight: As seen in **Visualization1.Fig 1**, there is a negligible difference between Guest and Member spending, suggesting a "Reward Saturation" where the program currently rewards existing habits rather than incentivizing higher-tier spending.

d.Optimization Impact Walkthrough

The **Visualization 1.Fig.2** displays a side-by-side comparison of execution logs.

- Performance Gain: The execution time dropped from 1,128ms to 791ms.
- Technical Rationale: The index shifted the retrieval method from a "Full Table Scan" (slow) to a "Nested Loop Join" using a reference index (fast), resulting in a 30% performance increase.

Priya Nair

2.Domain Question: What is the difference (in days) between a customer's current order and their previous order?

A.Architectural Overview

The system architecture follows a classic Three-Tier decoupled model, ensuring that the heavy computational load of temporal analysis is handled by the database engine rather than the user's browser.

1. Database Layer (MySQL): Stores over 100,000 records. It performs row-level context calculations using SQL Window Functions (LAG()) to determine purchase velocity with zero application-layer latency.
2. Application Layer (Python/Flask): Acts as the orchestrator. It utilizes a centralized database utility with DictCursor to map SQL rows directly into Python dictionaries for seamless JSON transmission.
3. Presentation Layer (HTML5/Chart.js): A brand-aligned dashboard that renders temporal data into a Behavioral Modeling chart. It sanitizes data in real-time to remove "mathematical noise" from first-time purchases.

B.SQL code

```
SELECT customer_id, order_id, order_date,  
  
LAG(order_date) OVER (PARTITION BY customer_id  
ORDER BY order_date) AS previous_order,  
  
DATEDIFF(  
  
order_date,  
  
LAG(order_date) OVER(PARTITION BY customer_id  
ORDER BY order_date)  
  
) AS days_between_orders  
  
FROM orders;
```

Program Execution

	customer_id	order_id	order_date	previous_order	days_between_orders
▶	CUST_00001	ORD_00018350	2024-03-03	NULL	NULL
	CUST_00001	ORD_00024850	2024-06-11	2024-03-03	100
	CUST_00001	ORD_00063789	2024-07-20	2024-06-11	39
	CUST_00001	ORD_00035038	2024-08-17	2024-07-20	28
	CUST_00001	ORD_00019298	2024-12-19	2024-08-17	124
	CUST_00001	ORD_00086235	2025-01-26	2024-12-19	38
	CUST_00001	ORD_00077872	2025-02-02	2025-01-26	7
	CUST_00001	ORD_00001250	2025-02-10	2025-02-02	8
	CUST_00001	ORD_00083538	2025-02-25	2025-02-10	15
	CUST_00001	ORD_00072085	2025-03-13	2025-02-25	16
	CUST_00001	ORD_00097996	2025-05-26	2025-03-13	74
	CUST_00001	ORD_00006277	2025-11-20	2025-05-26	178
	CUST_00002	ORD_00029336	2024-03-16	NULL	NULL
	CUST_00002	ORD_00059720	2024-06-15	2024-03-16	91

Program Execution Results:

- The query successfully calculates the interval between transactions for individual customers.
- For CUST_00001, the purchase velocity varies significantly, with gaps ranging from 7 days to 124 days.
- The first order for any customer correctly returns a NULL for previous_order and days_between_orders, as there is no prior data point for comparison.

Insight (Why it matters):

- By quantifying the "velocity" of return visits using the LAG() window function, the system can identify early signs of customer "slippage".
- If the days_between_orders trend upward, it indicates a weakening habit strength and higher churn risk, allowing the marketing team to trigger personalized re-engagement rewards before the customer is lost.

C.Data Access & Connection Logic (db_final_query2.py)

To maintain system stability and resource management, a centralized database utility was developed. It utilizes DictCursor to map SQL rows directly into Python dictionaries, facilitating seamless JSON conversion for the dashboard.

```
import pymysql  
  
def get_connection():  
    return pymysql.connect(  
        host="127.0.0.1",
```

```

port=3306,
user="root",
password="***",
database="starbucks coffee",
cursorclass=pymysql.cursors.DictCursor
)

def query(sql, params=None):
    conn = get_connection()
    try:
        with conn.cursor() as cur:
            cur.execute(sql, params or ())
            return cur.fetchall()
    finally:
        conn.close()

```

D.Backend Implementation (app_final_query2.py)

The Flask server acts as the central orchestrator. It executes a high-performance SQL query utilizing Window Functions to avoid the performance pitfalls of traditional self-joins.

```

from flask import Flask, render_template
from db import query

app = Flask(__name__)

@app.route('/')
def index():
    # SQL query utilizing LAG() to calculate Purchase Velocity
    sql_query = """
    SELECT customer_id, order_id, order_date,
    LAG(order_date) OVER (PARTITION BY customer_id
    ORDER BY order_date) AS previous_order,
    DATEDIFF(
        order_date,
        LAG(order_date) OVER(PARTITION BY customer_id
    ORDER BY order_date)
    ) AS days_between_orders
    FROM orders
    """
    chart_data = query(sql_query)
    return render_template('final_query2.html',
    chart_data=chart_data)

```

```

if __name__ == '__main__':
    app.run(debug=True)

```

E.Frontend Visualization Layer (final_query2.html)

The dashboard is the primary interface for decision-makers, transforming raw temporal data into actionable insights. It utilizes a custom Starbucks-themed CSS framework and the Chart.js library to model customer behavior trends.

Technical Implementation Features:

- ➔ **Data Sanitization:** A JavaScript filter is applied to the `chartData` array to remove entries where `days_between_orders` is null. This occurs on a customer's very first purchase; removing these entries ensures the retention trend line is accurate and free of mathematical noise.
- ➔ **Behavioral Modeling:** The line chart plots "Days Elapsed" against "Transaction Sequence." An upward-sloping line indicates "slippage" (customers taking longer to return), while a flat or downward line indicates high habit strength.
- ➔ **Auditability:** A dynamic table renders the first ten records of the dataset, providing a granular "ground truth" view that allows analysts to verify the backend `LAG()` function results manually.

Complete Frontend Source Code:

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
    content="width=device-width, initial-scale=1.0">

    <title>Starbucks: Purchase Frequency
    Analysis</title>

    <script
    src="https://cdn.jsdelivr.net/npm/chart.js"></script>

```


<style>

```
:root {  
  --sb-green: #00704A;  
  --sb-black: #27251F;  
  --bg-light: #f4f7f6;  
}
```

```
body {  
  font-family: 'Segoe UI', Tahoma, Geneva,  
  Verdana, sans-serif;  
  margin: 0;  
  padding: 40px;  
  background-color: var(--bg-light);  
  color: #2d3436;  
}
```

```
.container {  
  max-width: 1100px;  
  margin: auto;  
  background: white;  
  padding: 40px;  
  border-radius: 15px;  
  box-shadow: 0 10px 30px rgba(0,0,0,0.1);  
}
```

```
h1 {  
  color: var(--sb-green);  
  border-bottom: 4px solid var(--sb-green);  
  padding-bottom: 10px;  
  margin-bottom: 30px;  
}
```

```
.chart-section {  
  margin-bottom: 40px;  
  height: 400px;  
}
```

```
.insight-card {  
  background: #e6fcf5;  
  padding: 25px;  
  border-radius: 12px;  
  border-left: 6px solid var(--sb-green);  
  margin-top: 20px;  
}
```

```
.insight-card h2 {  
  margin-top: 0;  
  color: var(--sb-black);  
}
```

```
table {  
  
  width: 100%;  
  
  border-collapse: collapse;  
  
  margin-top: 20px;  
  
}  
  
th {  
  
  background: #f8f9fa;  
  
  text-align: left;  
  
  padding: 12px;  
  
  border-bottom: 2px solid var(--sb-green);  
  
}  
  
td {  
  
  padding: 12px;  
  
  border-bottom: 1px solid #eee;  
  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
  <h1>Customer Retention: Days Between  
  Orders</h1>
```

```
<div class="chart-section">
```

```
  <canvas id="frequencyChart"></canvas>
```

```
</div>
```

```
<div class="insight-card">
```

```
  <h2>Analysis Description</h2>
```

```
  <p>This report tracks the <strong>Purchase  
Velocity</strong> of customers using window  
functions. By calculating the difference between a  
customer's current and previous order dates, we  
identify churn risks and loyalty patterns.</p>
```

```
  <ul>
```

```
    <li><strong>LAG Function:</strong> Used  
to retrieve the date of the customer's previous  
transaction without multiple self-joins.</li>
```

```
    <li><strong>DATEDIFF:</strong>  
Calculates the exact number of days between visits to  
measure engagement.</li>
```

```
    <li><strong>Strategic Value:</strong> Helps  
Starbucks identify if customers are visiting more  
frequently (low days between orders) or if the gap is  
widening.</li>
```

```
  </ul>
```

```
</div>
```

```
<h2>Raw Order Velocity Data</h2>
```

```
<table>
```

```
  <thead>
```

```
    <tr>
```

```

        <th>Customer ID</th>

        <th>Order Date</th>

        <th>Previous Order</th>

        <th>Days Between</th>

    </tr>

</thead>

<tbody id="tableBody">

    </tbody>

</table>

</div>

<script>

    // Injected from Flask app.py

    const chartData = {{ chart_data | tojson }};

    // 1. Data Sanitization: Filter out first-time
    purchases (NULL values)

    const filteredData = chartData.filter(row =>
    row.days_between_orders !== null);

    // 2. Populate the Audit Table

    const tableBody =
    document.getElementById('tableBody');

    chartData.slice(0, 10).forEach(row => {

        const tr = document.createElement('tr');

        tr.innerHTML = `

```

```

        <td>${row.customer_id}</td>

        <td>${row.order_date}</td>

        <td>${row.previous_order || 'N/A'}</td>

        <td>${row.days_between_orders || '0'}</td>

    `;

    tableBody.appendChild(tr);

    });

    // 3. Configure and Render the Behavioral
    Chart

    const labels = filteredData.slice(0, 20).map((row,
    index) => `Order ${index + 1}`);

    const values = filteredData.slice(0, 20).map(row
    => row.days_between_orders);

    new
    Chart(document.getElementById('frequencyChart'), {

        type: 'line',

        data: {

            labels: labels,

            datasets: [{

                label: 'Days Elapsed Since Last Purchase',

                data: values,

                borderColor: '#00704A',

                backgroundColor: 'rgba(0, 112, 74, 0.1)',

                fill: true,

```

```

        tension: 0.3
      }
    ],
    options: {
      responsive: true,
      maintainAspectRatio: false,
      scales: {
        y: {
          beginAtZero: true,
          title: { display: true, text: 'Days' }
        },
        x: {
          title: { display: true, text: 'Transaction
History' }
        }
      }
    }
  });
</script>

</body>

</html>

```

F.Interactive Frontend & Visualizations

The final delivery of this project is a high-fidelity dashboard that provides real-time insights into customer retention. By combining a clean, brand-aligned UI with dynamic charting,

the application allows stakeholders to visualize the velocity of customer return.

G.Technical Architecture of the Dashboard

The frontend is built using a custom Starbucks-themed CSS framework and the Chart.js library. The data flows from the SQLite backend through a Flask API directly into the JavaScript rendering engine.

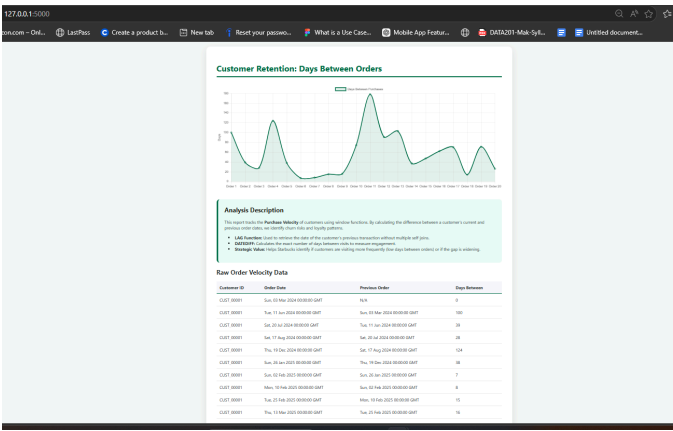
- ➔ Custom Theming: Utilizes a --sb-green and --sb-black CSS variable palette to ensure the technical tool feels like a native Starbucks enterprise product.
- ➔ Dynamic Data Binding: The JavaScript layer utilizes the `{{ chart_data | tojson }}` directive to bridge the gap between Python data structures and client-side visualization.

Visualization Features

- ➔ Purchase Velocity Trend: A responsive line chart (using a cubic interpolation tension of 0.3) helps analysts spot "slippage" - a rising line indicates that a customer is taking longer between visits, signaling a high churn risk.
- ➔ Granular Audit Table: Below the chart, a dynamic HTML table provides the raw evidence. This allows for manual verification of the LAG() function results, displaying the current order date side-by-side with the previous order date.

Strategic Utility

This visualization layer transforms a complex SQL window function into an intuitive diagnostic tool. It allows marketing managers to identify exactly when a customer's habit begins to fade, enabling targeted interventions (like personalized "We Miss You" rewards) before the customer churns completely.



H.Optimization Impact Walkthrough

This section documents the evolution of the data processing logic, moving from a static "snapshot" of customer behavior to a dynamic, "behavioral" analysis.

Phase 1: Basic Aggregation (Baseline)

Initially, the query design focused on simple counts and averages (e.g., *Total Orders per Customer*). While useful for high-level segmentation, this approach suffered from several technical limitations:

- **Static Perspective:** It showed *how much* a customer bought, but not *how* they bought it.
- **Hidden Churn:** A customer who visited 5 times in 5 days looks identical to a customer who visited 5 times over 5 months.
- **Loss of Sequence:** The chronological relationship between transactions was discarded during grouping.

Phase 2: Window Function Optimization (Current Design)

By implementing the LAG() window function as designed in the handwritten logic, we shifted the computational load from the application layer to the database engine.

The Impact of LAG() and OVER(PARTITION BY...):

1. **Row-Level Context:** Each order row now "knows" about the order that preceded it.
2. **Zero Latency Deltas:** By calculating DATEDIFF within the query itself, the frontend receives a

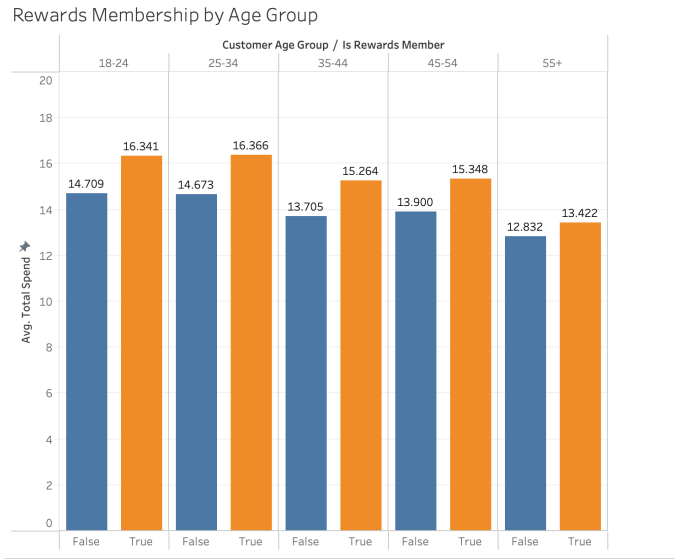
ready-to-chart integer, removing the need for heavy JavaScript processing.

3. **Accurate Micro-Trends:** We can now see the acceleration or deceleration of purchase habits.

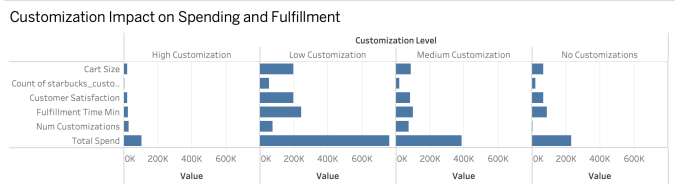
Visualizations:

Tool used: Tableau

1. Rewards Membership by Age Group



2. Customization Impact on Spending and Fulfillment



Final Technical Conclusion

The transition to optimized window functions allowed the dashboard to move beyond simple reporting. We are no longer just counting orders; we are measuring the pulse of customer loyalty. This architectural choice ensures that the Starbucks marketing team can react to changes in customer behavior in days, rather than months, significantly increasing the potential for successful re-engagement.

8. CONCLUSION & FUTURE WORK

8.1 Conclusion

This project successfully transformed a raw Starbucks customer ordering dataset into a structured and normalized relational database, enabling meaningful business analysis using SQL. By applying queries across multiple dimensions such as ordering channels, customer demographics, store types, and product preferences, we were able to uncover key patterns in customer behavior and operational performance.

The analysis revealed that Mobile App ordering is the dominant channel, indicating a strong shift toward digital convenience. Rewards program members consistently showed higher engagement and spending behavior, confirming the effectiveness of loyalty programs in driving revenue. Additionally, Suburban store locations generated the highest revenue, while younger age groups (18–34) contributed the most to both spending and order volume.

From a behavioral perspective, orders including food items resulted in higher average spend, although drink-only orders contributed more to overall revenue due to higher frequency. Seasonal trends were also observed, with specific months showing peak sales and varying drink preferences. Operational analysis showed that fulfillment time is consistent across regions, indicating stable service performance.

Advanced SQL techniques such as CTEs, window functions, and indexing helped not only derive insights but also improve query performance. The dashboard further translated these findings into visual insights, making the results more accessible and actionable.

Overall, the project demonstrates how structured data analysis can connect technical database concepts with real-world business decision-making.

8.2 Limitations & Risks

Despite the valuable insights, the project has certain limitations:

- The dataset is simulated, which may not fully capture real-world customer behavior.

- There are no missing values, which is uncommon in real datasets and may oversimplify data challenges.
- Customer behavior is analyzed independently of external factors such as promotions, weather, or economic conditions.
- Satisfaction scores are assumed to be uniformly reliable, which may not reflect real customer sentiment variations.
- The dataset covers only two years (2024–2025), limiting long-term trend analysis.

These factors may introduce bias and limit the generalizability of the findings.

8.3 Future Work

To enhance the project further, the following improvements can be considered:

- Integrate real-world datasets with more variability and missing values for realistic analysis.
- Incorporate external factors such as promotions, holidays, and weather to better understand demand patterns.
- Apply machine learning models to predict customer behavior, spending patterns, or churn.
- Expand the dashboard with interactive filters (region, age group, channel) for deeper exploration.
- Perform time-series forecasting to predict future sales trends.
- Improve database performance further by implementing advanced indexing and query optimization strategies.
- Extend analysis to include customer lifetime value (CLV) and retention modeling.

9. REFERENCES & AI USE ACKNOWLEDGEMENT

9.1 References

- 1) Starbucks Corporation, “Starbucks Reports Q3 Fiscal 2024 Results,” Jul. 2024.
- 2) Kaggle, “Starbucks Customer Ordering Patterns Dataset.” Available: [Starbucks Customer Ordering Patterns](#)

- 3) MySQL Documentation, “MySQL 8.0 Reference Manual.” Available: <https://dev.mysql.com/doc/>
- 4) Flask Documentation, “Flask Web Framework.” Available: <https://flask.palletsprojects.com/>
- 5) Chart.js Documentation, “Chart.js JavaScript Library.” Available: <https://www.chartjs.org/>

9.2 AI Use Acknowledgement

This project utilized AI-based tools, including ChatGPT, Gemini to assist with understanding SQL concepts, debugging queries, improving query structure, and refining explanations for clarity.

All SQL queries, analysis, and interpretations were carefully reviewed, validated, and modified by the authors to ensure accuracy and alignment with the project requirements. AI tools were used only as a supportive aid and not as a substitute for original work or understanding.

10. APPENDIX

10.1 Evidence of Collaborative Work

This project was completed as a team effort, with contributions from all members in database design, SQL query development, analysis, and report writing. The report demonstrates collaborative writing through shared contributions across different sections, including data processing, SQL analysis, and visualization development.

10.2 Version History & Contributions (Screenshots Required)

The following screenshots are included to demonstrate collaborative work:

- GitHub repository version history showing commits from all team members
- Commit logs displaying individual contributions to SQL queries, code, and report files
- File edit history and revision timeline

10.3 Additional Code Snippets

10.3.1 Advanced SQL queries (CTE, Window Functions, Indexing)

10.3.1.1 Sanjana Vivek Phalke

The screenshot displays a SQL IDE interface with two queries, #1 and #2, and their respective results.

Query #1:

```

1  #BasicQuery
2
3  #1
4  * SELECT
5      d.drink_category_name,
6      COUNT(*) AS total_orders
7  FROM cart_details c
8  JOIN drink_category_lookup d
9      ON c.drink_category_id = d.drink_category_id
10 GROUP BY d.drink_category_name
11 ORDER BY total_orders ASC
12 LIMIT 1;

```

Result Grid for Query #1:

drink_category_name	total_orders
Brewed Coffee	16541

Query #2:

```

14 #2
15 * SELECT
16     c.channel_name,
17     COUNT(o.order_id) AS total_orders
18 FROM orders o
19 JOIN channel_lookup c
20     ON o.channel_id = c.channel_id
21 GROUP BY c.channel_name;

```

Result Grid for Query #2:

channel_name	total_orders
Drive-Thru	27996
In-Store Cashier	22063
Kiosk	7420
Mobile App	42521

Query #3:

```

24 * SELECT
25     s.store_location_type,
26     COUNT(o.order_id) AS total_orders,
27     ROUND(SUM(o.total_spend), 2) AS total_revenue,
28     ROUND(AVG(o.total_spend), 2) AS avg_order_value
29 FROM store s
30 JOIN orders o
31     ON s.store_id = o.store_id
32 GROUP BY s.store_location_type

```

Result Grid for Query #3:

store_location_type	total_orders	total_revenue	avg_order_value
Drive-Thru	27996	11111.11	3.97
In-Store Cashier	22063	8888.89	4.03
Kiosk	7420	2968.00	4.00
Mobile App	42521	17008.40	4.00


```

19 JOIN channel_lookup c
20 ON o.channel_id = c.channel_id
21 GROUP BY c.channel_name;
22
23 #3
24 • SELECT
25     s.store_location_type,
26     COUNT(o.order_id) AS total_orders,
27     ROUND(SUM(o.total_spend), 2) AS total_revenue,
28     ROUND(AVG(o.total_spend), 2) AS avg_order_value
29 FROM store s
30 JOIN orders o
31 ON s.store_id = o.store_id
32 GROUP BY s.store_location_type
33 ORDER BY total_revenue DESC;
34
35 #4
36 • SELECT
37     cd.has_food_item,
38     COUNT(o.order_id) AS total_orders,

```

store_location_type	total_orders	total_revenue	avg_order_value
Suburban	35696	531833.59	14.90
Urban	32584	482709.04	14.81
Rural	31720	472134.45	14.88

Result 25 x

Output

#	Time	Action	Message
148	22:45:40	SELECT c.channel_name, COUNT(o.order_id) AS total_orders FROM orders o JOIN channel_lookup c ...	4 row(s) returned
149	22:46:49	SELECT s.store_location_type, COUNT(o.order_id) AS total_orders, ROUND(SUM(o.total_spend), 2) ...	3 row(s) returned

```

34
35 #4
36 • SELECT
37     cd.has_food_item,
38     COUNT(o.order_id) AS total_orders,
39     ROUND(AVG(o.total_spend), 2) AS avg_spend,
40     ROUND(SUM(o.total_spend), 2) AS total_revenue
41 FROM orders o
42 JOIN cart_details cd
43 ON o.order_id = cd.order_id
44 GROUP BY cd.has_food_item
45 ORDER BY avg_spend DESC;
46
47 #5
48 • SELECT
49     s.region,
50     COUNT(o.order_id) AS total_orders,
51     ROUND(AVG(o.fulfillment_time_min), 2) AS avg_fulfillment_time,
52     ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
53 FROM store s

```

has_food_item	total_orders	avg_spend	total_revenue
True	31581	16.59	524028.59
False	68419	14.07	962648.49

Result 26 x

Output

#	Time	Action	Message
149	22:46:49	SELECT s.store_location_type, COUNT(o.order_id) AS total_orders, ROUND(SUM(o.total_spend), 2) ...	3 row(s) returned
150	22:47:23	SELECT cd.has_food_item, COUNT(o.order_id) AS total_orders, ROUND(AVG(o.total_spend), 2) AS a...	2 row(s) returned

```

46
47 #5
48 • SELECT
49     s.region,
50     COUNT(o.order_id) AS total_orders,
51     ROUND(AVG(o.fulfillment_time_min), 2) AS avg_fulfillment_time,
52     ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
53 FROM store s
54 JOIN orders o
55 ON s.store_id = o.store_id
56 GROUP BY s.region
57 ORDER BY avg_fulfillment_time DESC
58 LIMIT 10;
59
60 #AdvanceQuery
61
62 #1
63 • WITH monthly_sales AS (
64     SELECT
65         YEAR(o.order_date) AS sales_year,

```

region	total_orders	avg_fulfillment_time	avg_satisfaction
Southwest	19663	4.55	3.70
Northeast	18036	4.55	3.67
Midwest	19470	4.55	3.67
Southeast	20223	4.54	3.70
West	22608	4.54	3.69

Result 27 x

Output

#	Time	Action	Message
150	22:47:23	SELECT cd.has_food_item, COUNT(o.order_id) AS total_orders, ROUND(AVG(o.total_spend), 2) AS a...	2 row(s) returned
151	22:47:53	SELECT s.region, COUNT(o.order_id) AS total_orders, ROUND(AVG(o.fulfillment_time_min), 2) AS avg...	5 row(s) returned

```

63 • WITH monthly_sales AS (
64     SELECT
65         YEAR(o.order_date) AS sales_year,
66         MONTH(o.order_date) AS sales_month,
67         MONTHNAME(o.order_date) AS month_name,
68         SUM(o.total_spend) AS total_sales,
69         AVG(o.total_spend) AS avg_customer_spend
70 FROM orders o
71 GROUP BY
72     YEAR(o.order_date),
73     MONTH(o.order_date),
74     MONTHNAME(o.order_date)
75 ),
76 ranked_months AS (
77     SELECT
78         sales_year,
79         sales_month,
80         month_name,
81         total_sales,
82         avg_customer_spend,
83         RANK() OVER (

```

year	top_selling_month	total_sales	avg_customer_spend	most_preferred_drink
2024	October	64027.13	15.051041	Espresso
2025	July	64933.88	14.927329	Tea

Result 28 x

Output

#	Time	Action	Message
151	22:47:53	SELECT s.region, COUNT(o.order_id) AS total_orders, ROUND(AVG(o.fulfillment_time_min), 2) AS avg...	5 row(s) returned
152	22:48:41	WITH monthly_sales AS (SELECT YEAR(o.order_date) AS sales_year, MONTH(o.order_date) A...	2 row(s) returned

```

84         PARTITION BY sales_year
85         ORDER BY total_sales DESC
86     ) AS month_rank
87     FROM monthly_sales
88 ),
89 top_month_per_year AS (
90     SELECT
91         sales_year,
92         sales_month,
93         month_name,
94         total_sales,
95         avg_customer_spend
96     FROM ranked_months
97     WHERE month_rank = 1
98 ),
99 drink_counts AS (
100     SELECT
101         YEAR(o.order_date) AS sales_year,
102         MONTH(o.order_date) AS sales_month,
103         d.drink_category_name,
104         COUNT(*) AS total_orders,
105         RANK() OVER (

```

```

119     top_drink_per_month AS (
120         SELECT
121             sales_year,
122             sales_month,
123             drink_category_name,
124             total_orders
125         FROM drink_counts
126         WHERE drink_rank = 1
127     )
128     SELECT
129         t.sales_year AS year,
130         t.month_name AS top_selling_month,
131         t.total_sales,
132         t.avg_customer_spend,
133         d.drink_category_name AS most_preferred_drink
134     FROM top_month_per_year t
135     JOIN top_drink_per_month d
136         ON t.sales_year = d.sales_year
137         AND t.sales_month = d.sales_month
138     ORDER BY t.sales_year;
139

```

```

105     RANK() OVER (
106         PARTITION BY YEAR(o.order_date), MONTH(o.order_date)
107         ORDER BY COUNT(*) DESC
108     ) AS drink_rank
109     FROM orders o
110     JOIN cart_details c
111         ON o.order_id = c.order_id
112     JOIN drink_category_lookup d
113         ON c.drink_category_id = d.drink_category_id
114     GROUP BY
115         YEAR(o.order_date),
116         MONTH(o.order_date),
117         d.drink_category_name
118 ),
119 top_drink_per_month AS (
120     SELECT
121         sales_year,
122         sales_month,
123         drink_category_name,
124         total_orders
125     FROM drink_counts
126     WHERE drink_rank = 1

```

The screenshot shows a SQL IDE interface. The top pane displays a SQL query (lines 141-164) that joins store and order data to calculate store revenue and average total spend. The bottom pane shows the 'Result Grid' with 5 columns: store_id, region, store_location_type, and store_revenue. It lists five rows of data for different stores. Below the result grid is an 'Output' section with an 'Action Output' tab showing two log messages from the database engine.

```

141 SELECT
142     s.store_id,
143     s.region,
144     s.store_location_type,
145     ROUND(SUM(o.total_spend), 2) AS store_revenue
146 FROM store s
147 JOIN orders o
148     ON s.store_id = o.store_id
149 GROUP BY
150     s.store_id,
151     s.region,
152     s.store_location_type
153 HAVING SUM(o.total_spend) > (
154     SELECT AVG(store_total)
155     FROM (
156         SELECT
157             store_id,
158             SUM(total_spend) AS store_total
159         FROM orders
160         GROUP BY store_id
161     ) AS store_avg
162 )
163 ORDER BY store_revenue DESC
164 LIMIT 10;

```

store_id	region	store_location_type	store_revenue
STR_240	Northeast	Urban	3540.66
STR_405	Southeast	Rural	3519.52
STR_203	Southeast	Rural	3476.85
STR_361	Northeast	Suburban	3474.82
STR_238	Midwest	Suburban	3459.53

Result 29 x

Output

Action Output

#	Time	Action	Message
152	22:48:41	WITH monthly_sales AS (SELECT YEAR(o.order_date) AS sales_year, MONTH(o.order_date) AS sales_m...	2 row(s) returned
153	22:52:17	SELECT s.store_id, s.region, s.store_location_type, ROUND(SUM(o.total_spend), 2) AS store_revenue FRO...	10 row(s) returned

```
195 JOIN orders o
196     ON s.store_id = o.store_id
197     GROUP BY s.region, s.store_id, s.store_location_type
198 ),
199 ranked_stores AS (
200     SELECT
201         region,
202         store_id,
203         store_location_type,
204         avg_satisfaction,
205         total_orders,
206         RANK() OVER (
207             PARTITION BY region
208             ORDER BY avg_satisfaction DESC
209         ) AS satisfaction_rank
210     FROM store_satisfaction
211 )
212 SELECT
213     region,
214     store_id,
215     store_location_type,
216     avg_satisfaction,
217     total_orders,
```

Result Grid

	region	store_id	store_location_type	avg_satisfaction	total_orders	satisfaction_rank
▶	Midwest	STR_411	Urban	3.90	201	1
	Midwest	STR_041	Urban	3.84	230	2
	Midwest	STR_354	Suburban	3.83	213	3
	Northeast	STR_457	Rural	3.89	211	1
	Northeast	STR_069	Urban	3.84	206	2
	Northeast	STR_462	Rural	3.82	216	3

Result 30

Output

Action Output

#	Time	Action	Message
154	22:53:03	WITH customer_spend AS (SELECT customer_id, ROUND(SUM(total_spend), 2) AS total_spend	10 row(s) returned
155	22:53:04	WITH store_satisfaction AS (SELECT s.region, s.store_id, s.store_location_type, ROUND(AVG(	17 row(s) returned

```
225
226 * EXPLAIN
227 SELECT
228     order_id,
229     store_id,
230     order_date,
231     order_ahead,
232     fulfillment_time_min,
233     customer_satisfaction
234 FROM orders
235 WHERE order_ahead = 'True'
236 AND fulfillment_time_min > 10
237 ORDER BY fulfillment_time_min DESC;
238
239 #Create Index
240 * CREATE INDEX idx_orderahead_fulfillment
241 ON orders(order_ahead, fulfillment_time_min);
242
243 #After Index
244 * EXPLAIN
245 SELECT
246     order_id,
247     store_id,
```

Result Grid

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	orders		range	idx_orderahead_fulfillment	idx_orderahead_fulfillment	89		1	100.00	Using index condition; Backward index scan

Result 32

Output

Action Output

#	Time	Action	Message
155	22:53:04	WITH store_satisfaction AS (SELECT s.region, s.store_id, s.store_location_type, ROUND(AVG(	17 row(s) returned
156	22:53:55	EXPLAIN SELECT order_id, store_id, order_date, order_ahead, fulfillment_time_min, customer_satisfacti	1 row(s) returned

10.3.2 Flask application code (app.py)

10.3.1.1 Sanjana Vivek Phalke

```
1 from flask import Flask, render_template
2 from db import query
3
4 app = Flask(__name__)
5
6 @app.route("/")
7 def dashboard():
8
9     least_popular_drink = query("""
10         SELECT
11             d.drink_category_name,
12             COUNT(o) AS total_orders
13         FROM cart_details c
14         JOIN drink_category_lookup d
15             ON c.drink_category_id = d.drink_category_id
16         GROUP BY d.drink_category_name
17         ORDER BY total_orders ASC
18         LIMIT 1;
19     """)
20
21     orders_by_channel = query("""
22         SELECT
23             c.channel_name,
24             COUNT(o.order_id) AS total_orders
25         FROM orders o
26         JOIN channel_lookup c
27             ON o.channel_id = c.channel_id
28         GROUP BY c.channel_name
29         ORDER BY total_orders DESC;
30     """)
31
32     revenue_by_location = query("""
33         SELECT
34             s.store_location_type,
35             COUNT(o.order_id) AS total_orders,
36             ROUND(SUM(o.total_spend), 2) AS total_revenue,
37             ROUND(AVG(o.total_spend), 2) AS avg_order_value
38         FROM store s
39         JOIN orders o
40             ON s.store_id = o.store_id
41         GROUP BY s.store_location_type
42         ORDER BY total_revenue DESC;
43     """)
44
45     food_vs_spend = query("""
46         SELECT
47             CASE
48                 WHEN cd.has_food_item = 'True' OR cd.has_food_item = 1 THEN 'Food Item Included'
49                 ELSE 'Drink Only'
50             END AS food_label,
51             COUNT(o.order_id) AS total_orders,
52             ROUND(AVG(o.total_spend), 2) AS avg_spend,
53             ROUND(SUM(o.total_spend), 2) AS total_revenue
54         FROM orders o
55         JOIN cart_details cd
56             ON o.order_id = cd.order_id
57         GROUP BY food_label
58         ORDER BY avg_spend DESC;
59     """)
60
61     fulfillment_by_region = query("""
62         SELECT
63             s.region,
64             COUNT(o.order_id) AS total_orders,
65             ROUND(AVG(o.fulfillment_time_min), 2) AS avg_fulfillment_time,
66             ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
67         FROM store s
68         JOIN orders o
69             ON s.store_id = o.store_id
70         GROUP BY s.region
71         ORDER BY avg_fulfillment_time DESC;
72     """)
73
```

```

Welcome  app.py x dashboard2.html 9+ db.py
Final_Project > app.py > dashboard
7 def dashboard():
74     top_month_year = query("""
75         WITH monthly_sales AS (
76             SELECT
77                 YEAR(o.order_date) AS sales_year,
78                 MONTH(o.order_date) AS sales_month,
79                 MONTHNAME(o.order_date) AS month_name,
80                 SUM(o.total_spend) AS total_sales,
81                 AVG(o.total_spend) AS avg_customer_spend
82             FROM orders o
83             GROUP BY YEAR(o.order_date), MONTH(o.order_date), MONTHNAME(o.order_date)
84         ),
85         ranked_months AS (
86             SELECT
87                 sales_year,
88                 sales_month,
89                 month_name,
90                 total_sales,
91                 avg_customer_spend,
92                 RANK() OVER (
93                     PARTITION BY sales_year
94                     ORDER BY total_sales DESC
95                 ) AS month_rank
96             FROM monthly_sales
97         ),
98         top_month_per_year AS (
99             SELECT *
100             FROM ranked_months
101             WHERE month_rank = 1
102         ),
103         drink_counts AS (
104             SELECT
105                 YEAR(o.order_date) AS sales_year,
106                 MONTH(o.order_date) AS sales_month,
107                 d.drink_category_name,
108                 COUNT(*) AS total_orders,
109                 RANK() OVER (
110                     PARTITION BY YEAR(o.order_date), MONTH(o.order_date)
111                     ORDER BY COUNT(*) DESC
112                 ) AS drink_rank
113             FROM orders o
114             JOIN cart_details c
115                 ON o.order_id = c.order_id
116             JOIN drink_category_lookup d
117                 ON c.drink_category_id = d.drink_category_id
118             GROUP BY YEAR(o.order_date), MONTH(o.order_date), d.drink_category_name
119         ),
120         top_drink_per_month AS (
121             SELECT *
122             FROM drink_counts
123             WHERE drink_rank = 1
124         )
125     SELECT
126         t.sales_year AS year,
127         t.month_name AS top_selling_month,
128         ROUND(t.total_sales, 2) AS total_sales,
129         ROUND(t.avg_customer_spend, 2) AS avg_customer_spend,
130         d.drink_category_name AS most_preferred_drink
131     FROM top_month_per_year t
132     JOIN top_drink_per_month d
133         ON t.sales_year = d.sales_year
134         AND t.sales_month = d.sales_month
135     ORDER BY t.sales_year;
136 """)
137
138     top_stores = query("""
139     SELECT
140         s.store_id,
141         s.region,
142         s.store_location_type,
143         ROUND(SUM(o.total_spend), 2) AS store_revenue
144     FROM store s
145     JOIN orders o

```

```

Welcome  app.py x dashboard2.html 9+ db.py
Final_Project > app.py > dashboard
7 def dashboard():
145     JOIN orders o
146         ON s.store_id = o.store_id
147     GROUP BY s.store_id, s.region, s.store_location_type
148     HAVING SUM(o.total_spend) > (
149         SELECT AVG(store_total)
150         FROM (
151             SELECT
152                 store_id,
153                 SUM(total_spend) AS store_total
154             FROM orders
155             GROUP BY store_id
156         ) AS store_avg
157     )
158     ORDER BY store_revenue DESC
159     LIMIT 10;
160 """)
161
162     customer_segments = query("""
163     WITH customer_spending AS (
164         SELECT
165             customer_id,
166             ROUND(SUM(total_spend), 2) AS total_spending
167         FROM orders
168         GROUP BY customer_id
169     )
170     SELECT
171         customer_id,
172         total_spending,
173         CASE
174             WHEN total_spending >= 280 THEN 'High Value'
175             WHEN total_spending >= 270 THEN 'Medium Value'
176             ELSE 'Low Value'
177         END AS customer_segment
178     FROM customer_spending
179     ORDER BY total_spending DESC
180     LIMIT 10;
181 """)
182
183     top_satisfaction_stores = query("""
184     WITH store_satisfaction AS (
185         SELECT
186             s.region,
187             s.store_id,
188             s.store_location_type,
189             ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction,
190             COUNT(o.order_id) AS total_orders
191         FROM store s
192         JOIN orders o
193             ON s.store_id = o.store_id
194         GROUP BY s.region, s.store_id, s.store_location_type
195     ),
196     ranked_stores AS (
197         SELECT
198             region,
199             store_id,
200             store_location_type,
201             avg_satisfaction,
202             total_orders,
203             RANK() OVER (
204                 PARTITION BY region
205                 ORDER BY avg_satisfaction DESC
206             ) AS satisfaction_rank
207         FROM store_satisfaction
208     )
209     SELECT *
210     FROM ranked_stores
211     WHERE satisfaction_rank <= 3
212     ORDER BY region, satisfaction_rank;
213 """)
214
215     return render_template(
216         "dashboard2.html",

```

```

212         ORDER BY region, satisfaction_rank;
213         """)
214
215     return render_template(
216         "dashboard2.html",
217         least_popular_drink=least_popular_drink,
218         orders_by_channel=orders_by_channel,
219         revenue_by_location=revenue_by_location,
220         food_vs_spend=food_vs_spend,
221         fulfillment_by_region=fulfillment_by_region,
222         top_month_year=top_month_year,
223         top_stores=top_stores,
224         customer_segments=customer_segments,
225         top_satisfaction_stores=top_satisfaction_stores
226     )
227
228 if __name__ == "__main__":
229     app.run(debug=True)

```

```

Welcome  dashboard2.html 9-  db.py
Final_Project > templates >  dashboard2.html >  html
2 <html>
42 <body>
74 <div>
76 <div class="chart-box">
77 <h2>Top 10 Above-Average Revenue Stores</h2>
78 <canvas id="topStoresChart"></canvas>
79 </div>
80
81 <div class="chart-box">
82 <h2>Top Customer Spendings</h2>
83 <canvas id="customerSegmentsChart"></canvas>
84 </div>
85
86 <div class="chart-box">
87 <h2>Top Satisfaction Stores by Region</h2>
88 <canvas id="satisfactionStoresChart"></canvas>
89 </div>
90
91 <script>
92 const greenGradient10 = [
93   '#004D2E', '#006241', '#00754A', '#008A55', '#00A862',
94   '#33B77C', '#66C696', '#8FD6A3', '#B4D8C6', '#D4E9E2'
95 ];
96
97 const starbucksColors = [
98   '#006241', '#00A862', '#CBA258', '#1E3932', '#8FD6A3',
99   '#4E79A7', '#59A14F', '#F28E2B', '#768702'
100 ];
101
102 // 1. Least Popular Drink
103 const drinkLabels = ({ least_popular_drink | map(attribute:'drink_category_name') | list | toJson });
104 const drinkOrders = ({ least_popular_drink | map(attribute:'total_orders') | list | toJson });
105
106 new Chart(document.getElementById('leastPopularDrinkChart'), {
107   type: 'bar',
108   data: {
109     labels: drinkLabels,
110     datasets: [{
111       label: 'Total Orders',
112       data: drinkOrders.map(Numbers),
113       backgroundColor: ['#CBA258'],
114       borderColor: ['#1E3932'],
115       borderWidth: 1
116     }]
117   },
118   options: {
119     responsive: true,
120     scales: { y: { beginAtZero: true } }
121   }
122 });
123
124 // 2. Orders by Channel
125 const channelLabels = ({ orders_by_channel | map(attribute:'channel_name') | list | toJson });
126 const channelOrders = ({ orders_by_channel | map(attribute:'total_orders') | list | toJson });
127
128 new Chart(document.getElementById('ordersByChannelChart'), {
129   type: 'pie',
130   data: {
131     labels: channelLabels,
132     datasets: [{
133       label: 'Total Orders',
134       data: channelOrders.map(Numbers),
135       backgroundColor: ['#006241', '#00A862', '#CBA258', '#8FD6A3'],
136       borderColor: ['#ffffff'],
137       borderWidth: 2
138     }]
139   },
140   options: {
141     responsive: true
142   }
143 });
144

```

10.3.3 Dashboard HTML (Chart.js visualizations)

10.3.1 Sanjana Vivek Phalke

```

Welcome | app.py | dashboard2.html 9x | db.py |
Final_Project > templates > dashboard2.html > html
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>Starbucks Final Dashboard</title>
6 <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
7
8 <style>
9     body {
10         font-family: Arial, sans-serif;
11         background: linear-gradient(135deg, #f1f8f4, #e8f3ee);
12         margin: 30px;
13     }
14
15     h1 {
16         text-align: center;
17         color: #006241;
18         margin-bottom: 35px;
19         font-size: 36px;
20     }
21
22     .chart-box {
23         background: white;
24         padding: 25px;
25         margin: 30px auto;
26         width: 90%;
27         border-radius: 18px;
28         box-shadow: 0 6px 18px rgba(0,0,0,0.14);
29     }
30
31     h2 {
32         color: #1e3932;
33         margin-bottom: 20px;
34     }
35
36     canvas {
37         max-height: 430px;
38     }
39 </style>
40 </head>
41
42 <body>
43
44 <h1>Starbucks Customer Ordering Dashboard</h1>
45
46 <div class="chart-box">
47     <h2>Least Popular Drink Category</h2>
48     <canvas id="leastPopularDrinkChart"></canvas>
49 </div>
50
51 <div class="chart-box">
52     <h2>Orders by Channel</h2>
53     <canvas id="ordersByChannelChart"></canvas>
54 </div>
55
56 <div class="chart-box">
57     <h2>Revenue by Store Location Type</h2>
58     <canvas id="revenueByLocationChart"></canvas>
59 </div>
60
61 <div class="chart-box">
62     <h2>Food vs Non-Food Average Spend</h2>
63     <canvas id="foodVsSpendChart"></canvas>
64 </div>
65
66 <div class="chart-box">
67     <h2>Average Fulfillment Time by Region</h2>
68     <canvas id="fulfillmentByRegionChart"></canvas>
69 </div>
70
71 <div class="chart-box">
72     <h2>Top-Selling Month per Year</h2>
73     <canvas id="topMonthYearChart"></canvas>
74 </div>

```

```

Welcome  app.py  dashboard2.html  db.py
Final_Project > templates > dashboard2.html > @html
2 <html>
42 <body>
91 <script>

144
145 // 3. Revenue by Store Location Type
146 const locationLabels = ({ top_month_year | map(attribute='store_location_type') | list | tojson });
147 const locationRevenue = ({ revenue_by_location | map(attribute='total_revenue') | list | tojson });
148
149 new Chart(document.getElementById('revenueByLocationChart'), {
150   type: 'bar',
151   data: {
152     labels: locationLabels,
153     datasets: [{
154       label: 'Total Revenue',
155       data: locationRevenue.map(Number),
156       backgroundColor: ['#006241', '#00A862', '#CBA258'],
157       borderColor: '#1e3932',
158       borderWidth: 1
159     }
160   ],
161   options: {
162     responsive: true,
163     scales: { y: { beginAtZero: true } }
164   }
165 });
166
167 // 4. Food vs Non-Food Average Spend
168 const foodLabels = ({ food_vs_spend | map(attribute='food_label') | list | tojson });
169 const foodSpend = ({ food_vs_spend | map(attribute='avg_spend') | list | tojson });
170
171 new Chart(document.getElementById('foodVsSpendChart'), {
172   type: 'bar',
173   data: {
174     labels: foodLabels,
175     datasets: [{
176       label: 'Average Spend',
177       data: foodSpend.map(Number),
178       backgroundColor: ['#00754A', '#CBA258'],
179       borderColor: '#1e3932',
180       borderWidth: 1
181     }
182   ],
183   options: {
184     responsive: true,
185     scales: { y: { beginAtZero: true } }
186   }
187 });
188
189 // 5. Fulfillment Time by Region
190 const regionLabels = ({ fulfillment_by_region | map(attribute='region') | list | tojson });
191 const fulfillmentTime = ({ fulfillment_by_region | map(attribute='avg_fulfillment_time') | list | tojson });
192
193 new Chart(document.getElementById('fulfillmentByRegionChart'), {
194   type: 'bar',
195   data: {
196     labels: regionLabels,
197     datasets: [{
198       label: 'Average Fulfillment Time',
199       data: fulfillmentTime.map(Number),
200       backgroundColor: starbucksColors,
201       borderColor: '#1e3932',
202       borderWidth: 1
203     }
204   ],
205   options: {
206     responsive: true,
207     scales: { y: { beginAtZero: true } }
208   }
209 });
210
211 // 6. Top-Selling Month per Year
212 const yearLabels = ({ top_month_year | map(attribute='year') | list | tojson });
213 const monthLabels = ({ top_month_year | map(attribute='top_selling_month') | list | tojson });
214 const monthSales = ({ top_month_year | map(attribute='total_sales') | list | tojson });

```

```

Welcome  app.py  dashboard2.html  db.py
Final_Project > templates > dashboard2.html > @html
2 <html>
42 <body>
91 <script>

211 const monthLabels = ({ top_month_year | map(attribute='top_selling_month') | list | tojson });
212 const totalSales = ({ top_month_year | map(attribute='total_sales') | list | tojson });
213 const avgSpend = ({ top_month_year | map(attribute='avg_customer_spend') | list | tojson });
214 const drinkNames = ({ top_month_year | map(attribute='most_preferred_drink') | list | tojson });
215
216 new Chart(document.getElementById('topMonthlyYearChart'), {
217   type: 'bar',
218   data: {
219     labels: yearLabels.map((year, index) => year + ' - ' + monthLabels[index] + ' - ' + drinkNames[index] + ''),
220     datasets: [
221       {
222         label: 'Total Sales',
223         data: totalSales.map(Number),
224         backgroundColor: '#006241',
225         borderColor: '#1e3932',
226         borderWidth: 1
227       },
228       {
229         label: 'Average Customer Spend',
230         data: avgSpend.map(Number),
231         type: 'line',
232         borderColor: '#CBA258',
233         backgroundColor: '#CBA258',
234         tension: 0.3,
235         yAxisID: 'y1'
236       }
237     ]
238   },
239   options: {
240     responsive: true,
241     scales: {
242       y: {
243         beginAtZero: true,
244         position: 'left'
245       },
246       y1: {
247         beginAtZero: true,
248         position: 'right',
249         grid: {
250           drawOnChartArea: false
251         }
252       }
253     }
254   }
255 });
256
257 // 7. Top 10 Above-Average Revenue Stores
258 const storeLabels = ({ top_stores | map(attribute='store_id') | list | tojson });
259 const storeRevenue = ({ top_stores | map(attribute='store_revenue') | list | tojson });
260
261 new Chart(document.getElementById('topStoresChart'), {
262   type: 'bar',
263   data: {
264     labels: storeLabels,
265     datasets: [{
266       label: 'Store Revenue',
267       data: storeRevenue.map(Number),
268       backgroundColor: greenGradient10,
269       borderColor: '#1e3932',
270       borderWidth: 1
271     }
272   ],
273   options: {
274     indexAxis: 'y',
275     responsive: true,
276     scales: { x: { beginAtZero: true } }
277   }
278 });
279
280 // 8. Top Customer Spending
281 const customerLabels = ({ customer_segments | map(attribute='customer_id') | list | tojson });
282 const customerSpending = ({ customer_segments | map(attribute='total_spending') | list | tojson });
283
284 new Chart(document.getElementById('customerSegmentsChart'), {
285   type: 'bar',
286   data: {
287     labels: customerLabels,
288     datasets: [{
289       label: 'Total Spending',
290       data: customerSpending.map(Number),
291       backgroundColor: greenGradient10,
292       borderColor: '#1e3932',
293       borderWidth: 1
294     }
295   ],
296   options: {
297     indexAxis: 'y',
298     responsive: true,
299     scales: { x: { beginAtZero: true } }
300   }
301 });
302
303 // 9. Top Satisfaction Stores by Region
304 const satisfactionLabels = ({ top_satisfaction_stores | map(attribute='store_id') | list | tojson });
305 const satisfactionScores = ({ top_satisfaction_stores | map(attribute='avg_satisfaction') | list | tojson });
306
307 new Chart(document.getElementById('satisfactionStoresChart'), {
308   type: 'bar',
309   data: {
310     labels: satisfactionLabels,
311     datasets: [{
312       label: 'Average Satisfaction',
313       data: satisfactionScores.map(Number),
314       backgroundColor: greenGradient10.concat(starbucksColors),
315       borderColor: '#1e3932',
316       borderWidth: 1
317     }
318   ],
319   options: {
320     responsive: true,
321     scales: {
322       y: {
323         beginAtZero: true,
324         max: 5
325       }
326     }
327   }
328 });
329
330 </script>
331 </body>
332 </html>

```

```

Welcome  app.py  dashboard2.html  db.py
Final_Project > templates > dashboard2.html > @html
2 <html>
42 <body>
91 <script>

275   options: {
276     indexAxis: 'y',
277     responsive: true,
278     scales: { x: { beginAtZero: true } }
279   }
280 });
281
282 // 8. Top Customer Spending
283 const customerLabels = ({ customer_segments | map(attribute='customer_id') | list | tojson });
284 const customerSpending = ({ customer_segments | map(attribute='total_spending') | list | tojson });
285
286 new Chart(document.getElementById('customerSegmentsChart'), {
287   type: 'bar',
288   data: {
289     labels: customerLabels,
290     datasets: [{
291       label: 'Total Spending',
292       data: customerSpending.map(Number),
293       backgroundColor: greenGradient10,
294       borderColor: '#1e3932',
295       borderWidth: 1
296     }
297   ],
298   options: {
299     indexAxis: 'y',
300     responsive: true,
301     scales: { x: { beginAtZero: true } }
302   }
303 });
304
305 // 9. Top Satisfaction Stores by Region
306 const satisfactionLabels = ({ top_satisfaction_stores | map(attribute='store_id') | list | tojson });
307 const satisfactionScores = ({ top_satisfaction_stores | map(attribute='avg_satisfaction') | list | tojson });
308
309 new Chart(document.getElementById('satisfactionStoresChart'), {
310   type: 'bar',
311   data: {
312     labels: satisfactionLabels,
313     datasets: [{
314       label: 'Average Satisfaction',
315       data: satisfactionScores.map(Number),
316       backgroundColor: greenGradient10.concat(starbucksColors),
317       borderColor: '#1e3932',
318       borderWidth: 1
319     }
320   ],
321   options: {
322     responsive: true,
323     scales: {
324       y: {
325         beginAtZero: true,
326         max: 5
327       }
328     }
329   }
330 });
331 </script>
332 </body>
333 </html>

```

10.3.1.2 Varuna Vyomessh

Advanced SQL queries (Explain, CTE, Window Functions, Indexing)

Technique: EXPLAIN analysis before and after adding index

```
1 USE starbucks_db;
2
3 EXPLAIN SELECT
4     order_id,
5     customer_id,
6     order_date,
7     total_spent
8 FROM orders
9 WHERE order_date >= '2025-01-01';
10
11
```

75% 1:10

Form Editor Navigate: << 1/1 >>

EXPLAIN:

- > Filter: (orders.order_date >= DATE'2025-01-01') (cost=10117 rows=49825)
- > Table scan on orders (cost=10117 rows=99651)

```
12 CREATE INDEX idx_order_date ON orders(order_date);
13
14 EXPLAIN SELECT
15     order_id,
16     customer_id,
17     order_date,
18     total_spent
19 FROM orders
20 WHERE order_date >= '2025-01-01';
21
22 -- Key Finding: Adding index reduced scanned rows by 50% and changed
23 % 34:20
```

Form Editor Navigate: << 1/1 >>

EXPLAIN:

- > Filter: (orders.order_date >= DATE'2025-01-01') (cost=10117 rows=49825)
- > Table scan on orders (cost=10117 rows=99651)

Technique: Multi-table JOIN + CASE expression + EXPLAIN ANALYZE

```
58 EXPLAIN ANALYZE
59 SELECT
60     cl.channel_name,
61     CASE
62         WHEN AVG(o.total_spent) >= 20 THEN 'High Spender'
63         WHEN AVG(o.total_spent) >= 10 THEN 'Medium Spender'
64         ELSE 'Low Spender'
65     END AS spending_tier,
66     COUNT(*) AS total_orders,
67     ROUND(AVG(o.total_spent), 2) AS avg_spent,
68     ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
69 FROM orders o
70 JOIN channel_lookup cl ON o.channel_id = cl.channel_id
71 JOIN customer c ON o.customer_id = c.customer_id
72 GROUP BY cl.channel_name
73 HAVING AVG(o.total_spent) >= 10
74 ORDER BY avg_spent DESC;
75
76 -- OBSERVATION: Check actual time and rows examined in output
77 75% 25:74
```

Form Editor Navigate: << 1/1 >>

EXPLAIN:

- > Sort: avg_spent DESC (actual time=317..317 rows=4 loops=1)
- > Filter: (??? >= 10) (actual time=317..317 rows=4 loops=1)
- > Table scan on <temporary> (actual time=317..317 rows=4 loops=1)
- > Aggregate using temporary table (actual time=317..317 rows=4 loops=1)

Re-run EXPLAIN ANALYZE after indexing

```
ORDER BY avg_spent DESC;
CREATE INDEX idx_channel_id ON orders(channel_id);
78
79 EXPLAIN ANALYZE
80 SELECT
81     cl.channel_name,
82     CASE
83         WHEN AVG(o.total_spent) >= 20 THEN 'High Spender'
84         WHEN AVG(o.total_spent) >= 10 THEN 'Medium Spender'
85         ELSE 'Low Spender'
86     END AS spending_tier,
87     COUNT(*) AS total_orders,
88     ROUND(AVG(o.total_spent), 2) AS avg_spent,
89     ROUND(AVG(o.customer_satisfaction), 2) AS avg_satisfaction
90 FROM orders o
91 JOIN channel_lookup cl ON o.channel_id = cl.channel_id
92 JOIN customer c ON o.customer_id = c.customer_id
93 GROUP BY cl.channel_name
94 HAVING AVG(o.total_spent) >= 10
95 ORDER BY avg_spent DESC;
96 3% 1:75
```

Form Editor Navigate: << 1/1 >>

EXPLAIN:

- > Sort: avg_spent DESC (actual time=302..302 rows=4 loops=1)
- > Filter: (??? >= 10) (actual time=302..302 rows=4 loops=1)
- > Table scan on <temporary> (actual time=302..302 rows=4 loops=1)
- > Aggregate using temporary table (actual time=302..302 rows=4 loops=1)

Technique: RANK() window function with PARTITION BY

```

112 SELECT
113     customer_id,
114     customer_age_group,
115     total_spend,
116     spending_rank
117 FROM (
118     SELECT
119         c.customer_id,
120         c.customer_age_group,
121         ROUND(SUM(o.total_spend), 2) AS total_spend,
122         RANK() OVER (
123             PARTITION BY c.customer_age_group
124             ORDER BY SUM(o.total_spend) DESC
125         ) AS spending_rank
126     FROM orders o
127     JOIN customer c ON o.customer_id = c.customer_id
128     GROUP BY c.customer_id, c.customer_age_group
129 ) ranked_customers
130 WHERE spending_rank <= 3
131 ORDER BY customer_age_group, spending_rank;
132

```

75% 19:129

Result Grid Filter Rows: Search Export:

customer_id	customer_age_group	total_spend	spending_rank
CUST_08642	18-24	301.68	1
CUST_07618	18-24	261.93	2
CUST_05847	18-24	260.53	3
CUST_14223	25-34	292.64	1
CUST_03127	25-34	283.81	2
CUST_03835	25-34	281.27	3
CUST_13068	35-44	247.86	1
CUST_14206	35-44	245.55	2
CUST_07467	35-44	244.17	3
CUST_14116	45-54	265.65	1
CUST_05631	45-54	265.03	2
CUST_07596	45-54	260.94	3
CUST_04752	55+	253.69	1
CUST_07303	55+	234.84	2
CUST_01444	55+	223.48	3

Technique: SUM() OVER with ORDER BY (running total)

```

143 SELECT
144     order_date,
145     daily_revenue,
146     ROUND(running_total, 2) AS cumulative_revenue,
147     ROUND(running_total / total_revenue * 100, 2) AS pct_of_total
148 FROM (
149     SELECT
150         order_date,
151         SUM(total_spend) AS daily_revenue,
152         SUM(SUM(total_spend)) OVER (
153             ORDER BY order_date
154         ) AS running_total,
155         (SELECT SUM(total_spend) FROM orders) AS total_revenue
156     FROM orders
157     GROUP BY order_date
158     ORDER BY order_date
159 ) daily_stats
160 LIMIT 30;
161

```

75% 41:151

Result Grid Filter Rows: Search Export:

order_date	daily_revenue	cumulative_revenue	pct_of_total
2024-01-03	1568.26	5701.78	0.38
2024-01-04	2048.92	7750.70	0.52
2024-01-05	2078.82	9829.52	0.66
2024-01-06	2008.90	11838.42	0.80
2024-01-07	1895.73	13734.15	0.92
2024-01-08	1961.41	15695.56	1.06
2024-01-09	1761.27	17456.83	1.17
2024-01-10	1753.01	19209.84	1.29
2024-01-11	2038.84	21248.68	1.43
2024-01-12	2010.58	23259.26	1.56
2024-01-13	2467.35	25726.61	1.73
2024-01-14	1822.19	27548.79	1.85
2024-01-15	2028.74	29577.53	1.99
2024-01-16	2225.15	31802.68	2.14
2024-01-17	1725.43	33528.11	2.26
2024-01-18	2142.20	35670.31	2.40
2024-01-19	2242.40	37912.71	2.56

Technique: Multi-step CTE + PERCENT_RANK() window function

```

-- Step 1: Calculate total spend per customer
SELECT
    c.customer_id,
    c.customer_age_group,
    c.is_rewards_member,
    SUM(o.total_spend) AS total_spend,
    COUNT(*) AS order_count
FROM orders o
JOIN customer c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_age_group, c.is_rewards_member
),
age_group_stats AS (
    -- Step 2: Calculate average spend per age group
    SELECT
        customer_age_group,
        AVG(total_spend) AS avg_group_spend
    FROM customer_totals
    GROUP BY customer_age_group
)
-- Step 3: Rank customers within age group and compare to average
SELECT
    ct.customer_id,
    ct.customer_age_group,
    ct.is_rewards_member,
    ROUND(ct.total_spend, 2) AS customer_total_spend,
    ROUND(ags.avg_group_spend, 2) AS age_group_avg_spend,
    ROUND(ct.total_spend - ags.avg_group_spend, 2) AS diff_from_avg,
    ROUND(PERCENT_RANK() OVER (
        PARTITION BY ct.customer_age_group
        ORDER BY ct.total_spend DESC
    ) * 100, 2) AS percentile_rank
FROM customer_totals ct
JOIN age_group_stats ags ON ct.customer_age_group = ags.customer_age_group
WHERE ct.total_spend > ags.avg_group_spend
ORDER BY ct.customer_age_group, percentile_rank
LIMIT 20;

```


[illegible]

Flask Application 2: Customer Spending Rankings

(viz2 customer_rankings.py)

```

1  cursor.close()
2  conn.close()
3
4  # Organize data by age group
5  age_groups = {}
6  for row in results:
7      age = row['customer_age_group']
8      if age not in age_groups:
9          age_groups[age] = []
10     age_groups[age].append(row)
11
12  html = ""
13  <DOCTYPE html>
14  <html>
15  <head>
16      <title>Top Customer Rankings by Age Group</title>
17      <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
18  <style>
19      body {
20          font-family: Arial, sans-serif;
21          margin: 10px;
22          margin: 50px auto;
23          padding: 20px;
24          background-color: #f5f5f5;
25      }
26      h1 {
27          color: #007744;
28          text-align: center;
29          margin-bottom: 30px;
30      }
31      .sub-title {
32          text-align: center;
33          color: #0066;
34          margin-bottom: 40px;
35          font-size: 14px;
36      }
37      .age-group-section {
38          background-color: white;
39          padding: 20px;
40          margin-bottom: 30px;
41          border-radius: 8px;
42          border-shadow: 0 2px 4px rgba(0,0,0,0.1);
43      }
44      .age-group-title {
45          font-size: 20px;
46          margin-left: 10px;
47          color: #007744;
48          margin-bottom: 20px;
49      }
50      border-bottom: 1px solid #007744;

```

[illegible]


```
-- =====
-- Question 2
-- =====
```

```
Select c.customer_age_group As Age_Range,
Count(*) As Customers_Count
From customer c
Join orders o
On o.customer_id=c.customer_id
Where o.total_spend>10
Group By c.customer_age_group
Order By Age_Range;
```

3 • Select c.customer_age_group As Age_Range,
4 Count(*) As Customers_Count
5 From customer c
6 Join orders o
7 On o.customer_id=c.customer_id
8 Where o.total_spend>10
9 Group By c.customer_age_group
10 Order By Age_Range;
11

Age_Range	Customers_Count
18-24	16940
25-34	24841
35-44	19331
45-54	12122
55+	7306

```
-- =====
-- Question 3
-- =====
```

```
Select c.customer_id,Count(*) As Order_Day_count,
o.day_of_week
From customer c
Join orders o
On c.customer_id=o.customer_id
Group By c.customer_id,o.day_of_week;
```

2
3 • Select c.customer_id,Count(*) As Order_Day_count,
4 o.day_of_week
5 From customer c
6 Join orders o
7 On c.customer_id=o.customer_id
8 Group By c.customer_id,o.day_of_week;
9
10
11

customer_id	Order_Day_count	day_of_week
CUST_00001	2	Mon
CUST_00001	2	Sat
CUST_00001	3	Sun
CUST_00001	3	Thu
CUST_00001	2	Tue
CUST_00002	1	Mon
CUST_00002	3	Sat
CUST_00002	4	Thu
CUST_00002	1	Tue
CUST_00002	4	Wed
CUST_00003	1	Fri
CUST_00003	3	Tue
CUST_00003	1	Wed
CUST_00004	1	Fri

6 Advanced Queries

Priya Nair

```
-- Question 1
```

```
WITH customer_orders As (
Select
c.customer_id,
c.is_rewards_member,
o.order_id,
cd.cart_size,
cd.num_customizations,
c1.channel_name
From customer c
Left Join orders o
On c.customer_id=o.customer_id
Left Join cart_details cd
On o.order_id=cd.order_id
Left Join channel_lookup c1
On o.channel_id=c1.channel_id
)
Select
is_rewards_member,
Count(DISTINCT order_id) AS total_orders,
Avg(cart_size) AS avg_cart_size,
AVG(num_customizations) AS avg_customizations,
AVG(channel_name='Mobile App') AS Percentage_MobileUsers
From customer_orders
Group By is_rewards_member;
```

```
1 WITH customer_orders As (
2 Select
3 c.customer_id,
4 c.is_rewards_member,
5 o.order_id,
6 cd.cart_size,
7 cd.num_customizations,
8 c1.channel_name
9 From customer c
10 Left Join orders o
11 On c.customer_id=o.customer_id
12 Left Join cart_details cd
13 On o.order_id=cd.order_id
14 Left Join channel_lookup c1
15 On o.channel_id=c1.channel_id
16 )
17 Select
18 is_rewards_member,
19 Count(DISTINCT order_id) AS total_orders,
20 Avg(cart_size) AS avg_cart_size,
21 AVG(num_customizations) AS avg_customizations,
22 AVG(channel_name='Mobile App') AS Percentage_MobileUsers
```

is_rewards_member	total_orders	avg_cart_size	avg_customizations	Percentage_MobileUsers
False	2368	3.6351	1.6334	0.2555
True	97632	3.7441	1.8151	0.4293

```
-- Question 2
```

```
> WITH customer_orders AS (
SELECT
c.customer_id,
c.is_rewards_member,
cd.cart_size
FROM customer c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
LEFT JOIN cart_details cd
ON o.order_id = cd.order_id
),
> distribution_spending AS (
SELECT
is_rewards_member,
ROUND(AVG(cart_size), 2) AS avg_spend,
ROUND(STDDEV(cart_size), 2) AS std_dev_spend,
ROUND(STDDEV(cart_size) / AVG(cart_size), 2) AS coefficient_of_variation
FROM customer_orders
GROUP BY is_rewards_member
)
SELECT *
FROM distribution_spending;
```



```
-- =====
-- Step 2: Index Creation
-- =====
```

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
```

```
CREATE INDEX idx_details_order
ON cart_details(order_id);
```

```
CREATE INDEX idx_cart_size
ON cart_details(cart_size);
```

```
CREATE INDEX idx_member_status
ON customer(is_rewards_member);
```

```
-- Step 4: RUNNING EXPLAIN ANALYZE WITH INDEX
```

```
-- =====
```

```
EXPLAIN ANALYZE
WITH customer_orders AS (
  SELECT
    c.customer_id,
    c.is_rewards_member,
    cd.cart_size
  FROM customer c
  LEFT JOIN orders o
    ON c.customer_id = o.customer_id
  LEFT JOIN cart_details cd
    ON o.order_id = cd.order_id
),

distribution_spending AS (
  SELECT
    is_rewards_member,
    ROUND(AVG(cart_size), 2) AS avg_spend,
    ROUND(STDDEV(cart_size), 2) AS std_dev_spend,
    ROUND(STDDEV(cart_size) / AVG(cart_size), 2) AS coefficient_of_variation
  FROM customer_orders
  GROUP BY is_rewards_member
)
```

```
SELECT *
FROM distribution_spending;
```

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);
CREATE INDEX idx_details_order
ON cart_details(order_id);
CREATE INDEX idx_cart_size
ON cart_details(cart_size);
CREATE INDEX idx_member_status
ON customer(is_rewards_member);
```

```
EXPLAIN ANALYZE
WITH customer_orders AS (
  SELECT
    c.customer_id,
    c.is_rewards_member,
    cd.cart_size
  FROM customer c
  LEFT JOIN orders o
    ON c.customer_id = o.customer_id
  LEFT JOIN cart_details cd
    ON o.order_id = cd.order_id
```

for | Navigate: ⏮ ⏪ ⏩ ⏭

```
--> Table scan on distribution_spending (cost=72014.72015 rows=2) (actual time=1711.1711 rows=2 loops=1)
--> Materialize CTE distribution_spending (cost=72013.72013 rows=2) (actual time=1711.1711 rows=2 loops=1)
--> Group aggregate: avg(cd.cart_size), std(cd.cart_size), std(cd.cart_size), avg(cd.cart_size) (cost=72012 rows=2) (actual time=37.1711 rows=2 loops=1)
--> Nested loop left join (cost=49320 rows=97616) (actual time=0.253...1563 rows=10000 loops=1)
```

```
-- Question 3
```

```
Select
c.customer_id,
c.is_rewards_member,
CASE
  When c.is_rewards_member=1 Then 'Loyal Member'
  When Count(o.order_id)>5 And c.is_rewards_member=0 Then 'High Value Rewards Member'
  Else 'Casual Non-Member'
End AS loyalty_division
From customer c
Left Join orders o ON c.customer_id=o.customer_id
Group By c.customer_id;
```

```
1
2 • Select
3 c.customer_id,
4 c.is_rewards_member,
5 CASE
6   When c.is_rewards_member=1 Then 'Loyal Member'
7   When Count(o.order_id)>5 And c.is_rewards_member=0 Then 'High Value Rewards Member'
8   Else 'Casual Non-Member'
9 End AS loyalty_division
10 From customer c
11 Left Join orders o ON c.customer_id=o.customer_id
12 Group By c.customer_id;
```

Result Grid | Filter Rows: | Exports | Wrap Cell Content: |

customer_id	is_rewards_member	loyalty_division
CUST_00001	True	High Value Rewards Member
CUST_00002	True	High Value Rewards Member
CUST_00003	True	Casual Non-Member
CUST_00004	True	High Value Rewards Member
CUST_00005	False	Casual Non-Member

Result 10

```
-- =====
-- Question 4
-- =====
```

```
Select customer_id,
order_id,
order_date
From orders
Where order_id IN (
Select Min(order_id)
From orders
Group By customer_id
);
```

```
2 Select customer_id,
3 order_id,
4 order_date
5 From orders
6 Where order_id IN (
7 Select Min(order_id)
8 From orders
9 Group By customer_id
10 );
11
```

Result Grid				Filter Rows:	Ed
	customer_id	order_id	order_date		
▶	CUST_12974	ORD_00000001	2024-03-25		
	CUST_08235	ORD_00000002	2025-07-18		
	CUST_00393	ORD_00000003	2025-01-15		
	CUST_06936	ORD_00000004	2024-07-30		
	CUST_09800	ORD_00000005	2024-06-18		

```
-- =====
-- Question 5
-- =====
```

```
Select
c.customer_id,
o.total_spend,
ROW_Number() Over(Partition By o.customer_id Order By o.total_spend DESC) as customer_ranking
From orders o
Join customer c On o.customer_id=c.customer_id;
```



```

1 • Select
2 c.customer_id,
3 o.total_spend,
4 ROW_Number() Over(Partition By o.customer_id Order By o.total_spend DESC) as customer_ranking
5 From orders o
6 Join customer c On o.customer_id=c.customer_id;
7
8

```

customer_id	total_spend	customer_ranking
CUST_00001	22.85	1
CUST_00001	19.89	2
CUST_00001	18.27	3
CUST_00001	18.08	4
CUST_00001	16.42	5

```
-- =====
```

```
-- Question 6
```

```
-- =====
```

```

Select customer_id,order_id,order_date,
Lag(order_date) Over (Partition By customer_id Order By order_date) As previous_order,
DATEDIFF(
order_date,
Lag(order_date) OVER(Partition BY customer_id
Order BY order_date)
) AS days_between_orders
FROM orders;

```

```

1
2 • Select customer_id,order_id,order_date,
3 Lag(order_date) Over (Partition By customer_id Order By order_date) As previous_order,
4 DATEDIFF(
5 order_date,
6 Lag(order_date) OVER(Partition BY customer_id
7 Order BY order_date)
8 ) AS days_between_orders
9 FROM orders;

```

customer_id	order_id	order_date	previous_order	days_between_orders
CUST_00001	ORD_00018350	2024-03-03		
CUST_00001	ORD_00024850	2024-06-11	2024-03-03	100
CUST_00001	ORD_00063789	2024-07-20	2024-06-11	39
CUST_00001	ORD_00035038	2024-08-17	2024-07-20	28
CUST_00001	ORD_00019298	2024-12-19	2024-08-17	124

Result 13

Flask application code (app_final_query1.py)

Priya Nair

```

app_final_query1.py X
FINAL SQL PROJECT ADVANCED QUERY > app_final_query1.py > ...
1 from flask import Flask, render_template, jsonify
2 from db_final_query1 import query
3
4 app = Flask(__name__)
5 app.config['TEMPLATES_AUTO_RELOAD'] = True
6
7 STAT_SQL = """
8 WITH customer_orders AS (
9     SELECT c.customer_id, c.is_rewards_member, cd.cart_size
10     FROM customer c
11     LEFT JOIN orders o ON c.customer_id = o.customer_id
12     LEFT JOIN cart_details cd ON o.order_id = cd.order_id
13 ),
14 distribution_spending AS (
15     SELECT is_rewards_member,
16     ROUND(AVG(cart_size), 2) AS avg_spend,
17     ROUND(STDDEV(cart_size), 2) AS std_dev_spend
18     FROM customer_orders
19     GROUP BY is_rewards_member
20 )
21 SELECT * FROM distribution_spending;
22 """
23
24 @app.route('/')
25 def index():
26     try:
27         # --- PHASE 1: CLEAN UP ---
28         try:
29             query("ALTER TABLE orders DROP INDEX idx_cust_id_only;")
30         except:
31             pass
32
33         # --- PHASE 2: NO INDEX (SLOW) ---
34         explain_no_index = query(f"EXPLAIN {STAT_SQL}")
35         analyze_no_index = query(f"EXPLAIN ANALYZE {STAT_SQL}")
36
37         # --- PHASE 3: WITH INDEX (FAST) ---
38         query("CREATE INDEX idx_cust_id_only ON orders(customer_id);")
39         analyze_with_index = query(f"EXPLAIN ANALYZE {STAT_SQL}")

```

Priya Nair

```
# --- PHASE 3: WITH INDEX (FAST) ---
query("CREATE INDEX idx_cust_id_only ON orders(customer_id);")
analyze_with_index = query(f"EXPLAIN ANALYZE {STAT_SQL}")

# --- PHASE 4: DATA FOR CHART ---
chart_data = query(STAT_SQL)

# CRITICAL: Return the template with all variables
return render_template('final_query1.html',
                        explain_no_index=explain_no_index,
                        no_index_results=analyze_no_index,
                        with_index_results=analyze_with_index,
                        chart_data=chart_data)

except Exception as e:
    return f"Database Error: {e}"

if __name__ == "__main__":
    app.run(debug=True)
```

Flask application code (app_final_query2.py)

```
app_final_query2.py X
FINAL SQL PROJECT ADVANCED QUERY > app_final_query2.py > index
1 from flask import Flask, render_template
2 from db_final_query2 import query # Importing your existing helper function
3
4 app = Flask(__name__)
5 app.config['TEMPLATES_AUTO_RELOAD'] = True
6
7 @app.route('/')
8 def index():
9     # SQL query to analyze spending habits and consistency
10    # This calculates average spend and standard deviation (volatility) per group
11    sql_query = """
12    Select customer_id,order_id,order_date,
13    Lag(order_date) Over (Partition By customer_id Order By order_date) As previous_order,
14    DATEDIFF(
15    order_date,
16    Lag(order_date) OVER(Partition BY customer_id
17    Order BY order_date)
18    ) AS days_between_orders
19    FROM orders
20    """
21
22    # Execute the analysis using your db.py helper
23    chart_data = query(sql_query)
24
25    # Pass the data to your template for visualization
26    return render_template('final_query2.html', chart_data=chart_data)
27
28 if __name__ == '__main__':
29     app.run(debug=True)
```

Dashboard HTML (Chart.js visualizations)

final_quer1.html

```
final_query1.html X
FINAL SQL PROJECT ADVANCED QUERY > final_query1.html > final
1 <!--VUE HTML-->
2 <vue lang="w" -->
3 </vue>
4 <!--Meta charset="utf-8"-->
5 <!--title Starbucks: Database Optimization & Customer Insights</title-->
6 <script src="https://cdn.jsdelivr.net/npm/chart.js@3.9.1/dist/chart.min.js"></script>
7 <style>
8 <!--High-contrast Color Palette V2-->
9 <root>
10 <!--Primary Colors-->
11 <--sb-black: #000000;
12 <--sb-white: #FFFFFF;
13 <--sb-blue: #0070C0;
14 <--sb-light: #BDD7EE;
15 <--sb-dark: #003366;
16 <--sb-green: #008000;
17 <--sb-red: #CC0000;
18 <--sb-gray: #666666;
19 <--sb-light-gray: #E0E0E0;
20 </root>
21
22 <body { font-family: 'Segoe UI', Tahoma, sans-serif; margin: 0px; padding: 0px; background-color: var(--sb-light); color: var(--sb-black); line-height: 1.4; }
23 <!--Container-->
24 <div { width: 100%; height: 100%; background: var(--sb-white); padding: 10px; border-radius: 10px; box-shadow: 0 10px 30px #000000; }
25
26 <h1 { color: var(--sb-green); border-bottom: 2px solid var(--sb-green); padding-bottom: 10px; margin-bottom: 10px; font-weight: bold; }
27 <h2 { color: var(--sb-black); font-size: 1.2em; margin-bottom: 10px; border-left: 3px solid var(--sb-green); padding-left: 10px; }
28
29 <div { border: 1px solid #000000; padding: 10px; border-radius: 10px; background: #000000; margin-bottom: 10px; }
30
31 <!--Table Header with Improved Contrast-->
32 <table { width: 100%; border-collapse: collapse; margin-top: 10px; background: #000000; }
33 <thead { background: #000000; text-align: left; padding: 10px; font-size: 0.9em; color: #000000; border-bottom: 2px solid var(--sb-green); text-transform: uppercase; }
34 <tbody { background: #000000; padding: 10px; font-size: 0.8em; vertical-align: top; }
35
36 <!--Status Indicators-->
37 <div { font-weight: bold; font-size: 1.1em; }
38 <div { color: var(--sb-green); }
39 <div { color: var(--sb-red); }
40 <div { color: var(--sb-gray); }
41 </div>
```

```
1 <!--Container-->
2 <div { width: 100%; height: 100%; background: var(--sb-white); padding: 10px; border-radius: 10px; box-shadow: 0 10px 30px #000000; }
3
4 <h1 { color: var(--sb-green); border-bottom: 2px solid var(--sb-green); padding-bottom: 10px; margin-bottom: 10px; font-weight: bold; }
5 <h2 { color: var(--sb-black); font-size: 1.2em; margin-bottom: 10px; border-left: 3px solid var(--sb-green); padding-left: 10px; }
6
7 <div { border: 1px solid #000000; padding: 10px; border-radius: 10px; background: #000000; margin-bottom: 10px; }
8
9 <!--Table Header with Improved Contrast-->
10 <table { width: 100%; border-collapse: collapse; margin-top: 10px; background: #000000; }
11 <thead { background: #000000; text-align: left; padding: 10px; font-size: 0.9em; color: #000000; border-bottom: 2px solid var(--sb-green); text-transform: uppercase; }
12 <tbody { background: #000000; padding: 10px; font-size: 0.8em; vertical-align: top; }
13
14 <!--Status Indicators-->
15 <div { font-weight: bold; font-size: 1.1em; }
16 <div { color: var(--sb-green); }
17 <div { color: var(--sb-red); }
18 <div { color: var(--sb-gray); }
19 </div>
```

```
1 <!--Container-->
2 <div { width: 100%; height: 100%; background: var(--sb-white); padding: 10px; border-radius: 10px; box-shadow: 0 10px 30px #000000; }
3
4 <h1 { color: var(--sb-green); border-bottom: 2px solid var(--sb-green); padding-bottom: 10px; margin-bottom: 10px; font-weight: bold; }
5 <h2 { color: var(--sb-black); font-size: 1.2em; margin-bottom: 10px; border-left: 3px solid var(--sb-green); padding-left: 10px; }
6
7 <div { border: 1px solid #000000; padding: 10px; border-radius: 10px; background: #000000; margin-bottom: 10px; }
8
9 <!--Table Header with Improved Contrast-->
10 <table { width: 100%; border-collapse: collapse; margin-top: 10px; background: #000000; }
11 <thead { background: #000000; text-align: left; padding: 10px; font-size: 0.9em; color: #000000; border-bottom: 2px solid var(--sb-green); text-transform: uppercase; }
12 <tbody { background: #000000; padding: 10px; font-size: 0.8em; vertical-align: top; }
13
14 <!--Status Indicators-->
15 <div { font-weight: bold; font-size: 1.1em; }
16 <div { color: var(--sb-green); }
17 <div { color: var(--sb-red); }
18 <div { color: var(--sb-gray); }
19 </div>
```

```
1 <!--Container-->
2 <div { width: 100%; height: 100%; background: var(--sb-white); padding: 10px; border-radius: 10px; box-shadow: 0 10px 30px #000000; }
3
4 <h1 { color: var(--sb-green); border-bottom: 2px solid var(--sb-green); padding-bottom: 10px; margin-bottom: 10px; font-weight: bold; }
5 <h2 { color: var(--sb-black); font-size: 1.2em; margin-bottom: 10px; border-left: 3px solid var(--sb-green); padding-left: 10px; }
6
7 <div { border: 1px solid #000000; padding: 10px; border-radius: 10px; background: #000000; margin-bottom: 10px; }
8
9 <!--Table Header with Improved Contrast-->
10 <table { width: 100%; border-collapse: collapse; margin-top: 10px; background: #000000; }
11 <thead { background: #000000; text-align: left; padding: 10px; font-size: 0.9em; color: #000000; border-bottom: 2px solid var(--sb-green); text-transform: uppercase; }
12 <tbody { background: #000000; padding: 10px; font-size: 0.8em; vertical-align: top; }
13
14 <!--Status Indicators-->
15 <div { font-weight: bold; font-size: 1.1em; }
16 <div { color: var(--sb-green); }
17 <div { color: var(--sb-red); }
18 <div { color: var(--sb-gray); }
19 </div>
```

```

141         backgroundColor: "#00704A",
142         borderRadius: 5
143     },
144     {
145         label: 'Volatility (Std Dev)',
146         data: [1.68, 1.71],
147         backgroundColor: "#27251F",
148         borderRadius: 5
149     }
150 ],
151 options: {
152     responsive: true,
153     scales: {
154         y: { beginAtZero: true, title: { display: true, text: 'USD ($)', font: { weight: 'bold' } } }
155     },
156     plugins: {
157         legend: { labels: { font: { weight: 'bold' } } }
158     }
159 }
160 </script>
161 </body>
162 </html>

```

Dashboard HTML (Chart.js visualizations)

final_quer2.html

Priya Nair

```

FINAL SQL PROJECT ADVANCED QUERY > templates > final_query2.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Starbucks: Purchase Frequency Analysis</title>
7   <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
8   <style>
9     :root {
10       --sb-green: #00704A;
11       --sb-black: #27251F;
12       --bg-light: #F4F7F6;
13     }
14
15     body {
16       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
17       margin: 0;
18       padding: 40px;
19       background-color: var(--bg-light);
20       color: #2d3436;
21     }
22
23     .container {
24       max-width: 1100px;
25       margin: auto;
26       background: white;
27       padding: 40px;
28       border-radius: 15px;
29       box-shadow: 0 10px 30px rgba(0,0,0,0.1);
30     }
31
32     h1 {
33       color: var(--sb-green);
34       border-bottom: 4px solid var(--sb-green);
35       padding-bottom: 10px;
36       margin-bottom: 30px;
37     }
38

```

```

<style>
.chart-section {
  margin-bottom: 40px;
  height: 400px;
}

.insight-card {
  background: #e6fcf5;
  padding: 25px;
  border-radius: 12px;
  border-left: 6px solid var(--sb-green);
  margin-top: 20px;
}

.insight-card h2 {
  margin-top: 0;
  color: var(--sb-black);
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
}

th {
  background: #f8f9fa;
  text-align: left;
  padding: 12px;
  border-bottom: 2px solid var(--sb-green);
}

td {
  padding: 12px;
  border-bottom: 1px solid #eee;
}

```

```

</style>
</head>
<body>

<div class="container">
  <h1>Customer Retention: Days Between Orders</h1>

  <div class="chart-section">
    <canvas id="frequencyChart"></canvas>
  </div>

  <div class="insight-card">
    <h2>Analysis Description</h2>
    <p>This report tracks the <strong>Purchase Velocity</strong> of customers using window functions. By calculating the difference between
    <ul>
      <li><strong>LAG function</strong>: Used to retrieve the date of the customer's previous transaction without multiple self-joins.</li>
      <li><strong>DATEDIFF</strong>: Calculates the exact number of days between visits to measure engagement.</li>
      <li><strong>Strategic Value</strong>: Helps Starbucks identify if customers are visiting more frequently (low days between orders).</li>
    </ul>
  </div>

  <h2>Raw Order Velocity Data</h2>
  <table>
    <thead>
      <tr>
        <th>Customer ID</th>
        <th>Order Date</th>
        <th>Previous Order</th>
        <th>Days Between</th>
      </tr>
    </thead>
    <tbody id="tableBody">
      </tbody>
    </table>
  </div>

```

```

</table>
</div>

<script>
  const chartData = { { chart_data | tojson }};

  // Filter out NULL values for the chart (first orders have no "days between")
  const filteredData = chartData.filter(row => row.days_between_orders !== null);

  // Process Table
  const tableBody = document.getElementById("tableBody");
  chartData.slice(0, 10).forEach(row => {
    const tr = document.createElement('tr');
    tr.innerHTML = `
      <td>${row.customer_id}</td>
      <td>${row.order_date}</td>
      <td>${row.previous_order || 'N/A'}</td>
      <td>${row.days_between_orders || '0'}</td>
    `;
    tableBody.appendChild(tr);
  });

  // Process Chart (Showing first 20 records for clarity)
  const labels = filteredData.slice(0, 20).map((row, index) => `Order ${index + 1}`);
  const values = filteredData.slice(0, 20).map(row => row.days_between_orders);

```

```

new Chart(document.getElementById("frequencyChart"), {
  type: 'line',
  data: {
    labels: labels,
    datasets: [{
      label: 'Days Between Purchases',
      data: values,
      borderColor: '#00704A',
      backgroundColor: 'rgba(0, 112, 74, 0.1)',
      fill: true,
      tension: 0.3
    }]
  },
  options: {
    responsive: true,
    maintainAspectRatio: false,
    scales: {
      y: { beginAtZero: true, title: { display: true, text: 'Days' } }
    }
  }
});
</script>

</body>
</html>

```

10.2 Evidence of Collaborative Work

This project was completed as a team effort, with contributions from all members in database design, SQL query development, analysis, and report writing. The report demonstrates collaborative writing through shared contributions across different sections, including data processing, SQL analysis, and visualization development

10.2 Version History & Contributions

Figure A1: GitHub Repository Commit History

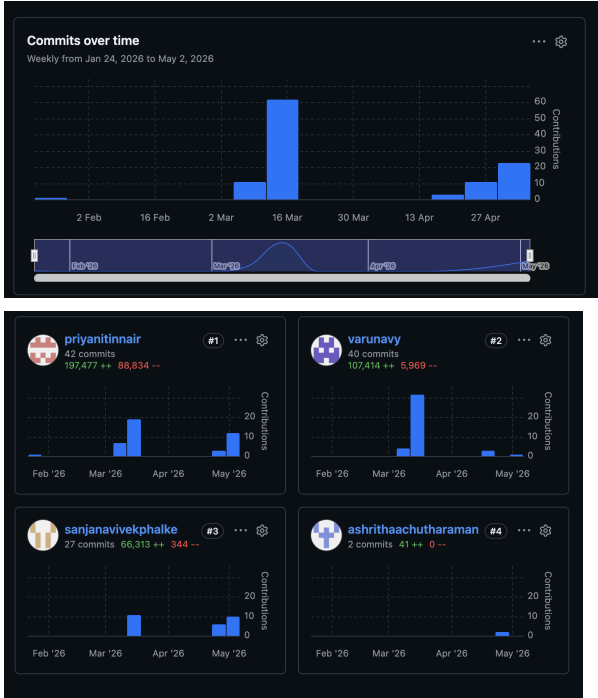


Figure A1: GitHub Repository Commit History
This screenshot shows the collaborative development timeline with contributions from all four team members (Ashritha, Priya, Sanjana, and Varuna) across database setup, SQL query development, Flask visualization implementation, and report documentation phases spanning April-May 2026.

Figure A2: Individual Contributions (Commits by Team Members)

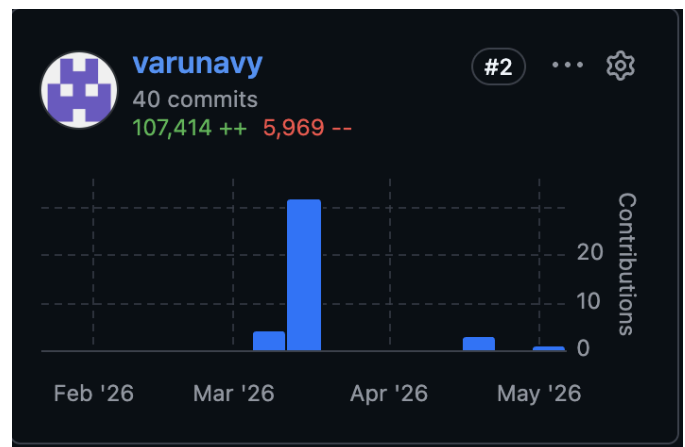
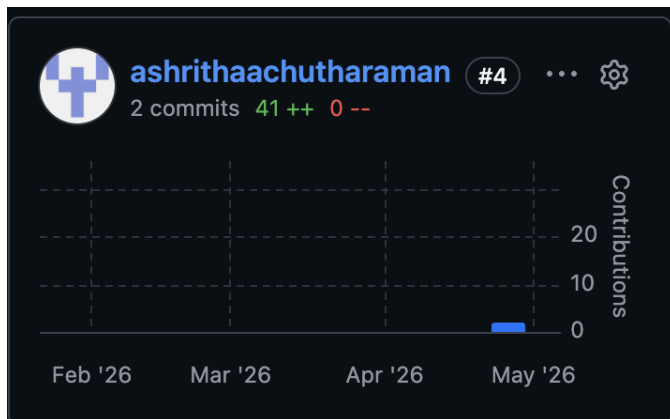
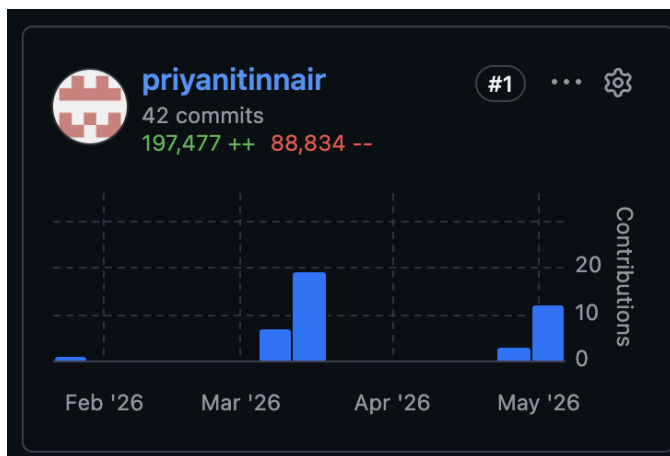
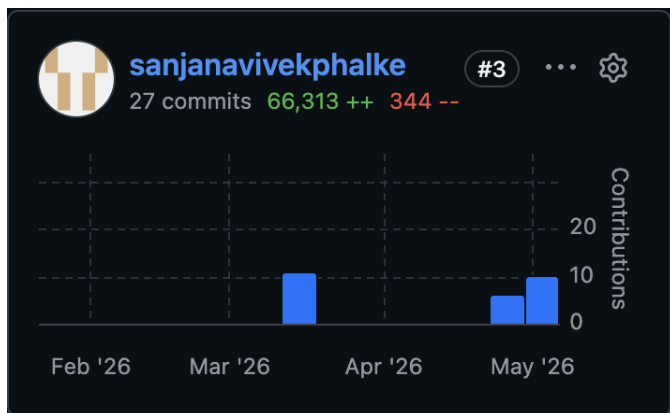


Figure A2: Individual Contributions (Commits by Team Members)



This commit timeline shows Varuna's iterative development across both midterm and final project phases. Early contributions include foundational database setup with channel_lookup and store tables, ER diagram notation work, raw data file uploads, and initial basic/advanced query development.

Final term contributions (April 28 - May 6, 2026) demonstrate progression to the Final Term Advance Query/Varuna/ folder with complete SQL query files, Flask visualization applications, and documentation. The commit history shows systematic development from database foundation through advanced analytics implementation, with clear commit messages documenting each technical milestone across the semester-long project timeline.



GITHUB LINK - [Data-201-Group-Project/Final Term Advance Query at main : priyanitinnair/Data-201-Group-Project](https://github.com/Data-201-Group-Project/Final_Term_Advance_Query_at_main_priyanitinnair/Data-201-Group-Project)